

Final Year Design Project

A Web-Based Unified School Management System for Academic and Administrative Operations
with Role Based Access



Saiha Atiq	BSCS22007
Malaika Ashraf	BSCS22015
Khadija Mustafa	BSCS22043
Zainab Khawaja	BSCS22049

Under the supervision of

Dr Salman Naseer

Bachelor of Science in Computer Science (2022-2026)

**DEPARTMENT OF INFORMATION TECHNOLOGY,
UNIVERSITY OF THE PUNJAB**

GUJRANWALA CAMPUS, GUJRANWALA.

**A Web-Based Unified School Management System for Academic and Administrative Operations
with Role Based Access**

**A project presented to
University of the Punjab, Gujranwala**

**In partial fulfilment
of the requirement for the degree of**

Bachelors of Science in Information Technology (2022-2026)

By	
Saiha Atiq	BSCS22007
Malaika Ashraf	BSCS22015
Khadija Mustafa	BSCS22043
Zainab Khawaja	BSCS22049

**DEPARTMENT OF INFORMATION TECHNOLOGY,
UNIVERSITY OF THE PUNJAB
GUJRANWALA CAMPUS, GUJRANWALA.**

DECLARATION

We hereby declare that this software, neither whole nor as a part has been copied out from any source. It is further declared that we have developed this software and accompanied report entirely on the basis of our personal efforts. If any part of this project is proved to be copied out from any source or found to be reproduction of some other, we will stand by the consequences. No portion of the work presented has been submitted for any application for any other degree or qualification of this or any other university or institute of learning.

Signature: -----

Student Name 1[Roll No]

Signature: -----

Student Name 3 [Roll No]

Signature: -----

Student Name 2 [Roll No]

Signature: -----

Student Name 4 [Roll No]

CERTIFICATE OF APPROVAL

It is to certify that the final year design project (FYDP) of BSCS “A Web-Based Unified School Management System for Academic and Administrative Operations with Role Based Access” was developed by **Saiha Atiq (BSCS22007)**, **Malaika Ashraf (BSCS22015)**, **Khadija Mustafa (BSCS22043)**, and **Zainab Khawaja (BSCS22049)** under the supervision of “Dr Salman Naseer” in my opinion; it is fully adequate, in scope and quality for the degree of Bachelors of Science in [Information Technology]

Signature: -----

FYDP Supervisor:

Signatures (Faculty Advisory Committee (FAC))

Signatures			
Name			
	FAC1	FAC2	FAC3

Signature: -----

Head of FYDP Coordination Office:

Signature:-----

Dated: _____

Chairperson, Department of [Information Technology]

Executive Summary

This Final Year Project presents the development of a **web-based School Management System (SMS)** aimed at improving the efficiency of academic and administrative processes in educational institutions. Traditional systems often rely on manual records and disconnected communication methods, leading to data inconsistencies, delays, and lack of transparency. The primary objective of this project is to provide a **centralized platform** that enables effective management of attendance, marks, student performance, announcements, and complaints while ensuring secure and structured data access for administrators, teachers, students, and parents.

The system integrates key functionalities such as attendance tracking, marks entry, digital student portfolios, and notice management into a unified solution. It also incorporates an **AI-based Risk Prediction** feature that analyzes student performance using attendance, academic results, and behavior data to identify at-risk students. This intelligent feature supports teachers and administrators in making timely, data-driven decisions through recommendations and simulation capabilities. Role-based access control ensures that each user interacts only with relevant features, enhancing both usability and security.

The development follows a modular approach using modern technologies. The frontend is built with React.js for responsive user interfaces, while the backend is developed using ASP.NET Core Web API for handling business logic and secure operations. Entity Framework Core and SQL Server are used for efficient database management. The system is designed in accordance with SRS and SDS guidelines to ensure a structured and practical implementation.

The implemented system demonstrates improved efficiency, reduced manual workload, and enhanced accessibility of information. It enables teachers to manage academic tasks effectively, allows students and parents to monitor progress in real time, and provides administrators with centralized control. Overall, the system enhances **transparency**, communication, and decision-making while introducing intelligent capabilities for improved educational management.

Acknowledgement

The acknowledgment section is an opportunity to express gratitude and appreciation to individuals and organizations who have contributed to the completion of your Final Year Project (FYP).

Signature: -----

Student Name 1[Roll No]

Signature: -----

Student Name 3 [Roll No]

Signature: -----

Student Name 2 [Roll No]

Signature: -----

Student Name 4 [Roll No]

Abbreviations

Table of all the abbreviations and acronyms used in the FYP along with their respective brief description. The abbreviation list is ordered alphabetically.

AI	Artificial Intelligence
API	Application Programming Interface
ASP.NET	Active Server Pages .NET — Microsoft web framework
BSCS	Bachelor of Science in Computer Science
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CPU	Central Processing Unit
CSS	Cascading Style Sheets
CRUD	Create, Read, Update, Delete — basic database operations
CSV	Comma-Separated Values — file format for tabular data
DB	Database
DRY	Do Not Repeat Yourself — software design principle
EF	Entity Framework — Microsoft ORM for .NET
ETL	Extract, Transform, Load — data migration process
FAC	Faculty Advisory Committee
FK	Foreign Key — relational database constraint
FR	Functional Requirement
FT	Functional Testing
FYP	Final Year Project
FYDP	Final Year Design Project
GB	Gigabyte — unit of digital storage
GET	HTTP GET method — retrieves a resource
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
ID	Identifier — unique record key
IEEE	Institute of Electrical and Electronics Engineers

IT	Integration Testing
JSON	JavaScript Object Notation — lightweight data format
JWT	JSON Web Token — standard for secure authentication
LMS	Learning Management System
MB	Megabyte — unit of digital storage
ML	Machine Learning
ML.NET	Microsoft open-source machine learning framework for .NET
MVC	Model-View-Controller — architectural design pattern
NFR	Non-Functional Requirement
OOP	Object-Oriented Programming
ORM	Object-Relational Mapper — maps objects to database tables
OS	Operating System
PATCH	HTTP PATCH method — partially updates a resource
PDF	Portable Document Format
PK	Primary Key — unique identifier in a database table
POST	HTTP POST method — submits data to a resource
PR	Pull Request — code review mechanism in Git
PT	Performance Testing
PUT	HTTP PUT method — replaces or updates a resource
RAM	Random Access Memory
RBAC	Role-Based Access Control
REST	Representational State Transfer — API architectural style
RTO	Recovery Time Objective — maximum tolerable downtime
SDK	Software Development Kit
SDCA	Stochastic Dual Coordinate Ascent — ML optimization algorithm
SDS	Software Design Specification
SMART	Specific, Measurable, Achievable, Relevant, Time-bound
SMS	School Management System
SMTP	Simple Mail Transfer Protocol

SQL	Structured Query Language
SQLEXPRESS	SQL Server Express — lightweight edition of Microsoft SQL Server
SRP	Single Responsibility Principle — software design principle
SRS	Software Requirements Specification
SSMS	SQL Server Management Studio
TLS	Transport Layer Security — encryption protocol
TTI	Time to Interactive — web performance metric
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
URL	Uniform Resource Locator — web address
UT	Unit Testing
UX	User Experience
WCAG	Web Content Accessibility Guidelines

Table of Contents

Contents

1. Introduction	18
1.1 Problem Statement	18
1.2 Problem Solution	19
1.3 Objectives of the Proposed System.....	20
1.4 Scope.....	21
1.5 System Components	22
1.5.1 Module 1: Admin Module	22
1.5.2 Module 2: Teacher Module.....	22
1.5.3 Module 3: Student Module	23
1.5.4 Module 4: Parent Module	23
1.5.5 Module 5: AI Risk Prediction Module.....	23
1.6 Related System Analysis/Literature Review	24
1.7 Vision Statement	25
1.8 System Limitations and Constraints.....	25
1.9 Tools and Technologies.....	26
1.10 Project Deliverables.....	27
1.11 Project Planning	28
1.12 Summary	29
2 Analysis	31
2.1 User classes and characteristics	31
2.2 Requirement Identifying Technique	31
2.3 Functional Requirements	32
2.3.1 User Authentication and Authorization	32
2.3.2 Attendance Management	34
2.3.3 Marks and Student Performance	34
2.3.4 Student Portfolio Generation	38
2.3.5 Noticeboard & Notifications.....	39
2.3.6 Complaint and Behavior Management	40
Fee Reminder	41

2.3.7	Audit Logs & System Monitoring	42
2.4	Non-Functional Requirements	43
2.4.1	Reliability.....	43
2.4.2	Usability	44
2.4.3	Performance	44
2.4.4	Security	45
2.5	External Interface Requirements.....	46
2.5.1	User Interfaces Requirements	46
2.5.2	Software interfaces.....	47
2.5.3	Hardware interfaces	48
2.5.4	Communications interfaces.....	49
2.6	Summary	50
3	<i>System Design</i>.....	52
3.1	Design Considerations	52
3.2	Design Models	54
3.3	Architectural Design.....	62
3.4	Data Design.....	67
3.4.1	Data Dictionary	68
3.5	User Interface Design	88
3.5.1	Screen Images	90
3.5.2	Screen Objects and Actions.....	92
3.4	Behavioural Model	93
3.5	Design Decisions.....	100
3.6	Summary	105
4	<i>Implementation</i>	107
4.1	Algorithm	107
4.2	External APIs/SDKs	113
4.3	Code Repository	115
4.3.1	Metrics of Git Repository	116
4.4	Summary	117
5	<i>Introduction</i>	119
5.1	Unit Testing (UT)	119
5.2	Functional Testing (FT).....	123

5.3	Integration Testing (IT).....	127
5.4	Performance Testing (PT).....	131
5.5	Summary	134
6	Introduction	136
6.1	Conversion Method	136
6.2	Deployment.....	137
6.2.1	Data Conversion.....	141
6.2.2	Training.....	142
6.3	Post Deployment Testing	144
6.4	Challenges.....	145
6.5	Summary	147
7	Introduction	149
7.1	Evaluation.....	149
7.2	Traceability Matrix	150
7.3	Conclusion	153
7.4	Future Work	154
	References	156
	Appendix-A Use Case Description (Fully Dressed Format)	160
	Appendix-B General Coding Standards & Guidelines.....	177
	<i>Appendix-C Application Prototype.....</i>	<i>180</i>

List of Figures

Figure 1 Project Planning gantt chart.....	28
Figure 2 Class Diagram	54
Figure 3 Sequence Diagram 1- User Flow Login	55
Figure 4 Sequence Diagram 2- Teacher Academic Management Flow	55
Figure 5 Sequence Diagram 3- Student Portfolio Generation Flow	56
Figure 6 State Diagram 1- Attendance Record	57
Figure 7 State Diagram 2- Complain Mangement.....	58
Figure 8 State Diagram 3- Student Portfolio	59
Figure 9 State Diagram 4- fee Payment.....	60
Figure 10 State Diagram 5- User account.....	61
Figure 11 System Architecture Diagram	63
Figure 12 UML Component Diagram.....	66
Figure 13 Screen Image 1- Login Screen	90
Figure 14 Screen Image 2- Student Portfolio	90
Figure 15 Screen Image 3- Attendance Management.....	91
Figure 16 Screen Image 4- Complain Feedback.....	91
Figure 17 Screen Image 5- Student Portfolio Generation.....	92
Figure 18 Sequence Diagram 1- User Flow Login	93
Figure 19 Sequence Diagram 2- Teacher Academic Management Flow	94
Figure 20 Sequence Diagram 3- Student Portfolio Generation Flow	95
Figure 21 State Diagram 1- Attendance Record	96
Figure 22 State Diagram 2- Complain Mangement.....	97
Figure 23 State Diagram 3- Student Portfolio	98
Figure 24 State Diagram 4- fee Payment.....	99
Figure 25 State Diagram 5- User account.....	100
Figure 26 Login page	180
Figure 27 Admin Dashboard.....	180
Figure 28 Attendance and marks management screen.....	181
Figure 29 Complaint management screen.....	181
Figure 30 Noticeboard system screen	182
Figure 31 AI risk prediction screen	183
Figure 32 Marks entry module screen	183
Figure 33 student portfolio screen	184

List of Tables

Table 1 Related System Analysis with proposed project solution.....	24
Table 2 Tools and Technology for proposed project.....	26
Table 3 User classes and characteristics	31
Table 4 Defines the system requirements for secure user authentication and redirection to role-specific dashboards.	32
Table 5 Specifies administrative capabilities for creating, updating, deactivating, and managing user accounts and roles.	33
Table 6 Describes the functionality that allows teachers to record daily student attendance for assigned classes.....	34
Table 7 Defines the requirements for teachers to enter and manage student marks for tests and examinations.	34
Table 8 Specifies the process for uploading and managing student achievements and certificates as part of the digital portfolio.....	35
Table 9 Describes how teachers can record and manage student behaviour remarks for academic and disciplinary tracking.....	36
Table 10 Defines automated analysis of attendance, marks, and behaviour to generate student risk indicators.....	37
Table 11 Specifies the system’s ability to compile attendance, marks, behaviour, and achievements into a unified student portfolio.....	38
Table 12 Describes the functionality for authorized staff to publish notices to students and parents.	39
Table 13 Defines the notification mechanism for informing users about notices and important system updates.	39
Table 14 Defines the process through which students and parents can submit formal complaints.	40
Table 15 Describes how teachers and administrators manage and resolve submitted complaints.	41
Table 16 Specifies the system requirements for displaying student fee status to authorized users.	41
Table 17 Defines the system’s logging requirements for tracking critical actions and ensuring accountability.....	42
Table 18 Algorithm 1: Rule-Based Student Risk Classification	107
Table 19 Algorithm 2: ML.NET SDCA Classifier — Training Phase.....	108
Table 20 Algorithm 3: ML.NET Risk Prediction — Inference Phase.....	109
Table 21 Algorithm 4: Mark Student Attendance.....	109
Table 22 Algorithm 5: Enter and Calculate Student Marks.....	110
Table 23 Algorithm 6: Complaint Submission and Status Lifecycle	111
Table 24 Algorithm 7: Teacher Dashboard Statistics Aggregation	112
Table 25 External API’s used in school management system	113
Table 26 UT01 — User Login with Valid Credentials.....	119
Table 27 UT02 — User Login with Invalid Password	120
Table 28 UT03 — Attendance Status Enum Mapping	121
Table 29 UT04 — Grade Calculation Logic — Boundary Values.....	121
Table 30 UT05 — Rule-Based Risk Score — High Risk Threshold	122
Table 31 FT01 — Mark Attendance for a Student	123

Table 32 FT02 — Prevent Duplicate Attendance Entry	124
Table 33 FT03 — Enter Marks and Verify Percentage	124
Table 34 FT04 — Submit a Complaint.....	125
Table 35 FT05 — Post a School Announcement.....	126
Table 36 FT06 — Record a Student Achievement.....	126
Table 37 IT01 — AI Risk Prediction — Full Pipeline Integration (Key Feature)	127
Table 38 IT02 — AI Risk Prediction — Fallback to Rule-Based When Model Not Loaded	128
Table 39 IT03 — ML Model Training Triggered Successfully	129
Table 40 IT04 — Attendance Record Reflected in Risk Computation	129
Table 41 IT05 — Complaint Status Update Triggers Correct Lifecycle.....	130
Table 42 PT01 — API Response Time — Single Risk Prediction.....	131
Table 43 PT02 — Dashboard Statistics Load Time	132
Table 44 PT03 — ML Model Training Time	133
Table 45 PT04 — Frontend Dashboard Render Time	133
Table 46 Deployment Overview — Components and Actions	137
Table 47 Deployment and Development Challenges with Solutions	145
Table 48 Objectives Completion Status.....	149
Table 49 Traceability Matrix	150
Table 50 Functionality implementation status.....	153
Table 51 UC-1 User Login / Authentication.....	160
Table 52 UC-2 Mark Student Attendance	161
Table 53 UC-3 Enter Student Marks	163
Table 54 UC-4 AI-Based Student Risk Prediction	165
Table 55 UC-5 Submit and Manage Complaint	168
Table 56 UC-6 Post Announcement / Notice	170
Table 57 UC-7 Record Student Achievement or Certificate	172
Table 58 UC-8 Add Behaviour Remark	174

Chapter 1

Introduction

1. Introduction

This chapter presents an overview of the Final Year Design Project (FYDP), which involves the development of a **Web-Based School Management System (SMS)** to improve academic and administrative processes in educational institutions. The system aims to replace traditional manual methods with a centralized digital platform for managing attendance, marks, student records, announcements, and complaints.

In addition to core functionalities, the system integrates an **AI-based Risk Prediction feature** that analyzes student performance using attendance, academic results, and behavior data. This feature helps identify at-risk students and provides insights, recommendations, and simulation capabilities to support better decision-making by teachers and administrators.

This chapter outlines the problem statement, stakeholders, objectives, and scope of the system, along with project goals, assumptions, and technologies used. It establishes a clear foundation for system design, development, and implementation.

1.1 Problem Statement

Schools in Pakistan still rely on paper-based records, informal communication channels, and disconnected systems for managing academic and administrative tasks. As a result, processes such as attendance tracking, result management, and communication are often inefficient, delayed, and prone to errors. Most institutions lack:

- A centralized student portfolio integrating attendance, academic performance, achievements, and behavior records.
- A secure and structured communication system to replace informal platforms.
- A unified digital noticeboard for official announcements.
- A proper complaint and behavior management system with tracking and accountability.
- Intelligent tools to analyze student performance and identify at-risk students.

Due to the absence of analytical support, teachers and administrators are unable to make timely, data-driven decisions to improve student outcomes. Therefore, there is a need for a centralized, secure, and scalable School Management System that not only digitizes school operations but also incorporates AI-based risk prediction to enhance monitoring, early intervention, and overall educational effectiveness.

1.2 Problem Solution

The Web-Based School Management System resolves existing inefficiencies through a modular, role-based platform that centralises administrative and academic functions into a single interface. Built using React.js (frontend), ASP.NET Core Web API (backend), and SQL Server (database), the system replaces manual processes with structured, accessible digital workflows for all authorised users.

Attendance management is fully digitised, enabling teachers to mark and review daily attendance efficiently while preventing duplicate records. Marks entry is streamlined through subject and exam-type filters, with automatic percentage calculation and grade assignment to minimise errors.

The system incorporates a behaviour management module where teachers record Positive, Negative, or Neutral remarks. These inputs contribute directly to the AI-based risk engine. Additionally, a structured complaint management system ensures accountability through a defined lifecycle: Submitted, Under Review, Resolved, and Closed.

A key feature is the AI-Based Student Risk Prediction module, developed using ML.NET. It analyses attendance, academic performance, and behaviour to classify students into Low, Medium, or High risk levels in real time. A rule-based fallback ensures continuous functionality when training data is limited. Each prediction includes a breakdown of contributing factors, enabling informed and proactive intervention.

Overall, the system integrates attendance, marks, behaviour, complaints, achievements, and AI-driven insights into a unified platform with role-specific dashboards for administrators, teachers, students, and parents. The following objectives define the specific, measurable outcomes the system is designed to achieve:

1. Implement an AI-based student risk prediction system with real-time multiclass classification (under 500ms response time).
2. Digitise attendance management to eliminate manual record-keeping.
3. Automate marks entry with percentage and grade calculation.
4. Provide a structured complaint management system with lifecycle tracking.
5. Deliver role-based dashboards with real-time data access.
6. Integrate behaviour tracking into the AI risk evaluation process.
7. Enable targeted announcements with classification and priority levels.
8. Support recording and viewing of student achievements and certificates.

1.3 Objectives of the Proposed System

The following objectives have been defined using the SMART framework — Specific, Measurable, Achievable, Relevant, and Time-bound — and are ordered by priority, with the AI Risk Prediction feature listed first as the primary innovation of the project.

OBJ-1 (AI Risk Prediction): To develop and deploy an ML.NET SDCA multiclass classifier that predicts student academic risk (Low, Medium, High) with a response time under 500 ms, using three measurable inputs (attendance rate, average marks, behaviour score), and provides a rule-based fallback to ensure 100% availability of risk predictions at all times.

OBJ-2 (Attendance Management): To provide a digital attendance marking interface for teachers that eliminates paper-based registers, supports single-student and bulk-class marking, and enforces duplicate prevention with HTTP 409 conflict handling, to be completed within the first development sprint.

OBJ-3 (Marks Management): To enable teachers to enter examination marks per subject and exam type, with server-side automatic percentage and grade computation (A+ through F), reducing manual calculation to zero and ensuring marks are reflected in AI risk scores within one system request.

OBJ-4 (Complaint Management): To implement a four-stage complaint lifecycle (Submitted, Under Review, Resolved, Closed) with role-based assignment, teacher-only update restriction, and automatic DateClosed stamping upon resolution, targeting 100% traceability of all submitted complaints.

OBJ-5 (Behaviour Tracking): To allow teachers to record Positive, Negative, or Neutral behaviour remarks for students, with each negative remark contributing a quantifiable decrement to the student's behaviour score used in AI risk computation, ensuring cross-module data integration.

OBJ-6 (Notice Board): To provide a notice posting module that enables teachers and admins to publish announcements with audience targeting (School-Wide, Students Only, Parents Only, Class Specific), type classification, and priority levels, with immediate activation upon posting.

OBJ-7 (Achievements Module): To enable recording of student academic and extracurricular achievements with optional certificate link storage, linked to individual student profiles and visible in student dashboards.

OBJ-8 (Role-Based Dashboards): To deliver four personalised dashboards — Admin, Teacher, Student, and Parent — each displaying role-relevant statistics aggregated in real time from the live database, with all views protected by JWT-based role authentication.

1.4 Scope

What is Included

1. Fully digitalised attendance management with duplicate prevention and date-range reporting.
2. Marks entry, automatic grade computation, and subject/exam-type filtering.
3. AI-based student risk prediction using ML.NET with rule-based fallback and real-time factor breakdown.
4. Structured complaint management with a four-stage lifecycle and role-based update restrictions.
5. Behaviour remark recording (Positive, Negative, Neutral) contributing to AI risk scores.
6. Notice board with audience targeting and priority classification.
7. Student achievement and certificate recording with optional file link.
8. Role-based dashboards for Admin, Teacher, Student, and Parent.
9. JWT-based authentication with role enforcement across all API endpoints.
10. Docker-based containerised deployment support.

What is Excluded

1. Real-time parent-teacher messaging (Communication Hub) — excluded from the current project scope.
2. Mobile application development (iOS/Android) — the system is web-only in this iteration.
3. Integration with external Learning Management Systems (LMS) such as Google Classroom or Moodle.
4. Automated email or push notification alerts — risk visibility is dashboard-based only.
5. Multi-school or multi-branch tenancy — the system supports a single school instance.
6. Timetable or scheduling management.

Stakeholders

1. Administrator: Full system access — manages users, views all modules, triggers AI model training.
2. Teacher: Marks attendance, enters marks, records behaviour remarks, posts notices, manages assigned complaints.
3. Student: Views own attendance, marks, risk status, and achievements.

4. Parent: Views their child's attendance, marks, and risk level.

System Boundaries and Constraints

1. The system operates as a web application and requires internet browser access; no offline mode is provided.
2. All API endpoints are protected by JWT authentication. Unauthenticated requests receive HTTP 401.
3. The AI model requires a minimum of 10 student records or 18 seed samples for initial training.
4. SQL Server must be installed and accessible for all database operations.

Key Milestones

1. Milestone 1 (Weeks 1–3): Requirements gathering, SRS completion, system architecture design.
2. Milestone 2 (Weeks 4–8): Database schema, backend API development, core module implementation.
3. Milestone 3 (Weeks 9–12): AI module development, ML model training pipeline, frontend integration.
4. Milestone 4 (Weeks 13–15): Testing (unit, functional, integration, performance), bug resolution.
5. Milestone 5 (Weeks 16–18): Deployment, pilot testing, documentation finalisation.

1.5 System Components

The School Management System is composed of five primary role-based modules and one cross-cutting AI prediction module. Each module is implemented as a set of React.js frontend components and corresponding ASP.NET Core controllers with dedicated database entities.

1.5.1 Module 1: Admin Module

1. Manage all user accounts (create, update, activate/deactivate) for Admin, Teacher, Student, and Parent roles.
2. View all attendance, marks, complaints, notices, and achievements across the entire institution.
3. Trigger AI model training and monitor risk distribution across all students.
4. Access the admin dashboard showing school-wide statistics and analytics.

1.5.2 Module 2: Teacher Module

1. Mark daily attendance for students with Present, Absent, or Leave status.
2. Enter examination marks per subject and exam type; system auto-computes grades.

3. Record behaviour remarks (Positive, Negative, Neutral) for individual students.
4. Post school announcements with audience targeting, type, and priority.
5. Record student achievements and certificates with optional file links.
6. View and update the status of complaints assigned to the teacher.
7. Access the teacher dashboard showing aggregated real-time statistics.

1.5.3 Module 3: Student Module

1. View personal attendance records by date range with status indicators.
2. View examination marks, computed percentages, and letter grades per subject and exam.
3. View personal academic risk level (Low, Medium, High) and contributing factors.
4. View recorded achievements and certificate links.
5. View active school notices relevant to the student audience.

1.5.4 Module 4: Parent Module

1. View child's attendance records and overall attendance percentage.
2. View child's examination marks and grade summary.
3. View child's current academic risk level with factor breakdown.
4. View active school notices targeted at parents or school-wide audiences.

1.5.5 Module 5: AI Risk Prediction Module

1. Compute student risk factors: attendance rate, average marks percentage, and behaviour score.
2. Apply rule-based classification thresholds (High: attendance < 50% or marks < 35% or negative remarks >= 5; Medium: attendance < 75% or marks < 55% or negative remarks >= 3).
3. Train ML.NET SDCA Maximum Entropy multiclass classifier on real or seed training data.
4. Produce ML-generated risk predictions (Low, Medium, High) with mlModelUsed flag.
5. Return full risk report with factor values, both rule-based and ML predictions, and final risk level.
6. Provide on-demand model retraining triggered by administrators.

1.6 Related System Analysis/Literature Review

Several existing school management systems were examined to identify gaps that the proposed SMS addresses. Three representative systems are reviewed below.

Fedena: Fedena is a widely-used open-source school management system that provides modules for attendance, timetable, library management, and basic result tracking. However, it offers no predictive analytics or AI-driven insights into student performance. Risk identification is entirely manual and depends on individual teacher discretion, with no data-driven early warning mechanism.

Classter: Classter is a commercial, cloud-based SMS designed for K-12 institutions, offering attendance, assessment, and communication tools. While it provides configurable reporting, it does not include any machine learning capability and requires staff to manually interpret performance data. There is no automated classification of students by academic risk level.

OpenSIS: OpenSIS is an open-source student information system that provides attendance and grade management but operates in a modular, siloed fashion. Data from different modules is not aggregated for cross-domain analysis. Complaint and behaviour management are absent, and there is no integration between academic and conduct data.

Table 1 Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
Fedena (Open-source SMS)	Lacks AI-driven analytics. Risk identification is entirely manual and relies on teacher discretion. No predictive capability for early student intervention.	The proposed SMS provides automated AI-based risk prediction using ML.NET, classifying students as Low, Medium, or High risk based on live attendance, marks, and behaviour data.
Classter (Commercial SMS)	Offers basic reporting but no predictive modelling. The system requires manual data interpretation and does not notify teachers	The proposed system generates real-time risk predictions automatically and provides actionable recommendations, removing the need for manual data analysis by teaching staff.

	proactively about at-risk students.	
OpenSIS (Open-source SMS)	Attendance and marks management exist but are siloed. No integration between academic performance and student wellness. Complaints are handled outside the system.	The proposed SMS integrates attendance, marks, behaviour, and complaint data into a single platform with cross-module risk scoring, providing a holistic view of each student.

1.7 Vision Statement

For school administrators, teachers, students, and parents

Who need a centralised, data-driven platform to manage school operations efficiently and identify at-risk students proactively

The Web-Based School Management System (SMS)

Is a full-stack, AI-enhanced web application

That consolidates attendance, marks, behaviour, complaints, and AI-driven risk prediction into a single integrated platform with personalised role-based dashboards

Unlike existing systems such as Fedena, Classter, and OpenSIS, which lack predictive analytics, cross-module data integration, or structured behaviour management

Our product provides automated ML.NET-powered student risk classification (Low, Medium, High) using live multi-dimensional data, enabling schools to move from reactive to proactive academic intervention

1.8 System Limitations and Constraints

Limitations (External Factors)

7. The AI model's prediction accuracy improves with the size and quality of the training dataset. With fewer than 10 real student records, the system falls back to the 18-sample seed dataset, which may not capture the full distribution of real-world student profiles.
8. The system does not support offline operation. All features require a stable internet connection and a running backend server.

9. Risk predictions are computed from attendance, marks, and behaviour data only. External factors influencing academic performance — such as family circumstances or health — are not captured by the current model.
10. The system is designed for a single school instance. Scaling to multiple schools or branches would require architectural changes not within the current project scope.

Constraints (Internal Factors)

11. The system is built exclusively using the specified technology stack (React.js, ASP.NET Core, SQL Server, ML.NET). Alternative frameworks or databases are not considered.
12. The parent-teacher Communication Hub was excluded from the project scope due to time and resource constraints within the academic timeline.
13. The project must be completed within one academic semester, constraining the depth of testing and the volume of real-world user feedback that can be collected before submission.
14. Mobile application development is out of scope. The system is accessible via web browsers only, which may limit adoption among users who primarily use smartphones.

1.9 Tools and Technologies

The following table summarises the tools and technologies used in the development of the School Management System, including version information and the rationale for each selection.

Table 2 Tools and Technology for proposed project

Tools / Technology	Version	Rationale
React.js (Vite)	18.x	A component-based JavaScript library enabling fast, modular UI development. Vite provides a rapid development server and optimised production builds.
ASP.NET Core Web API	8.0	A high-performance, cross-platform framework for building RESTful APIs. Provides built-in JWT authentication, dependency injection, and middleware support.
SQL Server (SQLEXPRESS)	2019+	A robust relational database management system. Chosen for strong integration with Entity Framework Core and reliable support for complex school data relationships.

Entity Framework Core	8.0	An ORM that maps C# models to database tables, enabling type-safe database operations without writing raw SQL. Supports code-first migrations for schema management.
ML.NET	3.0	Microsoft's open-source machine learning framework for .NET. Enables training and deploying the SDCA Maximum Entropy multiclass classifier for student risk prediction within the existing backend.
Tailwind CSS	3.x	A utility-first CSS framework that enables consistent, responsive UI design without writing custom CSS stylesheets.
Git / GitHub	—	Version control and collaborative code management. Feature-branch workflow with pull request reviews ensures code quality throughout development.

1.10 Project Deliverables

Software Requirements Specification (SRS): A complete documentation of all functional and non-functional requirements, use cases, and system constraints.

Software Design Specification (SDS): Architectural diagrams, database schema, API contract documentation, and module design descriptions.

Prototype & Frontend Repository: After SDS, a UI prototype was developed using React.js and the frontend GitHub repository URL was shared with the supervisor for review and feedback.

Source Code Repository: Full source code for the React.js frontend, ASP.NET Core backend, ML.NET AI module, and database migration scripts, managed on GitHub (developed after prototype approval).

Trained AI Model File: The exported risk_model.zip file containing the trained ML.NET SDCA multiclass classifier.

Testing & Validation: System testing including functional verification, performance evaluation, and validation of AI predictions.

Final FYP Report: Complete academic report comprising all chapters (Introduction through Conclusion and Appendices).

Project Presentation: Live demonstration of the developed and tested system for the final examination panel.

1.11 Project Planning

The project was planned and executed across five structured phases spanning eighteen weeks. The Gantt chart below presents the timeline visually, followed by a description of each phase and its key milestone.

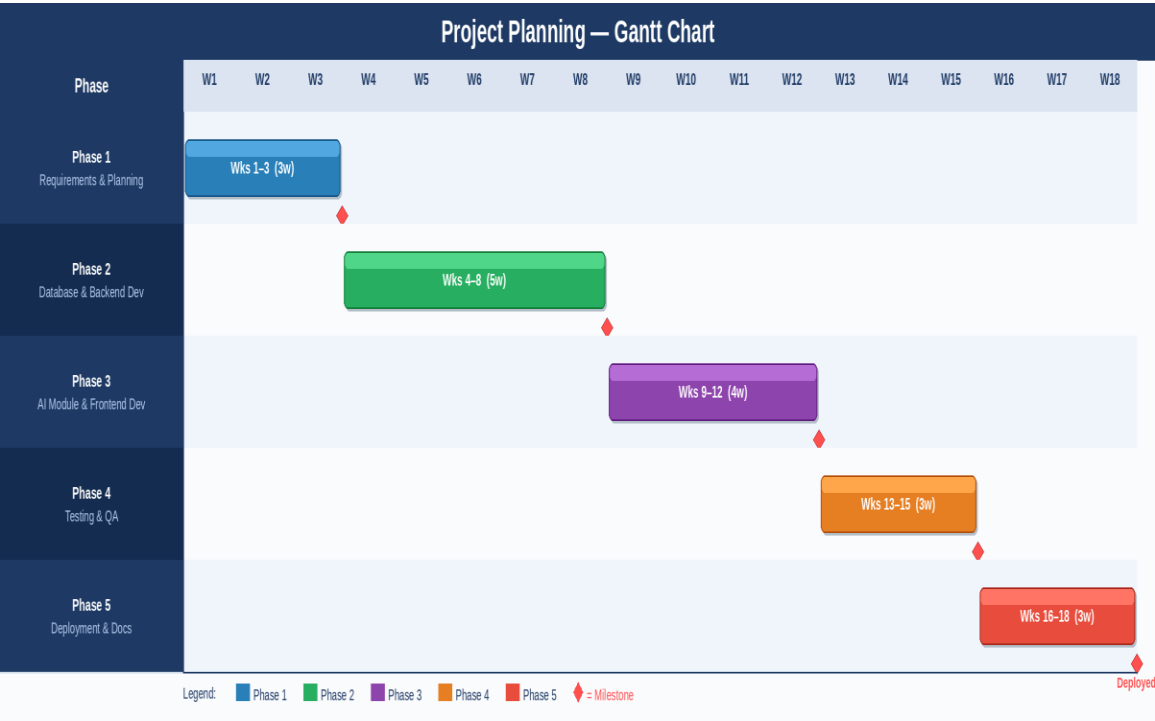


Figure 1 Project Planning gantt chart

Figure 1-1 Project Gantt Chart — 18-Week Development Timeline

Phase 1 — Requirements & Planning (Weeks 1–3): This phase involved identifying stakeholders, conducting requirement-gathering sessions, and producing the Software Requirements Specification (SRS). The system architecture was designed, the technology stack was confirmed, and initial wireframes were prepared. **Milestone:** SRS and architecture diagram completed.

Phase 2 — Database & Backend Development (Weeks 4–8): The SQL Server database schema was implemented using Entity Framework Core code-first migrations. All backend controllers were developed: Authentication, Attendance, Marks, Complaint, Notice, Achievement, Behaviour Remark, and Teacher Dashboard. All endpoints were tested via Swagger UI. **Milestone:** All core API endpoints operational.

Phase 3 — AI Module & Frontend Development (Weeks 9–12): The ML.NET risk prediction pipeline (RiskService, ModelTrainer, AIController) was built and integrated. The full React.js frontend was developed, covering all role-based dashboard components and the AI Risk Prediction panel. **Milestone:** End-to-end AI prediction pipeline active (mlModelUsed = true confirmed).

Phase 4 — Testing & Quality Assurance (Weeks 13–15): All twenty test cases from Chapter 5 were executed across unit, functional, integration, and performance categories. Identified bugs were resolved and regression tests were re-run to confirm fixes. **Milestone:** All test cases passed; performance targets met (risk prediction < 500 ms).

Phase 5 — Deployment & Documentation (Weeks 16–18): The system was deployed in a pilot school environment using the documented deployment procedure. Docker-based containerised deployment was configured. All FYP documentation chapters were finalised and formatted for submission. **Milestone:** System deployed, documentation submitted, final presentation prepared.

1.12 Summary

This chapter has established the complete foundation for the **Web-Based School Management System project**. The problem statement identified the critical inefficiencies present in current school management practices — particularly the absence of AI-driven predictive analytics and the fragmentation of attendance, marks, behaviour, and complaint data across disconnected systems. The proposed solution addresses these gaps through a **unified, modular, role-based digital platform** built on React.js, ASP.NET Core, SQL Server, and ML.NET.

The **eight SMART objectives** defined in Section 1.3 provide a clear, measurable framework for evaluating project success, with the AI-Based Student Risk Prediction module prioritised as the primary innovation. The scope definition clarifies what is included and excluded from the current development cycle, ensuring realistic and achievable deliverables within the project timeline. The five system modules — Admin, Teacher, Student, Parent, and AI Risk Prediction — collectively address the full range of school management requirements identified in the problem statement.

The **related system analysis** confirmed that no existing product provides the combination of cross-module data integration, structured behaviour tracking, and automated ML-powered risk classification that the proposed SMS delivers. The **vision statement** encapsulates the project's purpose and unique value proposition. Subsequent chapters will detail the requirements (Chapter 2), system design (Chapter 3), implementation (Chapter 4), testing (Chapter 5), deployment (Chapter 6), and conclusion (Chapter 7) of this system.

Chapter 2

Requirement Analysis

2 Analysis

2.1 User classes and characteristics

Table 3 User classes and characteristics

User Class	User Characteristics
Student	Accesses attendance, marks, results, and portfolio. Submits complaints and views notifications. Requires simple, intuitive interface.
Parent	Views child's progress, attendance, fees, and teacher messages. Receives school notifications. Needs secure and easy-to-use dashboard.
Teacher	Marks attendance, enters exam results, uploads certificates and supporting learning materials, posts announcements, and communicates with parents/students. Requires efficient workflow and clear data access.
Administrator	Manages students, teachers, classes, subjects, fees, and overall system data. Monitors reports, complaints, and notices. Needs full system access, control, and reporting tools.

2.2 Requirement Identifying Technique

- The requirements for the School Management System (SMS) were identified using a combination of structured and semi-structured techniques to ensure comprehensive coverage of all stakeholder needs. Initial **proposal analysis** provided baseline functional expectations and clarified the project scope. Informal **interviews and discussions** with teachers, students, and administrative staff helped capture real-world workflows, typical challenges, and user-specific requirements.
- To further refine and validate requirements, **use-case modeling** was employed to describe system interactions from the perspective of each user role. Preliminary **UI sketches and workflow storyboards** assisted in visualizing essential functions and receiving early stakeholder feedback. Additionally, reference to **existing school management processes** and sample academic data enabled the identification of practical constraints and operational rules.
- These combined techniques ensure that the resulting requirements are realistic, complete, and aligned with actual institutional needs, forming a reliable foundation for system design and development.

2.3 Functional Requirements

This section defines the **functional requirements** of the **School Management System (SMS)**. Functional requirements specify **what the system shall do** to support academic, administrative, and communication processes within a school environment.

Each requirement is stated in **clear, testable, and unambiguous terms** to ensure correct interpretation and verification. The requirements are **grouped by major system modules**, including Authentication, Student Management, Teacher Operations, Parent Interaction, Administration, and System Services.

2.3.1 User Authentication and Authorization

FR-1: User Login & Role-Based Access

Table 4 Defines the system requirements for secure user authentication and redirection to role-specific dashboards.

Identifier	FR-1
Title	User Authentication & Role-Based Routing
Requirement	The system shall authenticate users using email/username and password, and route them to a role-specific dashboard based on their assigned role (Admin, Teacher, Student, Parent).
Source	Project Proposal, Stakeholder Interviews (Teachers, Admin Staff)
Rationale	Ensures secure access and tailored system functionality.
Business Rule (if required)	• Only verified users with an active status may log in. • Each user account must be associated with exactly one role at a time. • Password must meet school-defined security standards (e.g., minimum length, complexity). • Users with inactive or suspended accounts shall not be allowed to access the system.
Dependencies	None

Priority	High
-----------------	------

Error Handling / Invalid Input Response:

- If incorrect credentials are entered → system displays “*Invalid username or password*”.
- If account is inactive → system displays “*Your account is deactivated. Contact Admin.*”
- If password does not meet requirements → reject and prompt valid format.
- System must block login after repeated failed attempts (security rule).

FR-2: User Management

Table 5 Specifies administrative capabilities for creating, updating, deactivating, and managing user accounts and roles.

Identifier	FR-2
Title	User Management by Admin
Requirement	The Admin shall be able to add, update, deactivate, or delete user accounts and assign appropriate roles.
Source	Admin Requirements, Proposal
Rationale	Ensures proper system access control and allows accurate management of school personnel and students.
Business Rule (if required)	• Only Admin can create or remove accounts. • Roles must be assigned at account creation.
Dependencies	FR-1
Priority	High

Error Handling / Invalid Input Response:

- If mandatory fields (name, role, email) are missing → show validation error.
- If Admin tries to delete a user with existing linked records → system shall prevent deletion and show error.
- If duplicate email/username exists → system shall reject the account creation.

2.3.2 Attendance Management

FR-3: Mark Attendance

Table 6 Describes the functionality that allows teachers to record daily student attendance for assigned classes.

Identifier	FR-3
Title	Mark Attendance
Requirement	Teachers shall be able to mark daily attendance for students in assigned classes using Present/Absent/Late options.
Source	Teacher Interviews, Proposal
Rationale	Attendance is critical for academic tracking and portfolio generation.
Business Rule (if required)	• Only assigned teachers can mark attendance for their classes. • Attendance must be recorded once per day per class.
Dependencies	FR-1, FR-2
Priority	High

Error Handling / Invalid Input Response:

- If a teacher attempts to mark attendance for a class not assigned → system shows “*Unauthorized action.*”
- If attendance already submitted → system shows “*Attendance already marked for today.*”
- If internet/server interruption occurs → partial entries should be autosaved (if feasible).

2.3.3 Marks and Student Performance

FR-4: Enter Marks

Table 7 Defines the requirements for teachers to enter and manage student marks for tests and examinations.

Identifier	FR-4
Title	Enter Marks
Requirement	Teachers shall be able to enter marks for tests, and exams for each student in their subjects.
Source	Teacher Input, Proposal
Rationale	Marks entry is vital for grading, performance evaluation, and portfolio compilation.
Business Rule (if required)	• Marks must be numeric. • Marks may not exceed defined maximum for each exam/test. • Only subject-assigned teachers may enter marks.
Dependencies	FR-2, FR-1
Priority	High

Error Handling / Invalid Input Response:

- If marks exceed allowed range → system rejects input.
- If teacher attempts to enter marks for unassigned subjects → system shows “*Access denied.*”
- If marks are missing for required fields → system prompts completion.

FR-5: Achievements Upload

Table 8 Specifies the process for uploading and managing student achievements and certificates as part of the digital portfolio.

Identifier	FR-5
Title	Achievements Upload
Requirement	Teachers shall be able to upload student achievements, certificates, awards, and extracurricular activity records to be included automatically in the student portfolio.
Source	Project Proposal – Portfolio Generator

Rationale	Supports a complete digital portfolio by storing all student accomplishments in one centralized location.
Business Rule (if required)	• Only Teachers assigned to the student's class can upload achievements. • Uploads must include title, date, category (Academic / Sports / Co-curricular / Certificates) • Uploaded files must follow allowed size and type constraints. • Students/Parents may view uploaded achievements but cannot modify them.
Dependencies	FR-2, FR-1
Priority	High

Error Handling / Invalid Input Response:

- If file size exceeds limit → system refuses upload and shows “*File too large.*”
- If file type is invalid → system displays allowed formats.
- If teacher tries uploading for unassigned student → access denied.
- If upload fails due to server error → display “*Upload failed. Try again.*” and log the error.

FR-6: Behavior remarks by teachers

Table 9 Describes how teachers can record and manage student behaviour remarks for academic and disciplinary tracking.

Identifier	FR-6
Title	Behavior Remarks by Teachers
Requirement	Teachers shall be able to record positive or negative behaviour remarks for each student, which will become part of the student's behaviour history and portfolio.
Source	Stakeholder Inputs – Behaviour & Complaint Management
Rationale	Provides a structured and formal way to track student behaviour trends, disciplinary issues, and positive contributions.

Business Rule (if required)	• Only assigned teachers may add behaviour remarks for their students. • Behaviour remarks automatically appear in the student's portfolio and may influence risk level. • teachers can only edit their remarks.
Dependencies	FR-2, FR-1
Priority	High

Error Handling / Invalid Input Response:

- If mandatory fields (description/category) are missing → system shows validation error.
- If unauthorized user tries adding remarks → system denies action.
- If remark exceeds character limit → system prompts trimming.
- If behavior entry clashes with missing student record → system shows “*Student not found.*”

FR-7: Student Risk Monitoring

Table 10 Defines automated analysis of attendance, marks, and behaviour to generate student risk indicators.

Identifier	FR-7
Title	Student Risk Monitoring
Requirement	The system shall automatically analyze student attendance, marks, and behaviour records to generate risk indicators (Low Risk, Medium Risk, High Risk)
Source	Project Proposal – Risk Monitoring Objective
Rationale	Helps teachers and administration identify academically or behaviourally struggling students early and take corrective actions.
Business Rule (if required)	• Risk levels must be calculated based on predefined criteria (e.g., low attendance, poor marks, repeated complaints). • Risk indicators must update automatically when new data (attendance/marks/complaints) is added.

	Only Admins and Teachers may view full risk analysis; Parents see only summarized indicators (if enabled).
Dependencies	FR-3, FR-4
Priority	High

Error Handling / Invalid Input Response:

- If required data is missing → system shows “*Insufficient data to generate risk level.*”
- If unauthorized user tries accessing the module → access denied.
- If risk calculation fails due to server error → log issue and display fallback status “*Risk unavailable*”.

2.3.4 Student Portfolio Generation

FR-8: Portfolio Compilation

Table 11 Specifies the system’s ability to compile attendance, marks, behaviour, and achievements into a unified student portfolio.

Identifier	FR-8
Title	Portfolio Compilation
Requirement	The system shall compile student attendance, marks, behavior notes, and achievements into a structured digital portfolio.
Source	Proposal
Rationale	Provides a consolidated view of student performance.
Business Rule (if required)	- Marks must be numeric . - Only relevant stakeholders can generate/view portfolios.
Dependencies	FR-3, FR-4
Priority	Medium

Error Handling / Invalid Input Response:

- If any required data is missing, system shows “*Portfolio incomplete—missing attendance/marks/etc.*”
- If unauthorized user tries to access portfolio → deny access.

2.3.5 Noticeboard & Notifications

FR-9: Post Notices

Table 12 Describes the functionality for authorized staff to publish notices to students and parents.

Identifier	FR-9
Title	Post Notices
Requirement	Admin/Teachers shall be able to post school-wide or class-specific notices visible on user dashboards.
Source	Admin
Rationale	Improves communication across all stakeholders.
Business Rule (if required)	• Only authorized staff can create notices. • Notice may be tagged as school-wide or class-specific.
Dependencies	FR-9
Priority	High

Error Handling / Invalid Input Response:

- If mandatory content is missing → system refuses publishing.
- If unauthorized user attempts posting → show “*Permission denied.*”

FR-10: Notify Users

Table 13 Defines the notification mechanism for informing users about notices and important system updates.

Identifier	FR-10
-------------------	-------

Title	Notify Users
Requirement	The system shall notify parents and students about new notices via dashboard alerts
Source	Parent Feedback
Rationale	Ensures timely communication of important announcements.
Business Rule (if required)	• Notifications shall follow user-specific communication settings. • Notifications must be generated instantly after posting a notice.
Dependencies	FR-6
Priority	Medium

Error Handling / Invalid Input Response:

- If notification delivery fails → system logs the failure for Admin review.
- If user disables notifications → system respects settings.

2.3.6 Complaint and Behavior Management

FR-12: Submit Complaint

Table 14 Defines the process through which students and parents can submit formal complaints.

Identifier	FR-12
Title	Submit Complaint
Requirement	Students/Parents shall be able to submit complaints
Source	Student Surveys
Rationale	Provides a formal channel for academic or behavioral concerns.
Business Rule (if required)	• Complaints shall be categorized (e.g., academic, discipline).

Dependencies	FR-1
Priority	Medium

Error Handling / Invalid Input Response:

- If mandatory fields are missing → display error.
- If user is not authenticated → deny submission.

FR-13: Complaint Resolution Workflow

Table 15 Describes how teachers and administrators manage and resolve submitted complaints.

Identifier	FR-13
Title	Complaint Resolution Workflow
Requirement	Teachers/Admin shall update complaint status (Pending, In Review, Resolved)
Source	Admin Decision
Rationale	Ensures transparent complaint handling.
Business Rule (if required)	• Only authorized staff may update status.
Dependencies	FR-9
Priority	Medium

Error Handling / Invalid Input Response:

- If unauthorized staff attempts update → system denies access.
- If complaint does not exist → show “Complaint not found.”

Fee Reminder

FR-14: Fee Status View

Table 16 Specifies the system requirements for displaying student fee status to authorized users.

Identifier	FR-14
Title	Fee Status View
Requirement	The system shall display the fee status (Paid/Unpaid/Pending) for each student.
Source	Finance Department
Rationale	Increases transparency for parents and students.
Business Rule (if required)	• Only Admin can update fee records. • Students/Parents can only view their own fee data.
Dependencies	FR-1
Priority	Medium

Error Handling / Invalid Input Response:

- If user tries to view another student's fee → access denied.
- If fee data is missing → show “Fee record not available.”

2.3.7 Audit Logs & System Monitoring

FR-15: Audit Logging

Table 17 Defines the system's logging requirements for tracking critical actions and ensuring accountability.

Identifier	FR-15
Title	Audit Logging
Requirement	The system shall record logs for login activity, attendance marking, marks entry, complaint updates, and account changes.
Source	Admin Security Policy
Rationale	Supports accountability and system monitoring.

Business Rule (if required)	• Logs must be kept for at least one year. • Logs shall be immutable.
Dependencies	All functional modules
Priority	High

Error Handling / Invalid Input Response:

- If logging fails → system shows an Admin alert and stores error in backup log.
- Unauthorized access to logs → strictly denied.

2.4 Non-Functional Requirements

This section specifies the **nonfunctional requirements (NFRs)** of the **School Management System (SMS)**, which define **how the system should perform** rather than specific functionalities. These requirements address system **quality attributes, performance standards, security, usability, reliability, scalability, and maintainability**.

2.4.1 Reliability

NFR-R1: System Uptime

The system shall maintain at least 98% uptime during school operational hours (7 AM – 6 PM).

NFR-R2: Backup Frequency

The system shall automatically create an incremental database backup every 24 hours at 9 PM, ensuring data protection.

NFR-R3: Data Integrity

The system shall ensure 99% data integrity, meaning records for attendance, marks, notices, and communication must not be lost or corrupted during CRUD operations.

NFR-R4: Recovery Time Objective (RTO)

In case of system failure, the system shall recover within 10 minutes after restoration procedures begin.

NFR-R5: Fault Tolerance

If a module fails (e.g., noticeboard service), the rest of the system shall continue working without interruption.

2.4.2 Usability

NFR-U1: Learning Curve

A new user (teacher/parent/student) shall be able to learn the system interface within 30 minutes without training.

NFR-U2: Task Completion

- A teacher shall be able to mark attendance in 3 clicks or fewer.
- A parent shall be able to check the student portfolio within 4 clicks.
- A student shall be able to submit a complaint within 3 clicks.

NFR-U3: Accessibility

The system shall be fully accessible on mobile, tablet, and desktop with responsive design (React + Tailwind/Bootstrap).

NFR-U4: Error Prevention

User input errors (e.g., missing required fields) shall be detected and user-friendly error messages displayed within 1 second.

NFR-U5: Language & Clarity

The interface shall maintain consistent terminology and iconography for all dashboards (Students, Teachers, Parents, Admin).

2.4.3 Performance

NFR-P1: Response Time

For all CRUD operations (attendance entry, portfolio view, noticeboard load, messaging), the system shall respond within:

- 2 seconds for simple operations
- 5 seconds for complex queries (e.g., portfolio generation)

NFR-P2: Concurrent Users

The system shall support 100 concurrent users without significant degradation in performance (since it is designed for a single school environment).

NFR-P3: Data Processing

The system shall process a full class attendance record (40–60 students) within 3 seconds.

NFR-P4: Page Rendering

95% of all pages shall load in under 3 seconds on a standard 4G connection.

2.4.4 Security

NFR-S1: Role-Based Access Control

The system shall enforce strict role-based access to protect sensitive data for Students, Teachers, Parents, and Admins.

NFR-S2: Unauthorized Access Prevention

Any unauthorized attempt to access another user's data shall be blocked and logged within 1 second.

NFR-S3: Data Protection

All sensitive data (student profiles, marks, attendance, complaints) shall meet basic confidentiality and integrity standards (no plaintext exposure).

NFR-S4: Session Management

User sessions shall automatically expire after 20 minutes of inactivity and require reauthentication.

NFR-S5: Audit Trail

The system shall maintain audit logs for:

- Attendance updates
- Marks entry
- Portfolio updates
- Complaint status changes
- Notice creation
- Each log shall include timestamp, user role, and action details.

2.5 External Interface Requirements

This section defines all interfaces through which the **School Management System (SMS)** interacts with its **users, external hardware, external software components, communication mechanisms, and supporting platforms**. These requirements ensure smooth communication, consistent user experience, and correct integration with external technologies.

2.5.1 User Interfaces Requirements

The School Management System is a web-based application used by four primary user roles: **Admin, Teacher, Student, and Parent**. The following logical user interface requirements apply to all modules.

GUI Design Standards

The system shall follow consistent UI/UX standards across all dashboards, including:

- Alignment with **Google Material Design** and **React component styling conventions**.
- Use of **Bootstrap or Tailwind CSS** for responsive and mobile-friendly layouts.
- Consistent use of fonts, icons, layouts, and interactive components across all pages.
- Standard button labels such as **Save, Update, Submit, Cancel, Back, Upload, and Delete**.
- A uniform **color scheme** representing institutional or educational themes (blue, green, white).
- A minimum screen resolution compatibility of **1366 × 768**, with support for responsive adaptation to mobile devices.

Navigation & Controls

- A left-side or top navigation bar shall be present on every dashboard with clearly labeled links.
- Common navigation items include: **Home, Notices, Messages, Attendance, Portfolio, and Logout**.
- A **Help** or **Support** button will be accessible from every screen.
- Consistent placement of **Back, Home, and Dashboard** icons.
- Tab/field navigation shall follow left-to-right logical movement for form-based inputs.
- Forms shall support error highlighting, placeholder text, and tooltip hints where necessary.

Display Conventions

- Success messages shall appear in green; error or warning messages in red.
- System messages shall appear in toast alerts or modal dialogs.
- Required fields shall be marked with a red asterisk (*).
- Data tables shall support pagination, filtering, and search.

- Icons shall be used for common actions (edit pencil, delete bin, upload cloud, etc.).

Accessibility Requirements

- The interface shall comply with **WCAG 2.0 Level AA** accessibility guidelines.
- High-contrast mode shall be supported for users with visual impairments.
- Provide text alternatives for icons and images using **alt-text** attributes.
- Font size shall be adjustable using browser zoom (up to 200%).
- All buttons and links shall be keyboard-accessible (TAB, ENTER, SHIFT+TAB).

Localization & Language Support

- English shall be the primary interface language.
- The system shall allow easy integration of additional languages (e.g., Urdu) by using:
 - Unicode-supported fonts
 - Separated label constants
 - Right-to-left layout support (if Urdu is added)

Mockups & UI Documentation

- Screen mockups and wireframes shall be provided in a **separate User Interface Design Document**.
- Mockups included in the SRS serve only as illustrative examples and not final design.

2.5.2 Software interfaces

This section describes all interactions between the SMS and external software systems, libraries, frameworks, and databases.

Backend Framework

- The system shall use **ASP.NET Core Web API v7.0+** for server-side logic.
- API endpoints shall follow RESTful conventions (GET, POST, PUT, DELETE).
- Authentication shall be implemented using **JWT (JSON Web Tokens)**.

Frontend Framework

- The frontend shall be built using **React.js v18+**.
- Fetch API shall be used for server communication.

Database Interface

- The system shall use **Microsoft SQL Server 2019+** as the primary database.

- Database interactions shall occur through:
 - **Entity Framework Core v7.0+** (ORM)
 - Code-first migrations for schema generation
- SQL Server must support:
 - User tables
 - Attendance records
 - Marks records
 - Complaints and behaviour logs
 - Portfolio data
 - Audit logs

External Libraries

The system shall integrate the following libraries:

- **JWT Authentication Library** for secure login
- **React Router** for navigation
- **React Icons / Lucide Icons** for visual consistency
- **Tailwind / Bootstrap** for UI layouts
- **Chart.js or Recharts** for analytics dashboards

External Systems (Optional Integrations)

If enabled:

- **Email service provider (SMTP)** for sending alerts
 - **SMS Gateway API** for fee reminders and notices
- These interfaces will require:
- Valid API key
 - Network availability
 - Compliance with security policies

2.5.3 Hardware interfaces

The School Management System does not require specialized hardware. It must operate on standard computing devices meeting the following requirements:

Supported Devices

- 1) Desktop computers
- 2) Laptops
- 3) Tablets

4) Mobile phones

All devices must support:

- A modern web browser
- Minimum 2 GB RAM (recommended 4 GB for teachers/admin due to dashboards)

Server Hardware

If self-hosted, the backend server shall have:

- Minimum **8 GB RAM**
- **100 GB storage**
- **Quad-core processor**
- Windows Server or Linux environment

Hardware Integration

No direct hardware integrations (barcode scanners, biometric devices, etc.) are included *in current scope*.

System architecture allows future extensions.

2.5.4 Communications interfaces

This section describes all communication protocols and network interactions of the SMS.

Network Protocols

The system shall communicate over:

- **HTTPS** (mandatory)
- **TLS 1.2+ encryption**
- **RESTful API calls** over port 443

Web Browser Compatibility

The system must support:

- Google Chrome
- Microsoft Edge
- Firefox
- Safari (for iOS)

API Communication

- All client-server communication shall follow JSON formatting.
- API responses must include:
 - Status code
 - Success/Failure message
 - Error details where applicable

System Availability Requirements

- The system shall be operable with internet speeds as low as **2 Mbps**.
- Requests should time out only after 30 seconds of inactivity.

2.6 Summary

This Software Requirements Specification (SRS) documents the complete requirements engineering process carried out for the development of the **Web-Based School Management System (SMS)**. The requirements were gathered and refined using multiple techniques, including stakeholder discussions, informal interviews, proposal analysis, use-case modeling, and storyboard-based workflow visualization. These techniques helped ensure that the identified requirements accurately reflect real-world academic and administrative processes within a school environment.

Functional requirements were systematically defined for all core modules, including **user authentication, attendance management, marks entry, student portfolio generation, risk monitoring, complaint handling, fee status management, and notifications**. Each requirement was documented with clear descriptions, business rules, dependencies, priorities, and error-handling conditions to ensure clarity, traceability, and testability.

Non-functional requirements were specified to address system quality attributes such as **reliability, usability, performance, and security**. These requirements were made measurable and verifiable to support validation during system testing. **External interface requirements** further defined how the system interacts with **users, software components, hardware platforms, and communication protocols**.

Use case analysis and storyboarding were employed to validate functional requirements from the perspective of different user roles and to visualize system behavior in practical scenarios. Overall, this **requirements engineering process** ensures that the proposed system aligns with project objectives, stakeholder expectations, and academic standards, providing a strong foundation for system design, implementation, and evaluation.

Chapter 3

Design and Architecture

3 System Design

Product Perspective

The Web-Based Unified School Management System is designed as a centralized, web-based application that integrates academic and administrative operations of an educational institution. The system operates as an independent platform accessible through standard web browsers and does not require client-side installation. It interacts with a centralized database server for storing and retrieving academic and administrative data and may integrate with external services such as email or notification systems for communication purposes.

The system follows a layered architecture where the presentation layer handles user interaction, the application layer manages business logic, and the data layer is responsible for persistent storage. This separation ensures scalability, maintainability, and ease of future enhancements. All system components communicate through well-defined interfaces to ensure reliable and secure data exchange.

Design Constraints

Several constraints influenced the system design. Performance constraints require the system to handle multiple concurrent users with minimal response time. Security constraints demand secure authentication, authorization, and data protection mechanisms to safeguard sensitive academic and administrative information. Compatibility constraints ensure that the system functions across commonly used web browsers and devices. Additionally, the design must be scalable to accommodate increasing numbers of users and records without major architectural changes.

3.1 Design Considerations

Assumptions

and

Dependencies:

It is assumed that users will have access to a stable internet connection and modern web browsers. The system depends on the availability of a reliable database management system and web server for proper functionality. It is also assumed that institutional data provided to the system is accurate and maintained regularly. The design assumes adherence to standard web technologies and frameworks to ensure smooth development and deployment.

Limitations:

The system is dependent on internet availability and may not function in offline environments. Performance may be affected under extremely high loads without additional server resources. Integration with third-party systems is limited to predefined interfaces and may require further

customization for advanced interoperability. Additionally, the system's functionality is constrained by the scope defined in the Software Requirements Specification (SRS).

Risks:

Potential risks include data security breaches, system performance degradation, and improper data handling by users. These risks are mitigated through secure authentication mechanisms, role-based access control, regular data backups, and optimized database design. Proper validation and error-handling mechanisms are incorporated to minimize system failures and ensure reliable operation.

3.2 Design Models

The system design is described using standard UML design models to clearly represent its structure and behavior under an object-oriented development approach.

Design Class Diagram (DCD)

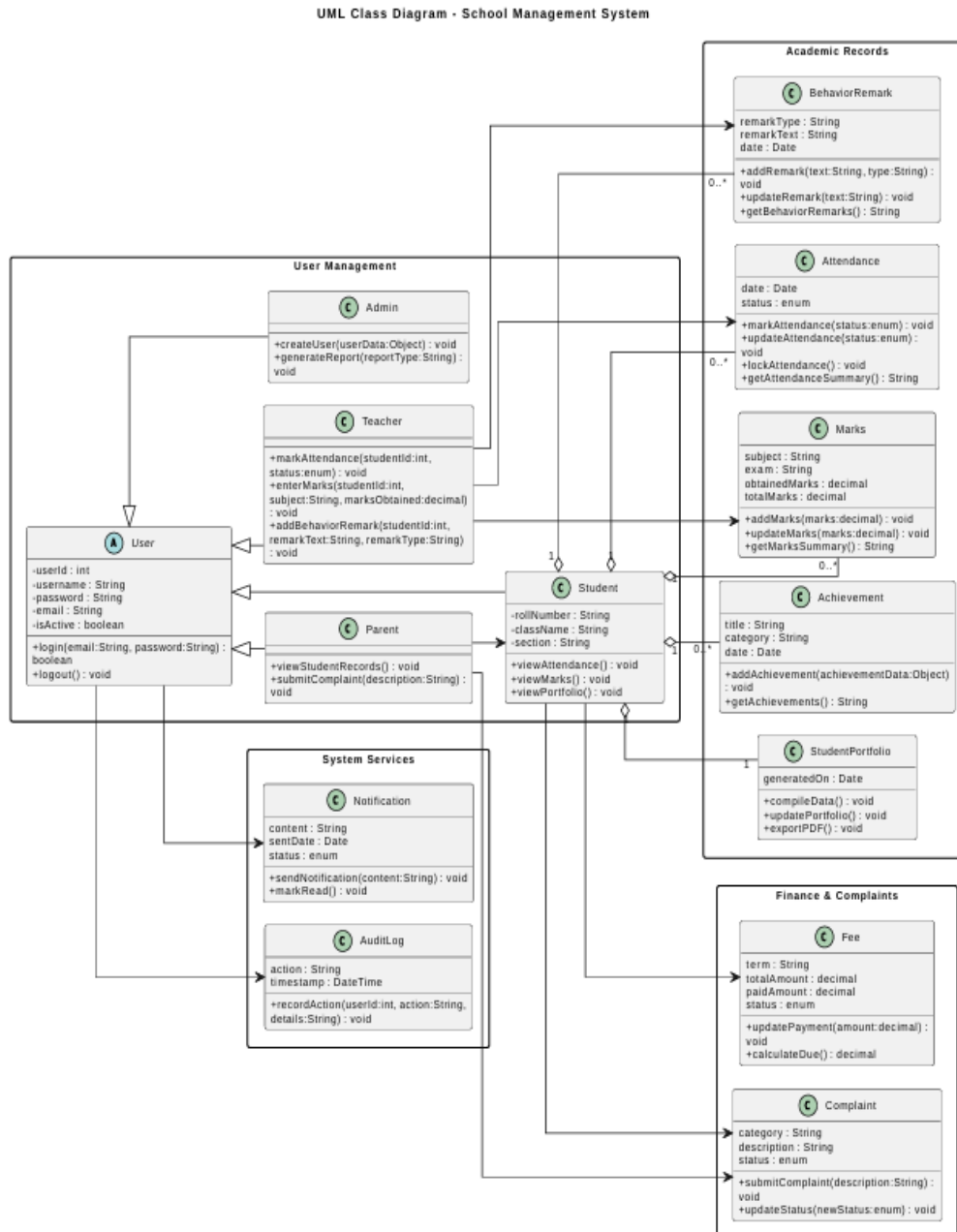


Figure 2 Class Diagram

Interaction Diagram (sequence)

User Flow login – Sequence Diagram

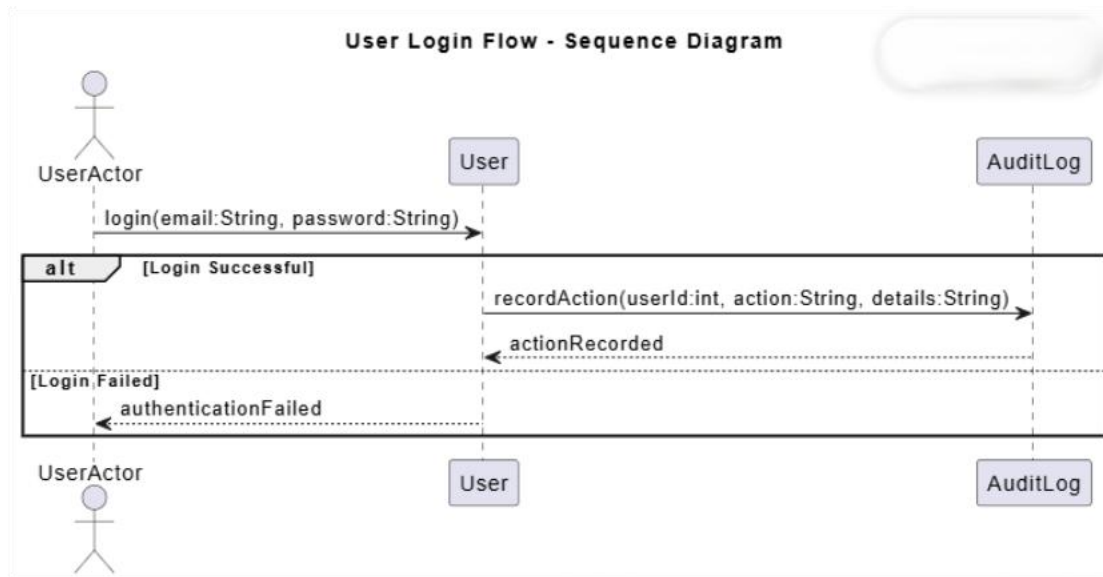


Figure 3 Sequence Diagram 1- User Flow Login

Fig explanation: This figure illustrates the user authentication process, where login credentials are validated and successful login activities are recorded through the audit logging service. It ensures secure access and accountability within the system.

Teacher Academic Management Flow – Sequence Diagram

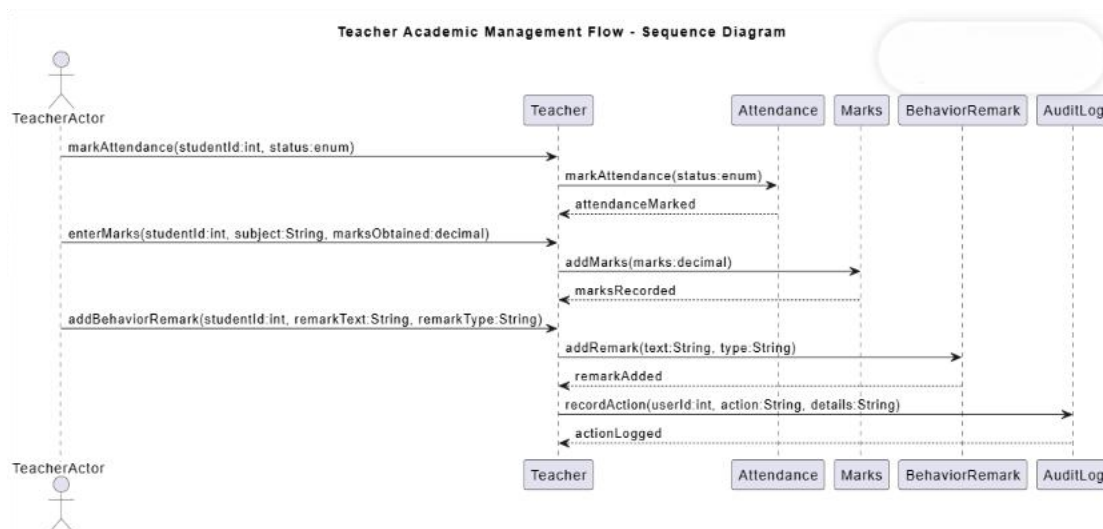


Figure 4 Sequence Diagram 2- Teacher Academic Management Flow

Fig explanation: This figure shows how a teacher manages academic activities, including marking attendance, entering marks, and adding behavior remarks. Each action is logged to maintain data integrity and accountability.

Student Portfolio Generation Flow – Sequence Diagram

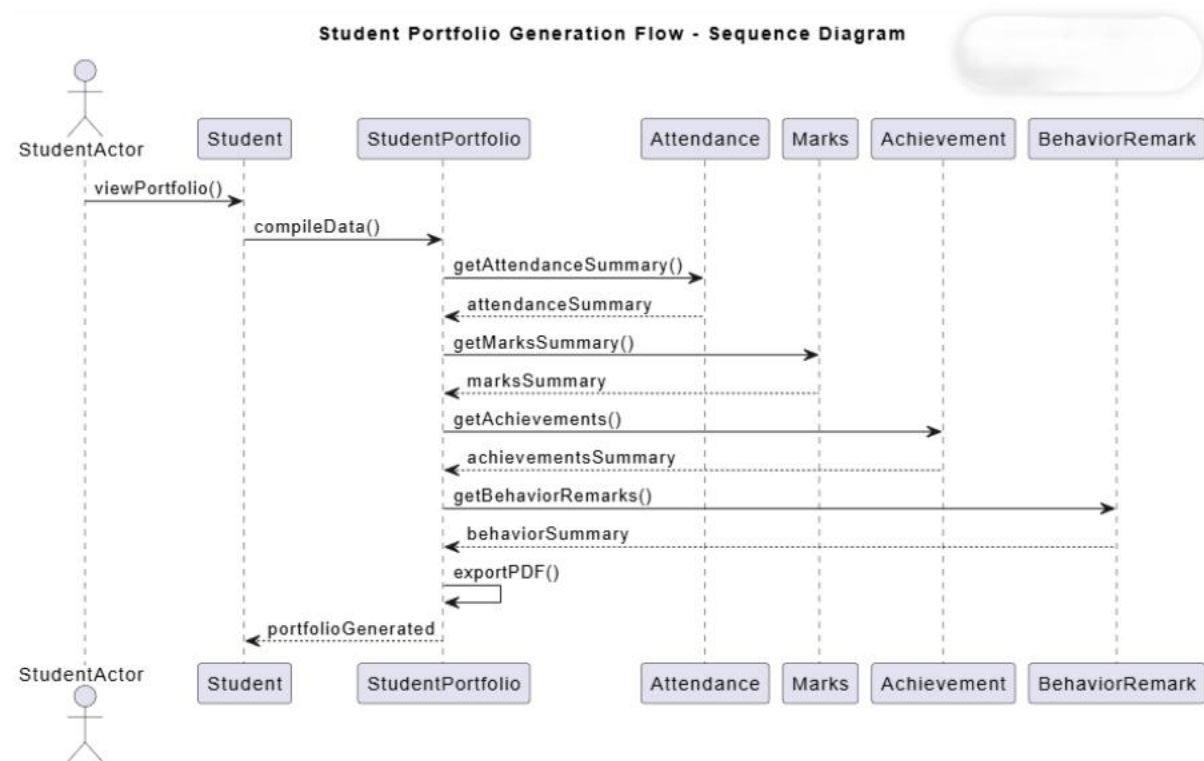


Figure 5 Sequence Diagram 3- Student Portfolio Generation Flow

Fig explanation: This figure depicts the process of generating a student portfolio by aggregating attendance, marks, achievements, and behavior records. The compiled data is then exported as a complete academic performance report.

State Transition Diagram

Attendance Record State Diagram

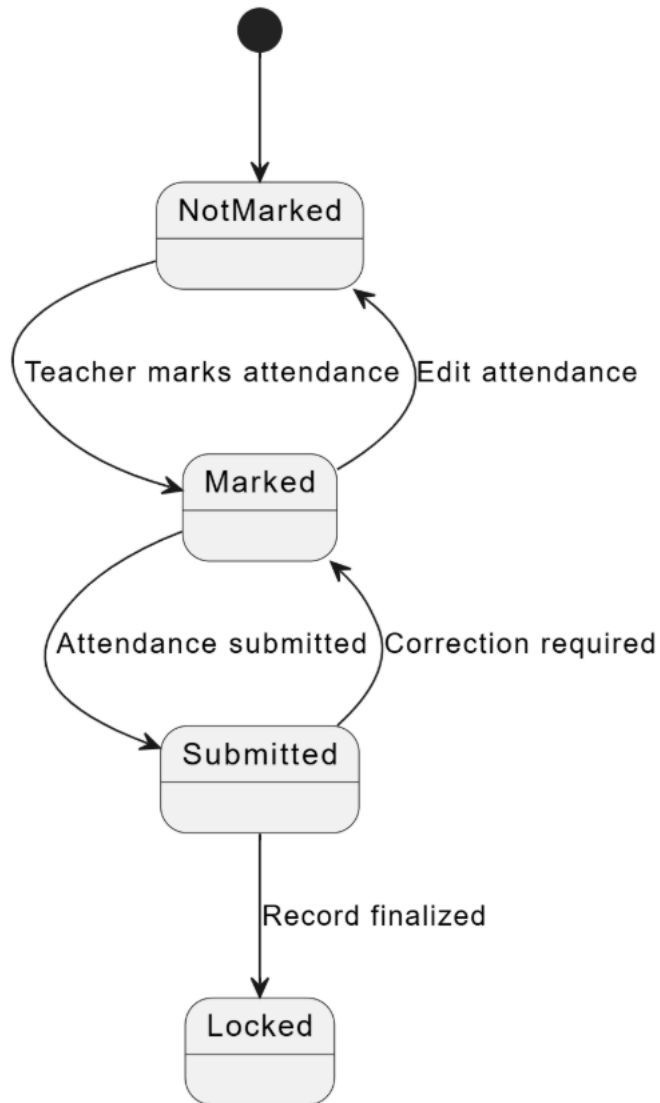


Figure 6 State Diagram 1- Attendance Record

Fig explanation: This state diagram models the attendance recording process. Attendance starts unmarked, is entered by the teacher, submitted for confirmation, and finally locked to prevent unauthorized changes.

Complaint Management State Diagram

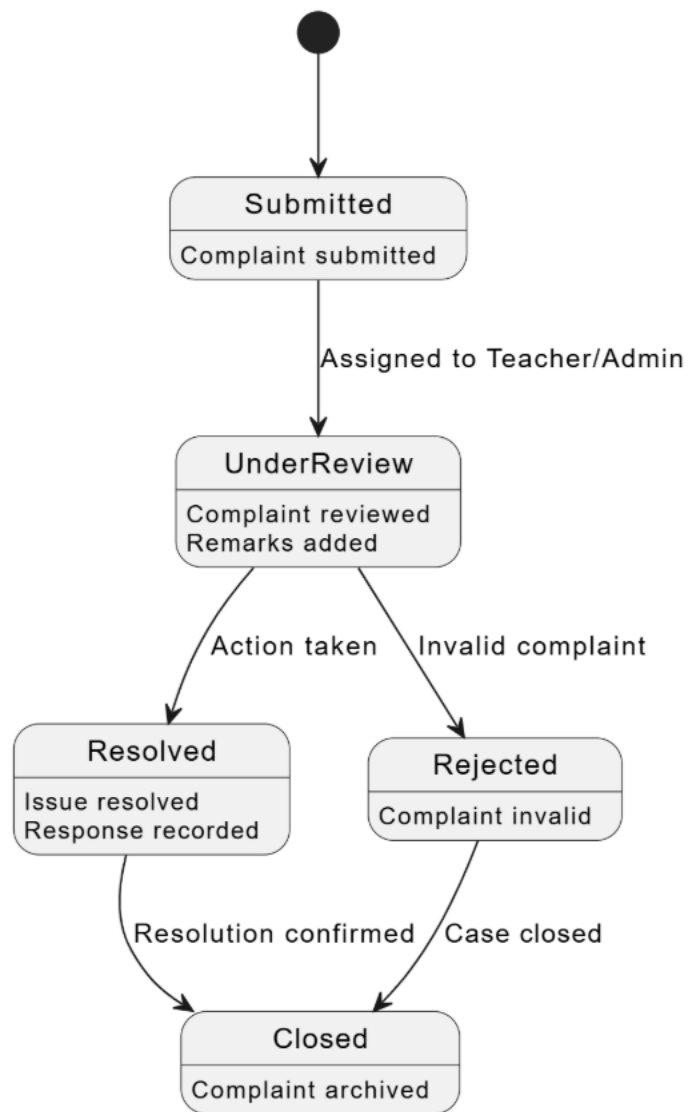


Figure 7 State Diagram 2- Complain Mangement

Fig explanation: This diagram illustrates the complaint handling workflow. A complaint is submitted by a student or parent, reviewed by staff, resolved or rejected, and finally closed after confirmation.

Student Portfolio State Diagram

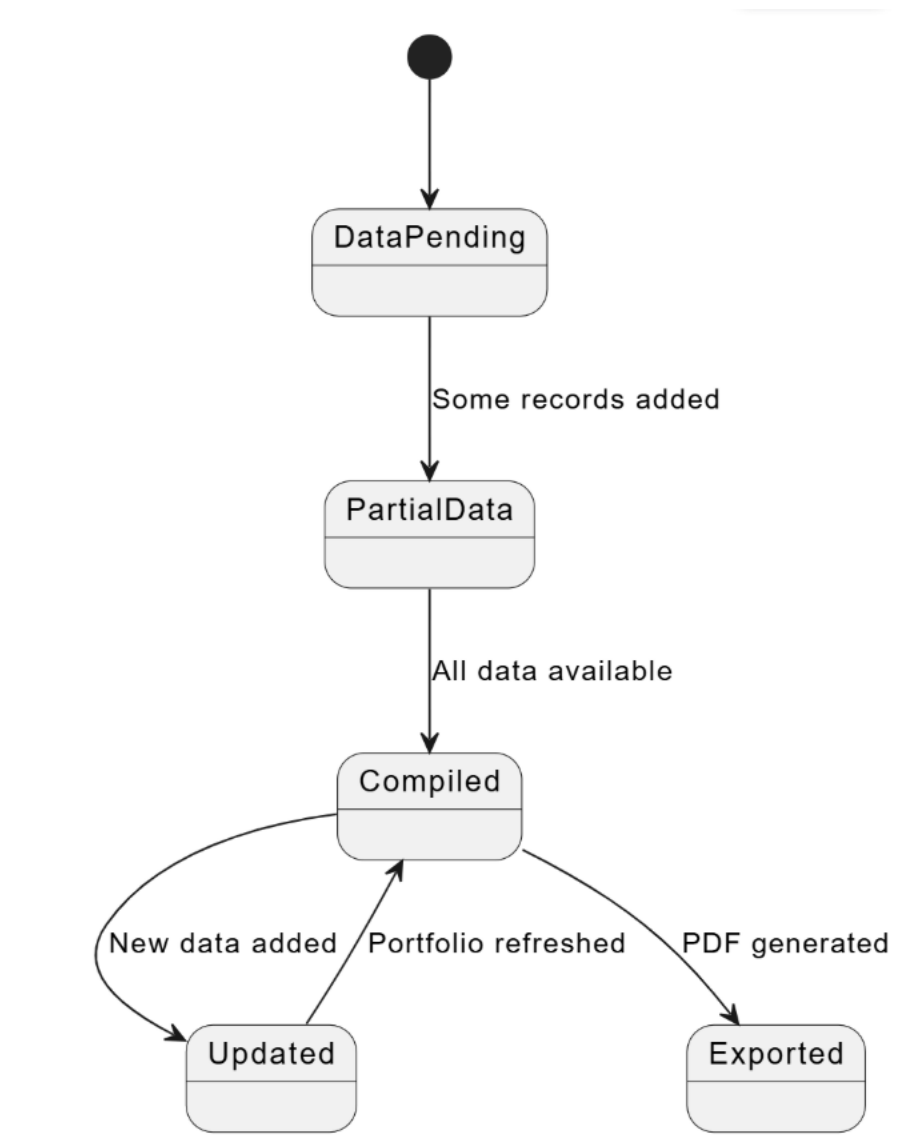


Figure 8 State Diagram 3- Student Portfolio

Fig explanation: This diagram represents the automated student portfolio generation process. The system collects academic data, compiles it into a portfolio, updates it when new data arrives, and allows export in PDF format.

Fee payment State Diagram

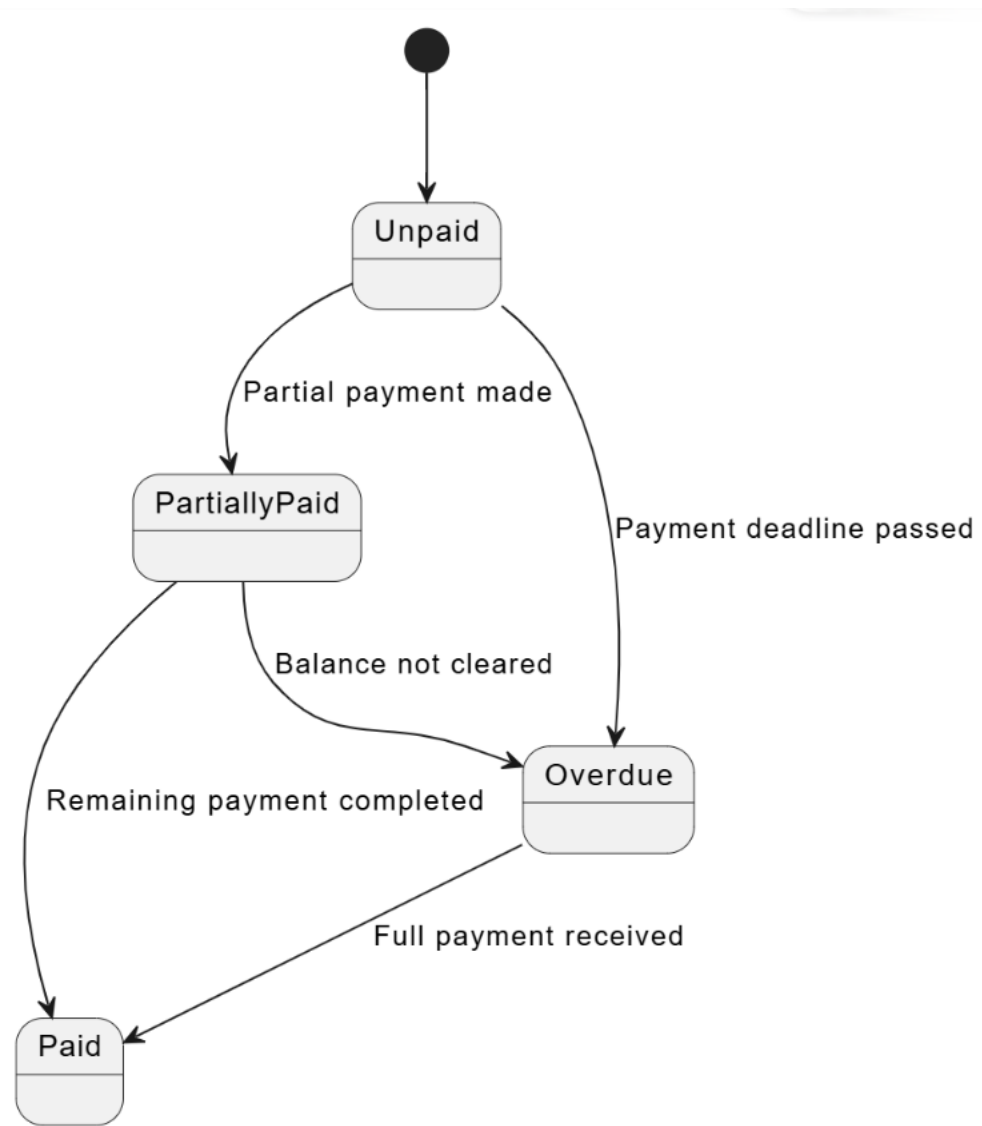


Figure 9 State Diagram 4- fee Payment

Fig explanation: This diagram shows the fee payment lifecycle of a student. It tracks payment status from unpaid to partially paid, overdue, and fully paid.

User Account State Diagram

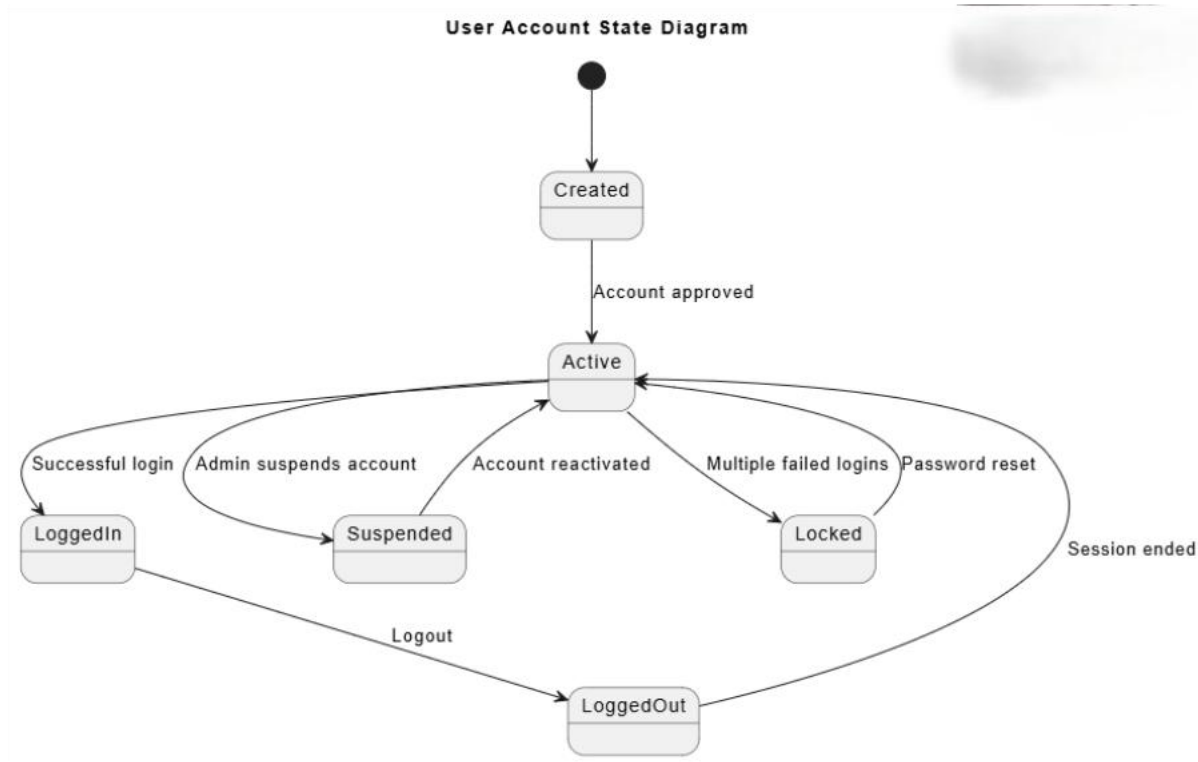


Figure 10 State Diagram 5- User account

Fig explanation: This state diagram represents the lifecycle of a user account in the system. It shows transitions from account creation to activation, login/logout behavior, and administrative actions such as suspension, locking, and reactivation.

3.3 Architectural Design

Component Structure and Inter-Module Relationships:

The system architecture is designed using a layered and component-based approach, following the MVC and client–server architectural patterns. The architecture separates concerns across presentation, application, and data tiers, enabling scalability, maintainability, and secure role-based access. Each layer communicates with the adjacent layer through well-defined interfaces and APIs to ensure controlled data flow and modular development.

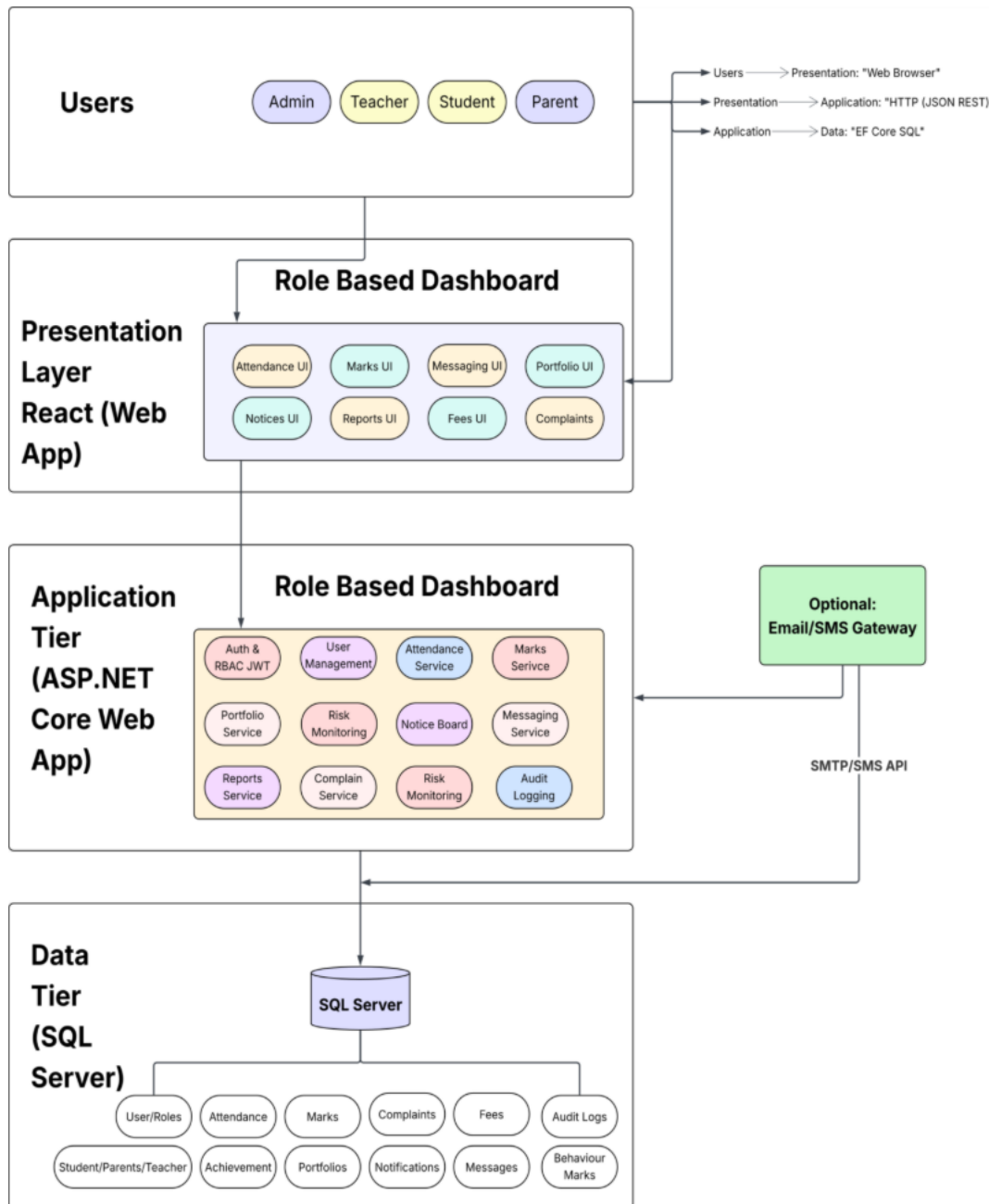


Figure 11 System Architecture Diagram

Fig explanation: This figure illustrates the system interaction flow in which user requests originate from the client-side interface and are processed by application services through the presentation layer. Business logic execution and data persistence are handled via the data access

layer. Role-Based Access Control (RBAC) and authentication mechanisms are enforced prior to any operation, ensuring secure and authorized access to system functionalities.

Mapping Components to Architecture Layers

Architecture Layer	MVC Role	Component / Module	Description
Client Layer	View	Web UI / Browser	Provides user interface for Admin, Teacher, Student, and Parent to view dashboards, forms, and reports
Application Layer	Controller	Controllors (Authentication, Attendance, Marks, Complaint, Fee)	Handles user requests, input validation, and coordination between the presentation layer and business logic
Business Layer	Model	User Management	Manages user accounts, roles, authentication, and role-based access control
Business Layer	Model	Academic Users	Represents core system entities including Student, Teacher, Parent, and Admin along with their operations
Business Layer	Model	Academic Records	Handles attendance, marks, behavior remarks, achievements, and student portfolios
Business Layer	Model	Complaints & Fees	Manages complaint registration, tracking, resolution, and fee payment processing

Service Layer	—	System Services	Provides cross-cutting services such as notifications, risk monitoring, and audit logging
Data Layer	—	Relational Database	Stores all persistent system data including users, academic records, complaints, fees, logs, and reports

UML Component diagram

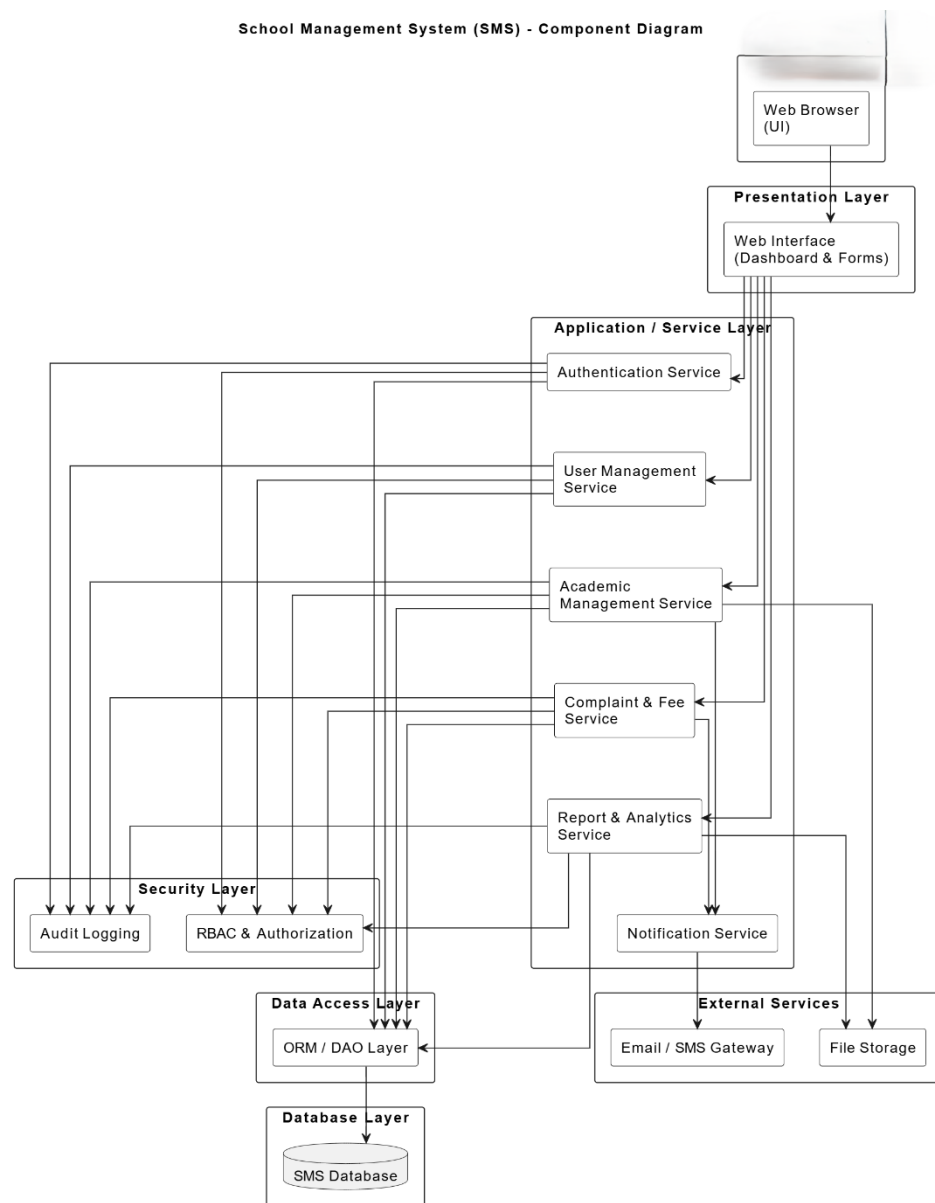


Figure 12 UML Component Diagram

Figure explanation: This Figure illustrates the UML Component Diagram of the School Management System, representing a layered MVC-based client-server architecture with integrated security, data access, and external services.

3.4 Data Design

Information Domain Transformation:

The information domain defined in the Software Requirements Specification (SRS) is transformed into structured data entities using a strictly **Object-Oriented (OO) design approach**. Each functional requirement is mapped to one or more domain objects that encapsulate both **data (attributes)** and **behavior (methods)**. This mapping ensures full traceability between the SRS and the Software Design Specification (SDS).

The system strictly supports the four user roles defined in the SRS: **Admin, Teacher, Student, and Parent**. No additional actors or roles are introduced at the design level. Role-specific behavior is enforced through object methods and access control rather than dynamic role assignment.

Persistent data are stored in a **relational database**, while temporary data such as user sessions and notifications are handled at the application layer. Business rules and validations are implemented through object methods, ensuring separation between data persistence and application logic.

Data Storage and Organization:

1. **Database Type:** Relational Database (MySQL / PostgreSQL)
2. **Design Paradigm:** Object-Oriented domain model mapped to relational tables
3. **Normalization Level:** Third Normal Form (3NF) to reduce redundancy
4. **Security Measures:**

Encrypted password storage

Role-based access control

Audit logging of critical actions

5. **Data Processing:**

CRUD operations performed via service-layer object methods

Referential integrity enforced using primary and foreign keys

Major System Entities:

The major system entities derived directly from the SRS are:

1. User (Admin, Teacher, Student, Parent)
2. Student
3. Parent
4. Teacher
5. Admin
6. Attendance
7. Marks

8. Achievement
9. BehaviorRemark
- 10.Complaint
- 11.Fee
- 12.Notification
- 13.Message
- 14.StudentPortfolio
- 15.AuditLog

3.4.1 Data Dictionary

Terminology	Description
Achievement	Object representing student achievements. Attributes: achievementId:int (PK), studentId:int (FK → Student), title:String, category:String, date:Date, filePath:String. Used to store awards, certificates, and extracurricular accomplishments.
Achievement.date	Date of achievement. Type: Date
Achievement.filePath	File path for attachment/document. Type: String
Achievement.title	Name/title of achievement. Type: String
Achievement.category	Type/category of achievement (e.g., Sports, Academic). Type: String
Achievement.achievementId	Primary key identifier. Type: int
Achievement.studentId	Foreign key to Student. Type: int
Achievement.addAchievement()	Method to add a new achievement. Parameters: achievementData:Object
Admin	Object representing administrative user. Attributes: adminId:int (PK), userId:int (FK → User), firstName:String, lastName:String. Admin manages users, fees, notices, reports, and system settings.

Admin.adminId	Primary key identifier. Type: int
Admin.userId	Foreign key to User. Type: int
Admin.firstName	Admin first name. Type: String
Admin.lastName	Admin last name. Type: String
Admin.createUser(userData:Object)	Method to create a new user account.
Admin.generateReport(reportType:String)	Generates system reports. Parameters: reportType:String
Admin.updateSettings(settings:Object)	Updates system settings. Parameters: settings:Object
AuditLog	Tracks system actions for accountability. Attributes: logId:int, userId:int (FK → User), action:String, timestamp:Timestamp, details:String
AuditLog.logId	Primary key. Type: int
AuditLog.userId	User performing the action. Type: int
AuditLog.action	Description of action. Type: String
AuditLog.timestamp	Action timestamp. Type: Timestamp
AuditLog.details	Additional details about action. Type: String
AuditLog.recordAction(userId:int, action:String, details:String)	Logs an action in the system.
Attendance	Tracks student attendance. Attributes: attendanceId:int, studentId:int (FK), teacherId:int (FK), date:Date, status:enum {Present, Absent, Late}
Attendance.attendanceId	Primary key. Type: int
Attendance.studentId	Foreign key to Student. Type: int
Attendance.teacherId	Foreign key to Teacher. Type: int

Attendance.date	Date of attendance. Type: Date
Attendance.status	Attendance status. Type: enum {Present, Absent, Late}
Attendance.markAttendance(status:enum)	Marks attendance for a student.
Attendance.updateAttendance(status:enum)	Updates existing attendance record.
Attendance.lockAttendance()	Locks attendance to prevent further edits.
BehaviorRemark	Stores teacher comments about student behavior. Attributes: remarkId:int, studentId:int (FK), teacherId:int (FK), remarkType:String, remarkText:String, date:Date
BehaviorRemark.remarkId	Primary key. Type: int
BehaviorRemark.studentId	Foreign key to Student. Type: int
BehaviorRemark.teacherId	Foreign key to Teacher. Type: int
BehaviorRemark.remarkType	Type of remark (Positive/Negative). Type: String
BehaviorRemark.remarkText	Remark content. Type: Text
BehaviorRemark.date	Date of remark. Type: Date
BehaviorRemark.addRemark(text:String, type:String)	Adds a new behavior remark.
BehaviorRemark.updateRemark(text:String)	Updates remark content.
Complaint	Represents complaints submitted by students or parents. Attributes: complaintId:int, submittedByUserId:int (FK), assignedToUserId:int (FK), category:String, description:String, status:enum {Submitted, UnderReview, Resolved, Rejected, Closed}, dateSubmitted:Date, dateClosed:Date
Complaint.complaintId	Primary key. Type: int

Complaint.submittedByUserId	Foreign key to submitting User. Type: int
Complaint.assignedToUserId	Foreign key to assigned User (Teacher/Admin). Type: int
Complaint.category	Complaint category. Type: String
Complaint.description	Complaint details. Type: Text
Complaint.status	Current status. Type: enum
Complaint.dateSubmitted	Submission date. Type: Date
Complaint.dateClosed	Resolution date. Type: Date
Complaint.submitComplaint(description:String)	Submits a new complaint.
Complaint.updateStatus(newStatus:enum)	Updates complaint status.
Complaint.addRemarks(remarks:String)	Adds remarks to complaint record.
Fee	Tracks student fee payments. Attributes: feeId:int, studentId:int (FK), term:String, totalAmount:decimal, paidAmount:decimal, dueDate:Date, status:enum {Paid, Partial, Pending, Overdue}
Fee.feeId	Primary key. Type: int
Fee.studentId	Foreign key to Student. Type: int
Fee.term	Academic term/semester. Type: String
Fee.totalAmount	Total fee amount. Type: decimal
Fee.paidAmount	Amount paid. Type: decimal
Fee.dueDate	Payment due date. Type: Date
Fee.status	Payment status. Type: enum
Fee.updatePayment(amount:decimal)	Updates paid amount.
Fee.calculateDue()	Calculates remaining due.

Marks	Stores assessment/exam results. Attributes: marksId:int, studentId:int (FK), teacherId:int (FK), subject:String, exam:String, marksObtained:decimal, totalMarks:decimal
Marks.marksId	Primary key. Type: int
Marks.studentId	Foreign key to Student. Type: int
Marks.teacherId	Foreign key to Teacher. Type: int
Marks.subject	Subject name. Type: String
Marks.exam	Exam/assessment name. Type: String
Marks.marksObtained	Marks obtained. Type: decimal
Marks.totalMarks	Total possible marks. Type: decimal
Marks.addMarks(marks:decimal)	Adds marks for student.
Marks.updateMarks(marks:decimal)	Updates existing marks.
Notification	Stores system notifications. Attributes: notificationId:int, userId:int (FK), type:String, content:String, dateSent:Date, status:enum {Unread, Read, Archived}
Notification.notificationId	Primary key. Type: int
Notification.userId	Recipient user. Type: int
Notification.type	Notification type (e.g., Info, Warning). Type: String
Notification.content	Message content. Type: Text
Notification.dateSent	Date sent. Type: Date
Notification.status	Read status. Type: enum
Notification.sendNotification(content:String)	Sends notification to user.
Notification.markRead()	Marks notification as read.

Parent	Represents a parent user. Attributes: parentId:int, userId:int (FK), studentId:int (FK), relation:String
Parent.parentId	Primary key. Type: int
Parent.userId	Foreign key to User. Type: int
Parent.studentId	Foreign key to Student. Type: int
Parent.relation	Relation to student (Father/Mother/Guardian). Type: String
Parent.viewStudentRecords()	Accesses linked student's academic records.
Portfolio	Compiled academic record. Attributes: portfolioId:int, studentId:int (FK), attendanceSummary:String, marksSummary:String, achievementsSummary:String, behaviorSummary:String, generatedOn:Date
Portfolio.portfolioId	Primary key. Type: int
Portfolio.studentId	Foreign key to Student. Type: int
Portfolio.attendanceSummary	Text summary of attendance. Type: String
Portfolio.marksSummary	Text summary of marks. Type: String
Portfolio.achievementsSummary	Text summary of achievements. Type: String
Portfolio.behaviorSummary	Text summary of behavior remarks. Type: String
Portfolio.generatedOn	Date generated. Type: Date
Portfolio.compileData()	Compiles portfolio data from different modules.
Portfolio.updatePortfolio()	Updates existing portfolio record.
Portfolio.exportPDF()	Exports portfolio as PDF.

Report	System-generated reports. Attributes: reportId:int, type:String, generatedOn:Date, filePath:String, adminId:int (FK)
Report.reportId	Primary key. Type: int
Report.type	Type of report (Attendance, Marks, Portfolio, Risk). Type: String
Report.generatedOn	Date generated. Type: Date
Report.filePath	File location. Type: String
Report.adminId	Generated by Admin. Type: int
Report.generateReport(type:String)	Creates a report file.
RiskMonitoring	Monitors student risk. Attributes: riskId:int, studentId:int (FK), riskLevel:String {Low, Medium, High}, monitoredOn:Date, notes:String
RiskMonitoring.riskId	Primary key. Type: int
RiskMonitoring.studentId	Foreign key to Student. Type: int
RiskMonitoring.riskLevel	Risk rating. Type: String
RiskMonitoring.monitoredOn	Date of assessment. Type: Date
RiskMonitoring.notes	Additional info. Type: String
RiskMonitoring.assessRisk()	Evaluates current risk level.
RiskMonitoring.updateRisk(level:String, notes:String)	Updates risk details.
Student	Represents student. Attributes: studentId:int, userId:int (FK), firstName:String, lastName:String, class:String, section:String, rollNumber:String
Student.studentId	Primary key. Type: int

Student.userId	Foreign key to User. Type: int
Student.firstName	First name. Type: String
Student.lastName	Last name. Type: String
Student.class	Class/grade. Type: String
Student.section	Section/group. Type: String
Student.rollNumber	Roll number. Type: String
Student.viewAttendance()	Views attendance records.
Student.viewMarks()	Views marks.
Student.viewPortfolio()	Accesses compiled portfolio.
Student.submitComplaint(description:String)	Submits complaint.
Teacher	Represents teacher. Attributes: teacherId:int, userId:int (FK), firstName:String, lastName:String, assignedSubjects:String
Teacher.teacherId	Primary key. Type: int
Teacher.userId	Foreign key to User. Type: int
Teacher.firstName	First name. Type: String
Teacher.lastName	Last name. Type: String
Teacher.assignedSubjects	Subjects teacher is responsible for. Type: String
Teacher.markAttendance(studentId:int, status:enum)	Marks attendance for a student.
Teacher.enterMarks(studentId:int, subject:String, marksObtained:decimal)	Enters marks for a student.
Teacher.addBehaviorRemark(studentId:int, remarkText:String, remarkType:String)	Adds behavior remark.

Teacher.reviewComplaint(complaintId:int, status:enum)	Updates complaint status.
Teacher.postNotice(title:String, content:String)	Posts a notice.
User	Generic system user. Attributes: userId:int, username:String, email:String, password:String (encrypted), role:enum {Admin, Teacher, Student, Parent}, isActive:boolean
User.userId	Primary key. Type: int
User.username	Login username. Type: String
User.email	User email. Type: String
User.password	Encrypted password. Type: String
User.role	User role. Type: enum
User.isActive	Account status. Type: boolean
User.login(email:String, password:String)	Authenticates user.
User.logout()	Logs out user.
User.updateProfile(updatedData:Object)	Updates profile data.
UserRole	Maps users to roles. Attributes: userRoleId:int, userId:int (FK), roleId:int (FK)
UserRole.userRoleId	Primary key. Type: int
UserRole.userId	Foreign key to User. Type: int
UserRole.roleId	Foreign key to Role. Type: int
Role	Defines a system role. Attributes: roleId:int, roleName:String, description:String
Role.roleId	Primary key. Type: int
Role.roleName	Name of role. Type: String

Role.description	Role description. Type: String
------------------	---------------------------------------

- **Structured Approach: Functions and Function Parameters:**

Function Name	Parameters	Description
login	email:String, password:String	Authenticates a user using email and password. Returns success/failure and role-based access.
logout	userId:int	Logs out the specified user, ending their session.
updateProfile	userId:int, updatedData:Object	Updates a user's profile with the provided data. updatedData can contain username, email, password, etc.
markAttendance	studentId:int, date:Date, status:enum {Present, Absent, Late}	Marks attendance for a student on a given date.
updateAttendance	attendanceId:int, status:enum {Present, Absent, Late}	Updates an existing attendance record.
lockAttendance	attendanceId:int	Locks attendance for a particular date to prevent further edits.
addMarks	studentId:int, subject:String, exam:String, marksObtained:decimal, totalMarks:decimal	Adds a new marks entry for a student for a specific exam and subject.
updateMarks	marksId:int, marksObtained:decimal	Updates marks for an existing record.

submitComplaint	submittedByUserId:int, assignedToUserId:int, description:String, category:String	Creates a new complaint record in the system.
updateComplaintStatus	complaintId:int, newStatus:enum {Submitted, UnderReview, Resolved, Rejected, Closed}	Updates the status of a complaint.
addComplaintRemarks	complaintId:int, remarks:String	Adds remarks/comments to an existing complaint.
compilePortfolio	studentId:int	Gathers student data (attendance, marks, achievements, behavior) to generate portfolio.
updatePortfolio	portfolioId:int, updatedData:Object	Updates an existing portfolio with new data.
exportPortfolioPDF	portfolioId:int	Generates a PDF document of a student's portfolio.
sendNotification	userId:int, content:String, type:String	Sends a notification to a user.
markNotificationRead	notificationId:int	Marks a notification as read.
recordAuditLog	userId:int, action:String, details:String	Records an action performed in the system for auditing purposes.
updateFeePayment	feeId:int, paidAmount:decimal	Updates the paid amount for a student fee record.
calculateFeeDue	feeId:int	Calculates the remaining due amount for a fee record.

- **Object-Oriented (OO) Approach: Objects, Attributes, Methods, & Method Parameters:**

Object: User

Attributes

Attribute	Type	Description
userId	int	Unique system identifier
username	String	Login username
email	String	User email
password	String	Encrypted password
role	enum {Admin, Teacher, Student, Parent}	User role
isActive	boolean	Account status

Methods

Method	Parameters	Description
login	email:String, password:String	Authenticates user and grants access
logout	—	Ends user session
updateProfile	updatedData:Object	Updates user's profile attributes

Object: Teacher

Attributes

Attribute	Type	Description
teacherId	int	Unique identifier
userId	int	FK → User
firstName	String	First name
lastName	String	Last name
assignedSubjects	String	Subjects taught

Methods

Method	Parameters	Description
markAttendance	studentId:int, status:enum {Present, Absent, Late}	Marks attendance for a student
enterMarks	studentId:int, subject:String, marksObtained:decimal	Records marks for a student
addBehaviorRemark	studentId:int, remarkText:String, remarkType:String	Adds behavioral remark
reviewComplaint	complaintId:int, status:enum	Updates complaint status
postNotice	title:String, content:String	Posts notice to students/parents

Object: Admin

Attributes

Attribute	Type	Description
-----------	------	-------------

adminId	int	Unique identifier
userId	int	FK → User
firstName	String	First name
lastName	String	Last name

Methods

Method	Parameters	Description
createUser	userData:Object	Creates a new system user
generateReport	reportType:String	Generates attendance, marks, portfolio, or risk report
updateSettings	settings:Object	Updates system settings

Object: Attendance

Attributes

Attribute	Type	Description
attendanceId	int	PK
studentId	int	FK → Student
teacherId	int	FK → Teacher
date	Date	Attendance date
status	enum {Present, Absent, Late}	Attendance status

Methods

Method	Parameters	Description
markAttendance	status:enum	Records attendance
updateAttendance	status:enum	Updates existing attendance
lockAttendance	—	Locks record

Object: Marks

Attributes

Attribute	Type	Description
marksId	int	PK
studentId	int	FK → Student
teacherId	int	FK → Teacher
subject	String	Subject name
exam	String	Exam name
marksObtained	decimal	Marks obtained
totalMarks	decimal	Maximum marks

Methods

Method	Parameters	Description
addMarks	marks:decimal	Adds marks for a student

updateMarks	marks:decimal	Updates marks
-------------	---------------	---------------

Object: Complaint

Attributes

Attribute	Type	Description
complaintId	int	PK
submittedByUserId	int	FK → User
assignedToUserId	int	FK → User
description	String	Complaint description
status	enum {Submitted, UnderReview, Resolved, Rejected, Closed}	Current status
submittedDate	Date	Date of submission
closedDate	Date	Date of closure

Methods

Method	Parameters	Description
submitComplaint	description:String	Submit new complaint
updateStatus	newStatus:enum	Update complaint status

addRemarks	remarks:String	Add remarks to complaint
------------	----------------	--------------------------

Object: StudentPortfolio

Attributes

Attribute	Type	Description
portfolioId	int	PK
studentId	int	FK → Student
generatedOn	Date	Generation date
lastUpdated	Date	Last update

Methods

Method	Parameters	Description
compileData	—	Compile portfolio data from attendance, marks, achievements, behavior
updatePortfolio	—	Update portfolio with new data
exportPDF	—	Export portfolio to PDF

Object: Achievement

Attributes

Attribute	Type	Description
achievementId	int	Unique identifier (PK)
studentId	int	FK → Student
title	String	Achievement title
category	String	Achievement category
date	Date	Date of achievement
filePath	String	Path to supporting document/file

Methods

Method	Parameters	Description
addAchievement	achievementData:Object	Adds a new achievement record for a student
updateAchievement	achievementId:int, updatedData:Object	Updates an existing achievement

Object: BehaviorRemark

Attributes

Attribute	Type	Description
remarkId	int	Unique identifier (PK)
studentId	int	FK → Student

teacherId	int	FK → Teacher
remarkText	Text	Remark content
remarkType	String	Type of remark (e.g., Positive, Negative, Neutral)
date	Date	Date the remark was recorded

Methods

Method	Parameters	Description
addRemark	text:String, type:String	Adds a new behavioral remark for a student
updateRemark	text:String	Updates an existing remark

Object: Fee

Attributes

Attribute	Type	Description
feeId	int	Unique identifier (PK)
studentId	int	FK → Student
totalAmount	decimal	Total fee amount
paidAmount	decimal	Amount already paid
dueDate	Date	Payment due date

status	enum {Unpaid, Partial, Paid, Overdue}	Current fee status
--------	---------------------------------------	--------------------

Methods

Method	Parameters	Description
updatePayment	amount:decimal	Updates payment for the student fee
calculateDue	—	Calculates remaining amount due

Object: Notification

Attributes

Attribute	Type	Description
notificationId	int	Unique identifier (PK)
userId	int	FK → User (recipient)
content	Text	Notification content
dateSent	Date	Date notification was sent
status	String	Status (e.g., Unread, Read)

Methods

Method	Parameters	Description
sendNotification	content:String	Sends notification to a user

markRead	—	Marks notification as read
----------	---	----------------------------

Object: AuditLog

Attributes

Attribute	Type	Description
logId	int	Unique identifier (PK)
userId	int	FK → User (who performed action)
action	String	Action description
timestamp	Timestamp	Time action was performed
details	Text	Additional details about the action

Methods

Method	Parameters	Description
recordAction	userId:int, action:String, details:String	Records an administrative or system action

3.5 User Interface Design

The system provides a **role-based interface**, tailored to each type of user: Admin, Teacher, Student, and Parent. After logging in, the user sees a **dashboard** that gives access only to features relevant to their role.

- **Admin:**

Can manage all users (students, teachers, parents)
Add/edit/delete records for fees, subjects, and classes

Generate reports (attendance, marks, portfolio, risk monitoring)

Review complaints and notifications

Audit logs to monitor system activity

- **Teacher:**

Record student attendance

Enter marks for exams and assignments

Add behavioral remarks

Upload student achievements

Review and resolve complaints

Post notices to students/parents

- **Student:**

View attendance, marks, and achievements

Access personal portfolio

Submit complaints

Receive notifications from teachers/admins

- **Parent:**

View their child's academic records (attendance, marks, portfolio)

Submit complaints regarding their child

User Feedback:

- The system provides **real-time feedback** for user actions:

Success messages (e.g., "Attendance successfully recorded")

Error messages (e.g., "Marks cannot exceed total marks")

Validation messages for input fields

Notifications for important events (new message, new complaint, fee due)

The interface is designed for **clarity, responsiveness, and ease of navigation**, ensuring users can complete tasks efficiently.

3.5.1 Screen Images

Login Screen

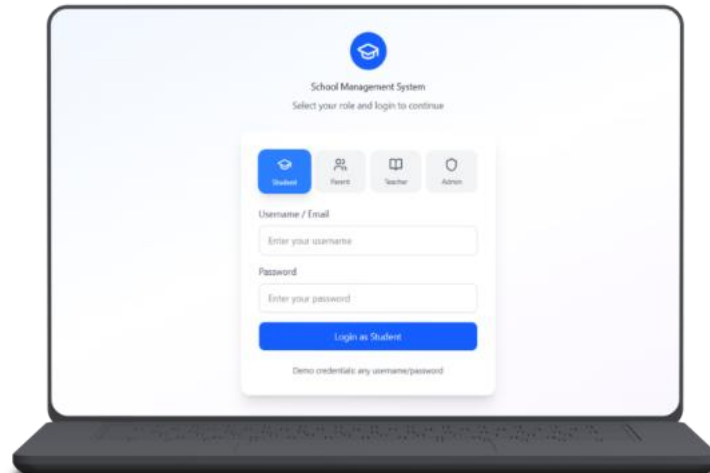


Figure 13 Screen Image 1- Login Screen

Student Portfolio Screen

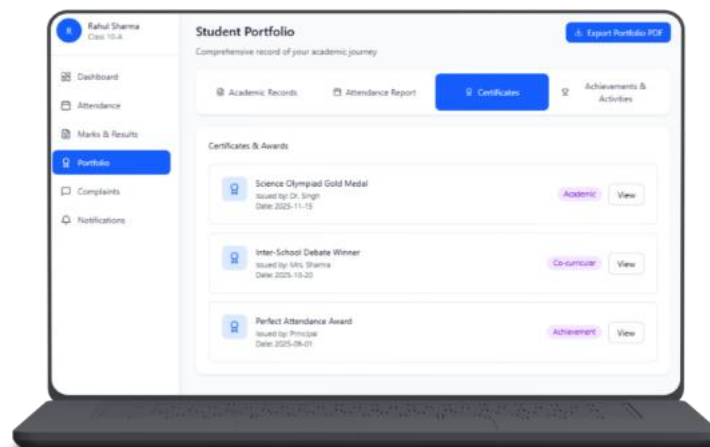


Figure 14 Screen Image 2- Student Portfolio

Attendance Management Screen(Teacher)

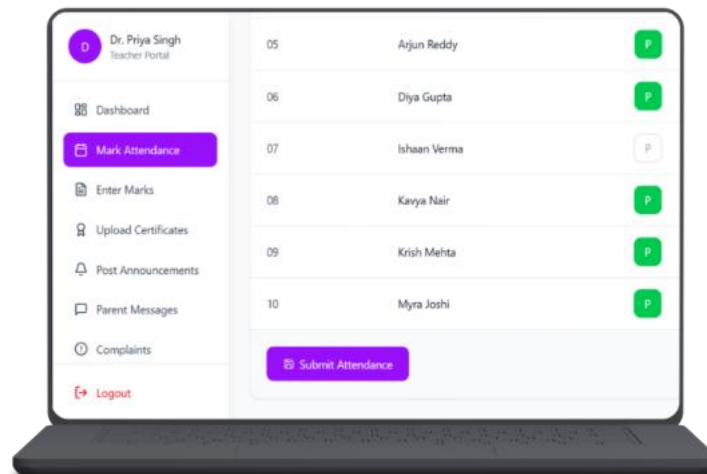


Figure 15 Screen Image 3- Attendance Management

Complaint Feedback Screen (Student)

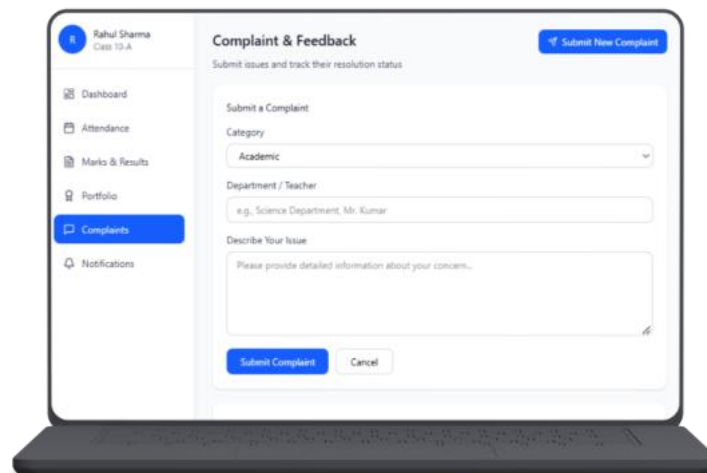


Figure 16 Screen Image 4- Complain Feedback

Student Portfolio Generation Screen

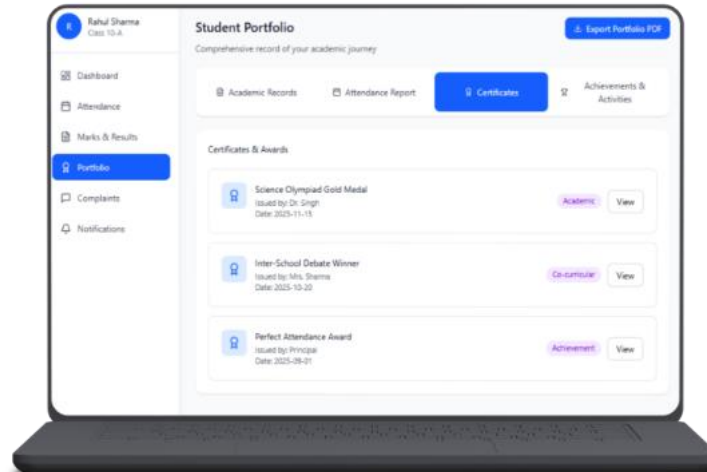


Figure 17 Screen Image 5- Student Portfolio Generation

3.5.2 Screen Objects and Actions

Use Case 1: Submit Complaint

- **Screen Objects:**
 - Complaint Text Area → where user types the complaint
 - Submit Button → submits complaint to system
 - Status Message Label → shows success/error
- **Actions:**
 - User enters complaint text
 - User clicks Submit
 - System validates input
 - System saves complaint
 - Status message shows confirmation

Use Case 2: Mark Attendance

- **Screen Objects:**
 - Student List Table → shows students' names
 - Attendance Controls → dropdown or radio buttons: Present / Absent / Late
 - Submit Button → saves attendance
 - Message Area → shows success/error

- **Actions:**
 - Teacher selects status for each student
 - Teacher clicks Submit
 - System validates input
 - System updates attendance records
 - Success message appears

3.4 Behavioural Model

Interaction Diagram (sequence)

User Flow login – Sequence Diagram

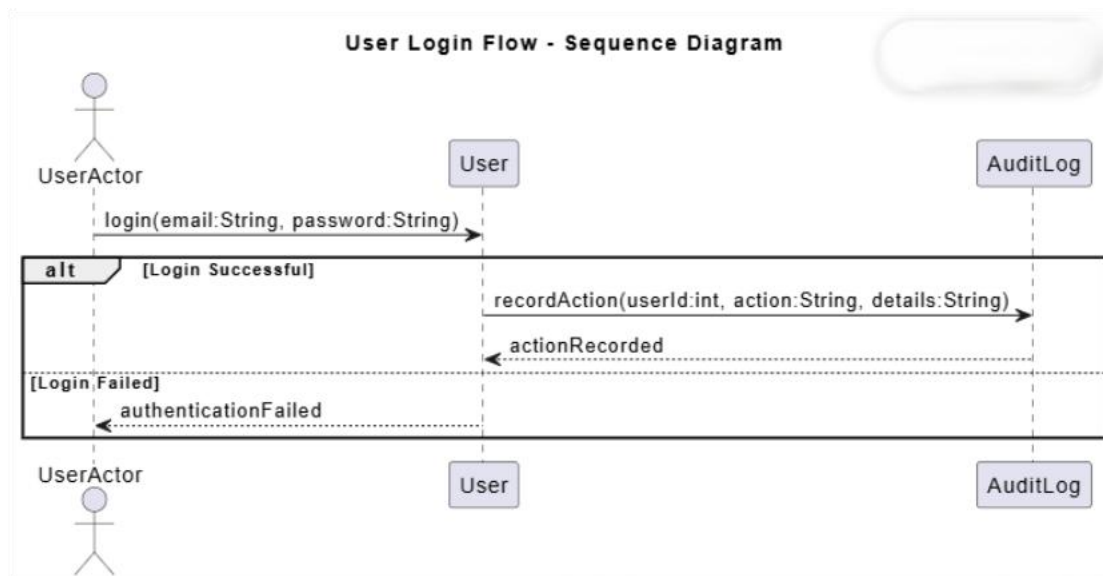


Figure 18 Sequence Diagram 1- User Flow Login

Fig explanation: This figure illustrates the user authentication process, where login credentials are validated and successful login activities are recorded through the audit logging service. It ensures secure access and accountability within the system.

Teacher Academic Management Flow – Sequence Diagram

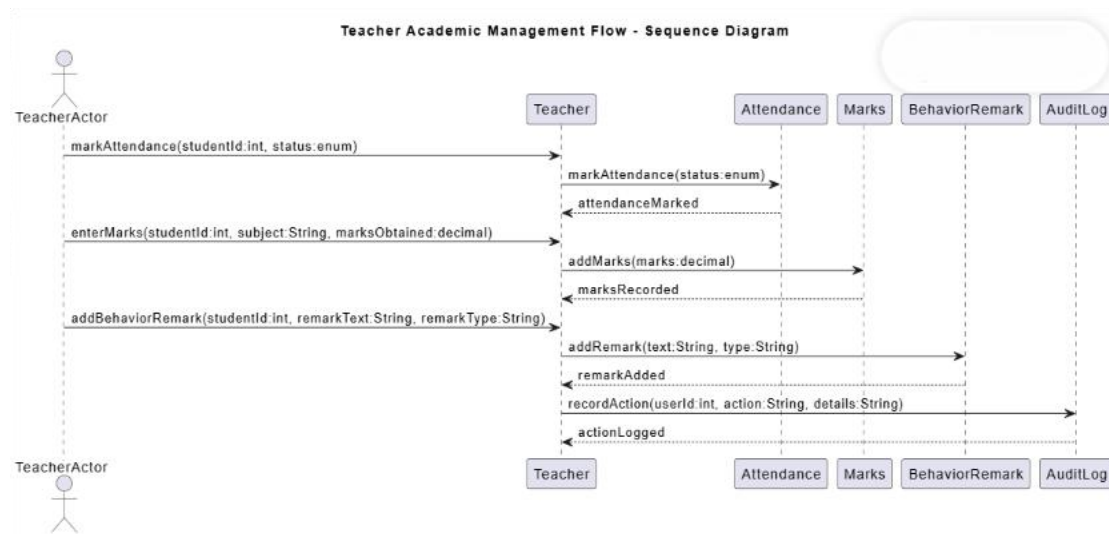


Figure 19 Sequence Diagram 2- Teacher Academic Management Flow

Fig explanation: This figure shows how a teacher manages academic activities, including marking attendance, entering marks, and adding behavior remarks. Each action is logged to maintain data integrity and accountability.

Student Portfolio Generation Flow – Sequence Diagram

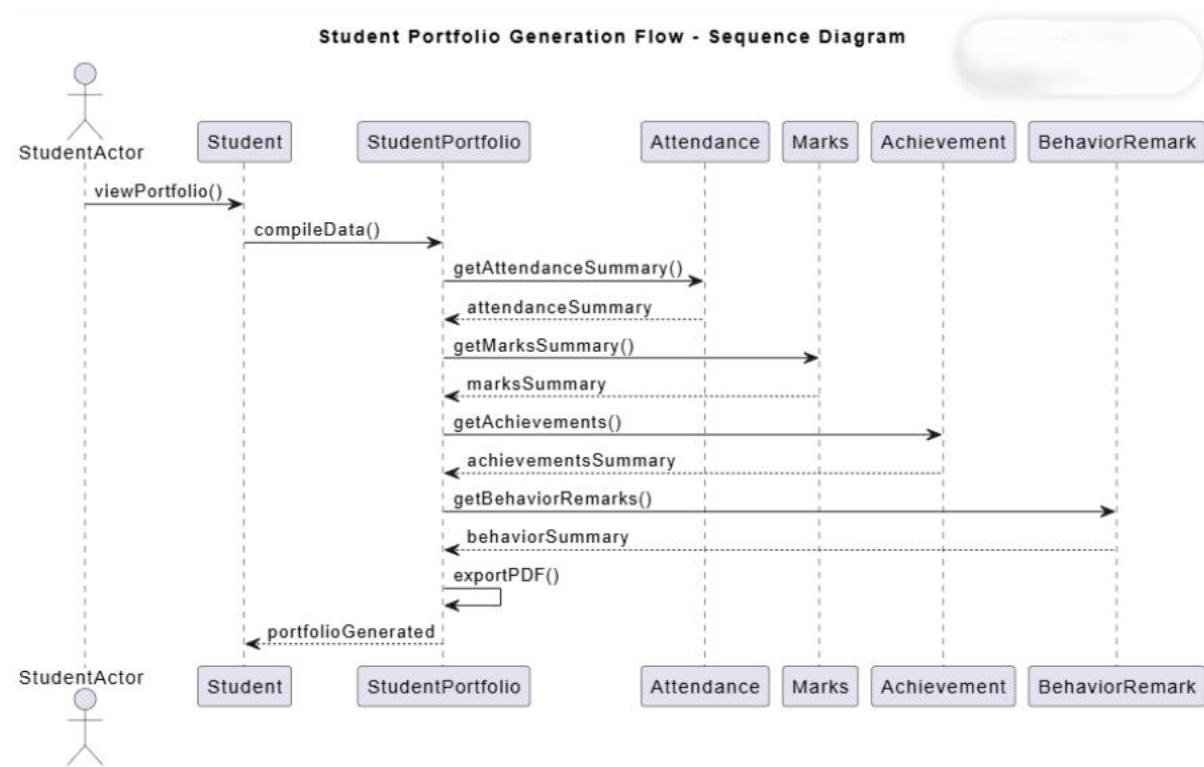


Figure 20 Sequence Diagram 3- Student Portfolio Generation Flow

Fig explanation: This figure depicts the process of generating a student portfolio by aggregating attendance, marks, achievements, and behavior records. The compiled data is then exported as a complete academic performance report.

State Transition Diagram

Attendance Record State Diagram

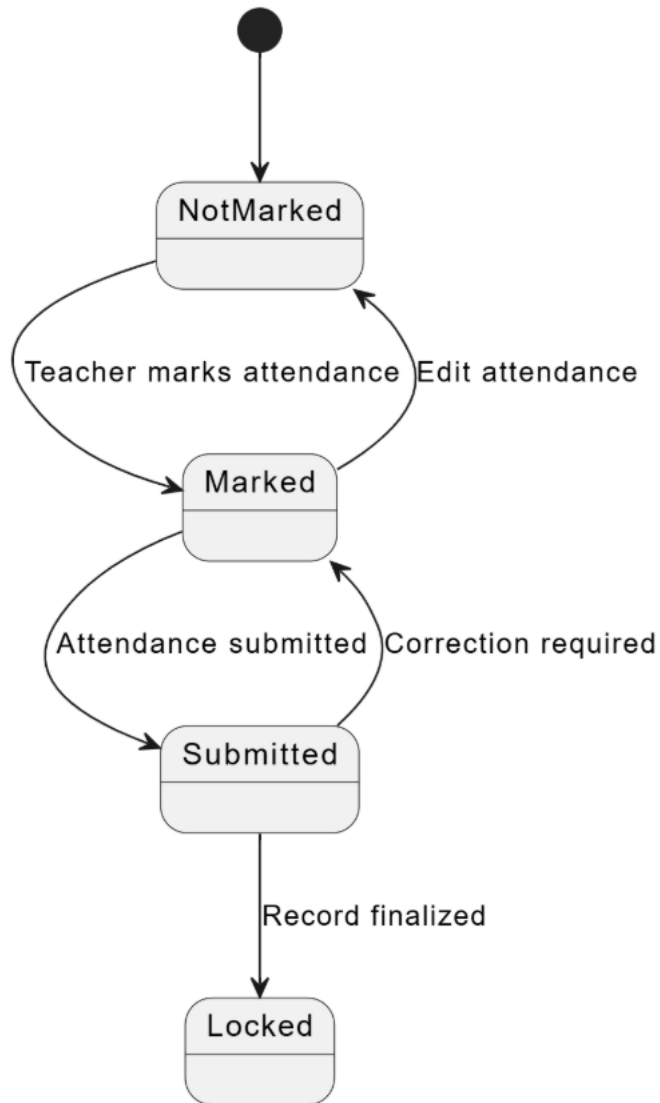


Figure 21 State Diagram 1- Attendance Record

Fig explanation: This state diagram models the attendance recording process. Attendance starts unmarked, is entered by the teacher, submitted for confirmation, and finally locked to prevent unauthorized changes.

Complaint Management State Diagram

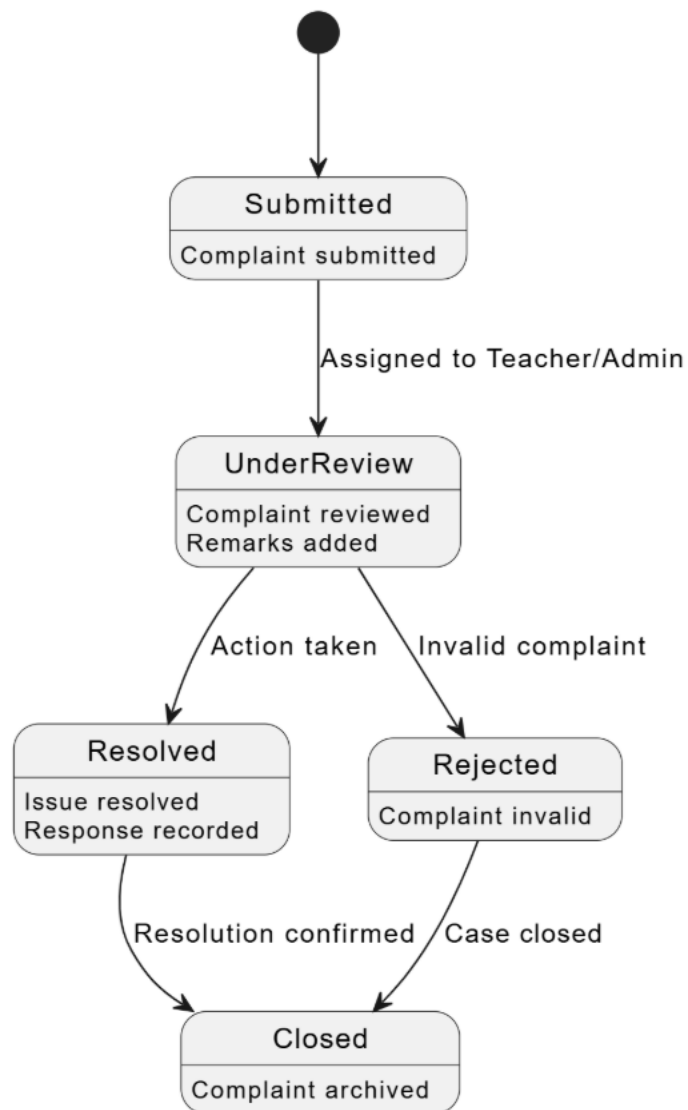


Figure 22 State Diagram 2- Complain Mangement

Fig explanation: This diagram illustrates the complaint handling workflow. A complaint is submitted by a student or parent, reviewed by staff, resolved or rejected, and finally closed after confirmation.

Student Portfolio State Diagram

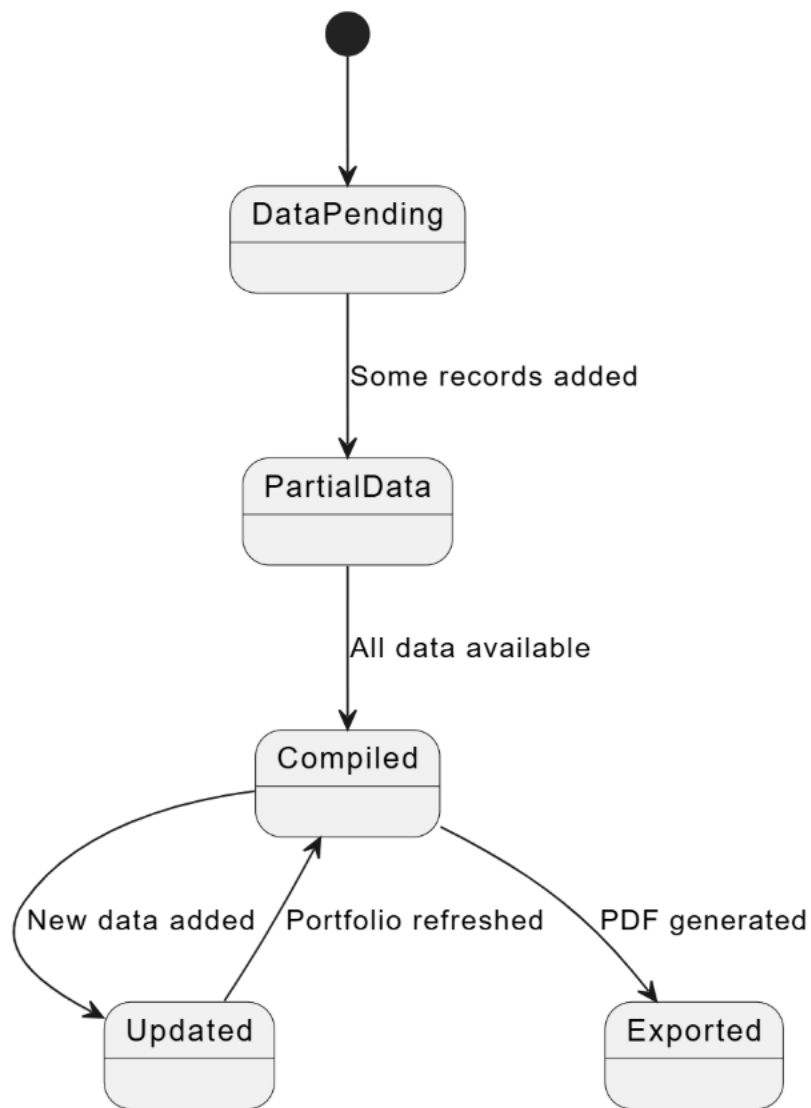


Figure 23 State Diagram 3- Student Portfolio

Fig explanation: This diagram represents the automated student portfolio generation process. The system collects academic data, compiles it into a portfolio, updates it when new data arrives, and allows export in PDF format.

Fee payment State Diagram

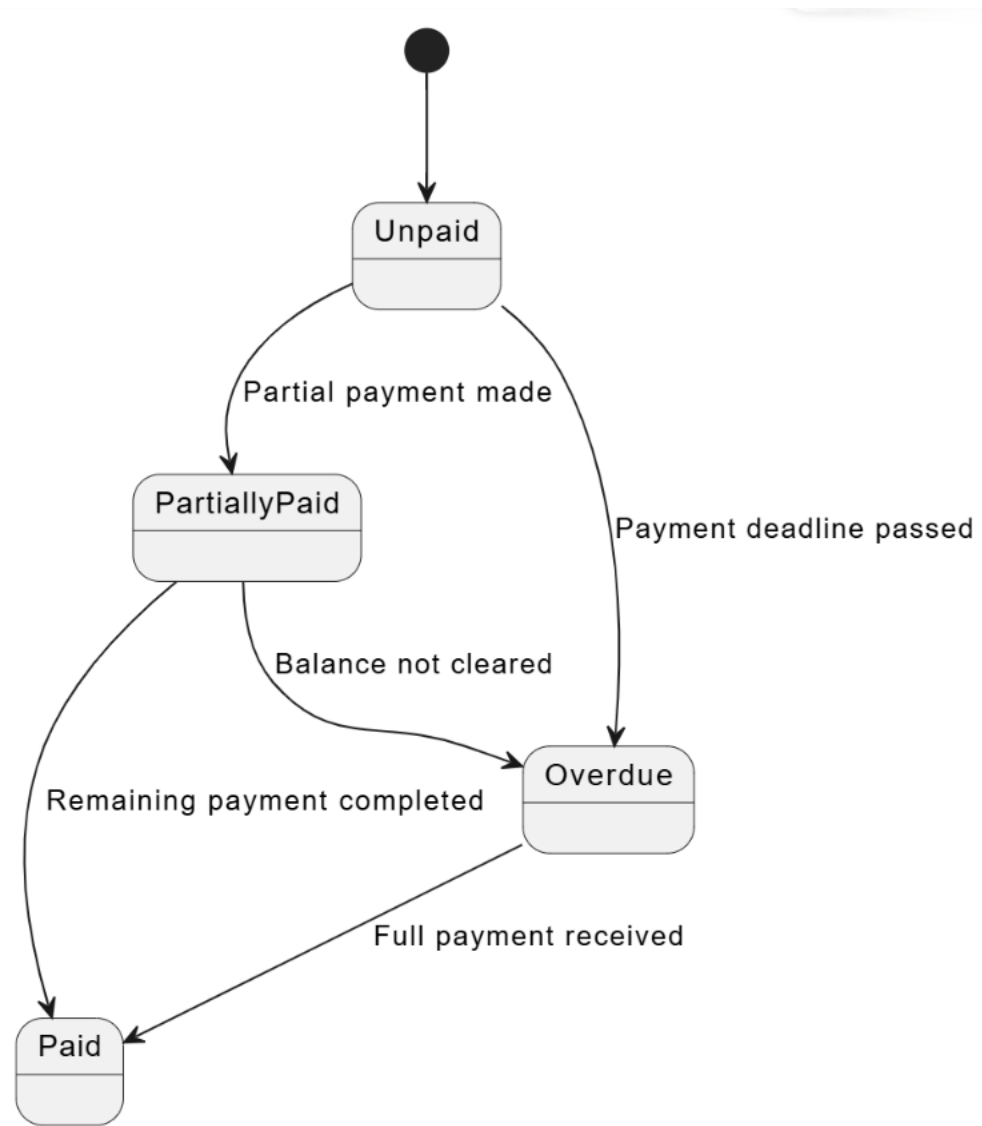


Figure 24 State Diagram 4- fee Payment

Fig explanation: This diagram shows the fee payment lifecycle of a student. It tracks payment status from unpaid to partially paid, overdue, and fully paid.

User Account State Diagram

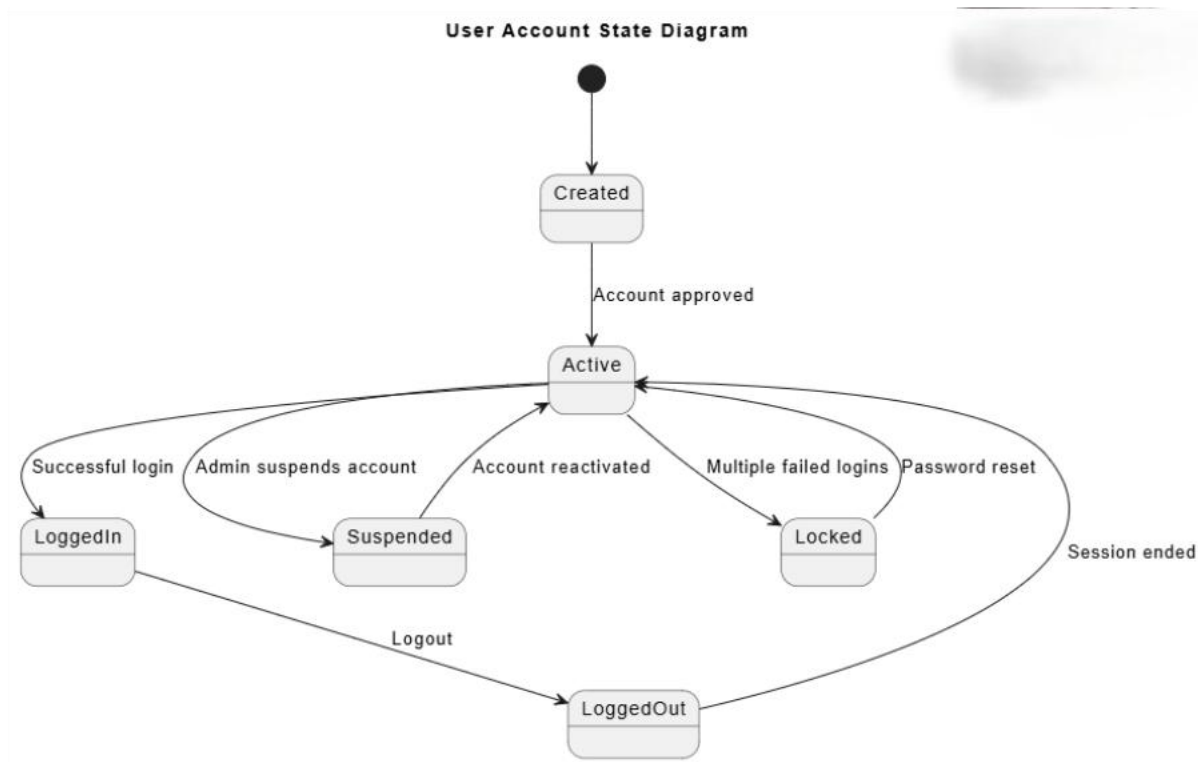


Figure 25 State Diagram 5- User account

Fig explanation: This state diagram represents the lifecycle of a user account in the system. It shows transitions from account creation to activation, login/logout behavior, and administrative actions such as suspension, locking, and reactivation.

3.5 Design Decisions

This section describes the major design decisions taken during the development of the **School Management System (SMS)**. Each decision is aligned with the requirements specified in the SRS and justified based on system efficiency, maintainability, scalability, and security.

1. Design Paradigm

Decision: Object-Oriented Design (OOD)

Description:

The system is designed using an object-oriented approach where real-world entities such as User, Student, Teacher, Parent, Attendance, Marks, and Fee are modeled as objects with attributes and behaviors.

Justification:

1. Improves modularity and reusability
2. Simplifies maintenance and future enhancement
3. Supports UML-based modeling (Class and Sequence Diagrams)

2. Architectural Style

Decision: Layered (Multi-Tier) Architecture

Description:

The system follows a layered architecture consisting of Presentation, Application, Data Access, and Database layers.

Justification:

1. Ensures separation of concerns
2. Allows independent modification of layers
3. Enhances scalability and testability

3. User and Role Management

Decision: Role-Based Access Control (RBAC)

Description:

The system supports four predefined roles: Admin, Teacher, Student, and Parent, as defined in the SRS.

Justification:

- Ensures secure and controlled access
- Prevents unauthorized operations
- Simplifies permission management

4. Database Design Approach

Decision: Relational Database Design

Description:

A relational database is used to store persistent system data.

Justification:

- Ensures structured data storage
- Maintains referential integrity
- Supports transactional consistency

5. Normalization Level

Decision: Third Normal Form (3NF)

Description:

The database schema is normalized up to Third Normal Form.

Justification:

1. Reduces redundancy
2. Prevents update anomalies
3. Improves data consistency

6. Object–Relational Mapping Strategy

Decision: Logical Mapping between Classes and Tables

Description:

Each UML class corresponds to a relational database table.

Justification:

1. Maintains consistency between design and implementation
2. Simplifies CRUD operations
3. Improves traceability between ER and Class Diagrams

7. Algorithmic Approach

Decision: Rule-Based and Transaction-Oriented Logic

Description:

Business processes such as attendance management, fee calculation, and complaint handling use rule-based logic.

Justification:

1. Avoids unnecessary computational complexity
2. Enhances readability and maintainability
3. Suitable for academic management systems

8. Behavioral Modeling Strategy

Decision: Use-Case Driven Sequence Modeling

Description:

Sequence diagrams are derived from major SRS use cases, particularly **User Login & Role-Based Access**.

Justification:

1. Login is the primary system entry point
2. Demonstrates interaction between multiple system components
3. Ensures consistency with SRS workflows

9. Security Design

Decision: Application-Level Security Enforcement

Description:

Security is enforced through encrypted password storage, role validation, and audit logging.

Justification:

- Protects sensitive academic and personal data
- Ensures accountability
- Prevents unauthorized access

10. Modularity and Component Design

Decision: Component-Based Modular Design

Description:

The system is divided into logical components such as User Management, Academic Records, Complaints & Fees, and System Services.

Justification:

1. Improves system organization
2. Enhances maintainability and scalability

3. Aligns with UML Component Diagram

11. Traceability and Documentation

Decision: Strict SRS-to-SDS Traceability

Description:

All design artifacts are derived directly from the SRS.

Justification:

- Ensures requirement coverage
- Prevents scope creep
- Supports academic validation

12. Technology Stack Selection

Decision: ASP.NET Core, React.js, and SQL Server

Description:

The backend of the system is implemented using **ASP.NET Core Web API**, the frontend using **React.js**, and **SQL Server** is used as the relational database.

Justification:

1. Supports scalable web-based applications
2. Enables secure and high-performance backend services
3. Aligns with industry standards and university curriculum
4. Facilitates maintainable and modular development

13. Communication Strategy

Decision: RESTful API Communication (JSON over HTTP)

Description:

The system follows a client–server model where the frontend communicates with the backend through RESTful APIs using JSON.

Justification:

1. Decouples frontend and backend components
2. Enables stateless communication
3. Simplifies testing and system integration

14. Error Handling and Validation Strategy

Decision: Server-Side and Client-Side Validation

Description:

Input validation and error handling are implemented at both client and server levels.

Justification:

- Client-side validation improves user experience
- Server-side validation ensures data integrity and security
- Prevents invalid or malicious data submission

3.6 Summary

The Software Design Specification (SDS) for the School Management System defines the **architectural design, component structure, data organization, and interaction mechanisms** of the proposed system. The design adopts a **modular and layered approach** to ensure scalability, maintainability, and role-based access control. Core system functionalities are divided into **user management, academic records, finance and complaints handling, and system services such as notifications and audit logging**. UML diagrams, including **class and sequence diagrams**, are used to clearly represent static structures and dynamic behavior of the system. The SDS provides a strong foundation for implementation by ensuring consistency with system requirements, promoting code reusability, and supporting future system enhancements.

Chapter 4

Implementation

4 Implementation

This chapter describes the implementation of the Web-Based School Management System (SMS), with a particular emphasis on its key distinguishing feature: **an AI-based Student Risk Prediction module**. The system follows a modular, **three-tier architecture** comprising a React.js (Vite) frontend, an ASP.NET Core Web API backend, and a SQL Server relational database managed through Entity Framework Core. Machine learning capabilities are delivered via **ML.NET**, which enables multiclass classification of students into risk categories (Low, Medium, High) based on academic and behavioral data.

The implementation is organized into independent, loosely coupled modules, each handling a distinct functional area of the school management system. The **core modules** implemented are: Attendance Management, Marks Management, AI Risk Prediction, Complaint Management, and Student Dashboard. Each module has dedicated frontend components, backend controllers, and database entities, ensuring maintainability, testability, and clear separation of concerns.

Key Technologies Used:

- 1. **Frontend:** React.js 18 with Vite build tool and TypeScript (.tsx components)
- 2. **Backend:** ASP.NET Core 8.0 Web API with JWT-based authentication
- 3. **Database:** SQL Server (SQLEXPRESS) with Entity Framework Core 8.0
- 4. **Machine Learning:** ML.NET v3.0 — SDCA Maximum Entropy multiclass classifier
- 5. **UI Styling:** Tailwind CSS with Lucide React icons and Recharts for data visualization

4.1 Algorithm

Table 18 Algorithm 1: Rule-Based Student Risk Classification

Algorithm 1 Rule-Based Student Risk Classification
Input: attendanceRate (%), averageMarks (%), behaviorRemarks records
Output: riskLevel {"High", "Medium", "Low"}
1: attendanceRate $\leftarrow$ (presentDays / totalSchoolDays) $\times$ 100 2: averageMarks $\leftarrow$ average(marksObtained / totalMarks $\times$ 100) for each marks entry 3: negativeRemarkCount $\leftarrow$ COUNT(behaviorRemarks WHERE remarkType = 'Negative') 4: behaviorScore $\leftarrow$ 100 – (negativeRemarkCount $\times$ 10)

```

5: if (attendanceRate < 50) OR (averageMarks < 35) OR (negativeRemarkCount >= 5) then
6:   riskLevel ← 'High'
7: else if (attendanceRate < 75) OR (averageMarks < 55) OR (negativeRemarkCount >= 3) then
8:   riskLevel ← 'Medium'
9: else
10:  riskLevel ← 'Low'
11: end if
12: return riskLevel

```

Time Complexity: $O(n)$ — where n is the total number of attendance and marks records for the student. Each metric is computed in a single linear pass over indexed database records.

Table 19 *Algorithm 2: ML.NET SDCA Classifier — Training Phase*

Algorithm 2 ML.NET SDCA Multiclass Classifier — Training Phase
Input: trainingRecords: List<{AttendanceRate, AverageMarks, BehaviorScore, Label}>
Output: Trained ML.NET model saved to risk_model.zip
<pre> 1: if studentCount >= 10 then 2: trainingData ← RiskService.ComputeAllAsync() // real DB records 3: else 4: trainingData ← GenerateSeedData() // 18 labelled samples 5: end if 6: pipeline ← 7: MapValueToKey(Label: string key) 8: Concatenate([AttendanceRate, AverageMarks, BehaviorScore] Features) 9: SdcaMaximumEntropy(labelColumn=Label, featureColumn=Features) 10: MapKeyToValue(PredictedLabel string) 11: trainedModel ← pipeline.Fit(trainingData) 12: Save trainedModel risk_model.zip </pre>

13: **return** trainedModel

Time Complexity: $O(n \times d \times i)$ — where n = number of training samples, $d = 3$ features, and i = number of SDCA optimization iterations.

Table 20 Algorithm 3: ML.NET Risk Prediction — Inference Phase

Algorithm 3 ML.NET Risk Prediction — Inference Phase
Input: studentId (integer)
Output: JSON: {ruleBasedRisk, mlRisk, finalRisk, attendanceRate, averageMarks, factors}
<pre>1: ruleBasedResult ← RiskService.ComputeAsync(studentId) 2: mlPrediction ← ruleBasedResult.RiskLevel // default fallback 3: if ModelTrainer.IsModelLoaded = true then 4: input ← {AttendanceRate, AverageMarks, BehaviorScore} 5: mlOutput ← predictionEngine.Predict(input) 6: if mlOutput null then 7: mlPrediction ← mlOutput.PredictedRisk 8: end if 9: end if 10: finalRisk ← mlPrediction // ML takes precedence 11: return {ruleBasedRisk, mlRisk, finalRisk, factors, mlModelUsed}</pre>

Time Complexity: $O(1)$ for inference — a single forward pass through the trained model's weight matrix. $O(n)$ for the preceding RiskService.ComputeAsync() call over indexed student records.

Table 21 Algorithm 4: Mark Student Attendance

Algorithm 4 Mark Student Attendance
Input: studentId, teacherId, date, status {Present(0), Absent(1), Leave(2)}

Output: 201 Created with attendanceId, or 409 Conflict if duplicate

```

1: validate studentId > 0 AND teacherId > 0
2: if student NOT FOUND OR teacher NOT FOUND then
3:   return 404 Not Found
4: end if
5: existing ← QUERY Attendances WHERE studentId = input.studentId
6:           AND date.Date = input.date.Date
7: if existing null then
8:   return 409 Conflict {message, existingAttendanceId}
9: end if
10: record ← {StudentId, TeacherId, Date, Status, IsLocked = false}
11: INSERT record into Attendances
12: return 201 Created {attendanceId}

```

Time Complexity: $O(1)$ — primary-key and composite-index lookup on (studentId, date). Bulk class marking scales as $O(n)$ where n is the number of students.

Table 22 *Algorithm 5: Enter and Calculate Student Marks*

Algorithm 5 Enter and Calculate Student Marks

Input: studentId, teacherId, subject, examType, marksObtained, totalMarks

Output: Saved marks record with computed percentage and letter grade

```

1: validate marksObtained totalMarks AND totalMarks > 0
2: if validation fails then
3:   return 400 Bad Request
4: end if
5: percentage ← (marksObtained / totalMarks) × 100
6: if percentage ≥ 90 then grade ← 'A+'

```

```

7: else if percentage >= 80 then grade ← 'A'
8: else if percentage >= 70 then grade ← 'B+'
9: else if percentage >= 60 then grade ← 'B'
10: else if percentage >= 50 then grade ← 'C'
11: else grade ← 'F'
12: INSERT {studentId, teacherId, subject, examType, marksObtained, totalMarks, percentage, grade}
13: return 201 Created {marksId, percentage, grade}

```

Time Complexity: $O(1)$ per record insertion. Class-average calculation over m records is $O(m)$.

Table 23 Algorithm 6: Complaint Submission and Status Lifecycle

Algorithm 6 Complaint Submission and Status Lifecycle
Input: submitterId, category, description, assignedToUserId; OR complaintId, newStatus, remarks
Output: 201 Created (submission) or 200 OK (status update)
<pre> 1: // — Submission — 2: validate description NOT empty 3: complaint ← {submittedByUserId, category, description, 4: status = Submitted, dateSubmitted = NOW()} 5: INSERT complaint into Complaints 6: return 201 Created {complaintId} 7: 8: // — Status Update — 9: complaint ← FIND Complaints WHERE complaintId = id 10: if complaint NOT FOUND then return 404 Not Found 11: if currentUser.Role = 'Teacher' AND complaint.assignedToUserId = currentUser then 12: return 403 Forbidden 13: end if </pre>

```

14: complaint.Status ← newStatus
15: if newStatus ∈ {Resolved, Closed} then complaint.DateClosed ← NOW()
16: SAVE changes
17: return 200 OK {updatedStatus}

```

Time Complexity: $O(1)$ — single primary-key lookup for status updates; $O(1)$ for new submissions.

Table 24 *Algorithm 7: Teacher Dashboard Statistics Aggregation*

Algorithm 7 Teacher Dashboard Statistics Aggregation
Input: currentUserId (extracted from JWT token)
Output: JSON statistics: {totalStudents, todayAttendance, pendingComplaints, activeNotices, behaviorRemarks}
<pre> 1: teacher ← FIND Teachers WHERE teacher.UserId = currentUserId 2: if teacher NOT FOUND then return 404 Not Found 3: today ← currentUTCDate 4: totalStudents ← COUNT(DISTINCT studentId) FROM Attendances WHERE teacherId = teacher.TeacherId 5: todayAttendance ← COUNT(*) FROM Attendances WHERE teacherId = teacher.TeacherId AND date = today 6: pendingComplaints ← COUNT(*) FROM Complaints WHERE assignedToUserId = currentUserId 7: AND status NOT IN {Resolved, Closed} 8: activeNotices ← COUNT(*) FROM Notices WHERE postedByUserId = currentUserId AND isActive = true 9: behaviorRemarks ← COUNT(*) FROM BehaviorRemarks WHERE teacherId = teacher.TeacherId 10: return {totalStudents, todayAttendance, pendingComplaints, activeNotices, behaviorRemarks} </pre>

Time Complexity: $O(n)$ overall — each COUNT query runs over indexed foreign-key columns; dominated by the largest relevant table scan.

4.2 External APIs/SDKs

The following table lists all third-party libraries, frameworks, SDKs, and external tools integrated into the School Management System. Each entry describes its version, purpose within the project, and the specific system layer or component in which it is used.

Table 25 External API's used in school management system

Name & Version	Description	Purpose	Where Used
ML.NET v3.0	Open-source machine learning framework by Microsoft for .NET applications.	Multiclass classification for student risk prediction using the SDCA Maximum Entropy algorithm.	SMS_Backend / ML namespace (ModelTrainer, AIController)
ASP.NET Core v8.0	Cross-platform, high-performance framework for building RESTful Web APIs.	Provides the backend API layer for all system modules including authentication and data operations.	All backend Controllers and Program.cs
Entity Framework Core v8.0	Object-Relational Mapper (ORM) for .NET that enables database operations using C# objects.	Data access layer — queries, inserts, and updates to SQL Server without writing raw SQL.	ApplicationDbContext, all repository operations
SQL Server (SQLEXPRESS)	Relational database management system used to	Stores student, attendance, marks, complaints,	Database layer (SchoolManagementDB)

Name & Version	Description	Purpose	Where Used
	store all persistent application data.	and risk monitoring records.	
React.js v18 (Vite)	JavaScript UI library for building component-based, reactive single-page applications.	Renders all frontend views: dashboards, attendance, marks, AI risk panel, and complaint management.	All frontend components (.tsx)
Lucide React	Lightweight, consistent icon library for React.	Provides all UI icons including status indicators, navigation, and action buttons.	All frontend .tsx components
React Router v6	Declarative routing library for React single-page applications.	Handles navigation between modules: Dashboard, Attendance, Marks, AI Risk, Complaints, etc.	App.tsx routing configuration
Tailwind CSS	Utility-first CSS framework for rapid UI styling.	Styles all frontend components with responsive layout, color	All frontend components (className props)

Name & Version	Description	Purpose	Where Used
		themes, and typography.	
JWT (System.IdentityModel.Tokens.Jwt)	JSON Web Token library for secure, stateless authentication.	Issues and validates authentication tokens for role-based access (Admin, Teacher, Parent, Student).	AuthController, Program.cs middleware
Recharts	Composable charting library built on React components.	Visualizes AI risk distribution, attendance trends, and marks analytics on dashboards.	AI Risk Dashboard, Admin analytics components

4.3 Code Repository

Version control for the School Management System project was managed using **Git**, an industry-standard distributed version control system. All source code — including backend controllers, ML model training logic, frontend React components, and database migration scripts — was maintained within a centralized **Git repository** hosted on **GitHub**. This approach provided a complete and auditable history of every change made to the codebase throughout the development lifecycle.

Collaboration between team members was facilitated through a structured branching strategy. The repository maintained two permanent branches: **main** (production-ready, stable code) and **develop** (integration branch for completed features). Each new feature or module was developed in isolation on a dedicated feature branch — for example, `feature/ai-risk-prediction`, `feature/attendance-module`, or `feature/complaint-management` — and merged into **develop** upon completion and successful review. This approach ensured that in-progress work did not disrupt the stability of the main branch.

Code quality was enforced through mandatory pull requests (PRs) for all merges. Each PR required at least one peer review before approval, ensuring that all contributions met the team's coding standards and did not introduce regressions. Issues and task assignments were tracked directly within the GitHub Issues system, providing full traceability from a feature request or bug report through to its resolution in the codebase.

Git Repository Link: <https://github.com/h-saey/school-management-system-fydp>

4.3.1 Metrics of Git Repository

The following metrics provide a quantitative and qualitative summary of the project's version control activity, reflecting the team's development discipline and collaboration practices.

Commits

A commit represents a saved snapshot of changes to the codebase at a specific point in time. Each commit in this project was accompanied by a descriptive message following a conventional format (e.g., feat: add ML.NET risk prediction pipeline, fix: resolve attendance duplicate bug). The total number of commits reflects the frequency and granularity of development activity across all team members.

Branches

Branches are isolated lines of development within the repository. This project maintained a feature-branch workflow with dedicated branches per module (e.g., feature/ai-risk-prediction, feature/marks-management). The total branch count, including merged and active branches, reflects the number of distinct development workstreams undertaken during the project.

Pull Requests (PRs)

A pull request is a formal mechanism for proposing and reviewing code changes before merging them into a shared branch. All merges to the develop and main branches were performed exclusively through pull requests. The PR count equals the number of completed feature integrations, and the merge rate (approved PRs / total PRs) reflects the team's review effectiveness.

Issues

GitHub Issues were used to track both planned features (as user stories) and discovered defects (as bug reports). Each issue was assigned a label (e.g., enhancement, bug, documentation), a milestone, and a responsible team member. The ratio of opened to closed issues provides a measure of project completion at any given point in development.

Contributors

Contributors are the team members who made at least one commit to the repository. The contributor count, along with each member's commit volume and lines of code changed, reflects individual participation and workload distribution across the team.

Code Reviews

Code reviews were conducted as part of the pull request approval process. Each review involved examining the proposed code for correctness, adherence to coding standards, security concerns, and overall design quality. The number of review comments and revision cycles per PR serves as a metric of code quality scrutiny applied before any feature was accepted into the shared codebase.

4.4 Summary

This chapter presented a comprehensive account of the **implementation** of the Web-Based School Management System, covering all major functional modules and the technical mechanisms that underpin them. The modular architecture — built on **React.js, ASP.NET Core, Entity Framework Core, and SQL Server** — ensures that each system component is independently maintainable, testable, and extensible.

The implementation successfully addresses all **core project objectives** defined in the requirements phase. Attendance management, marks recording, complaint handling, and behavior tracking are fully operational and integrated with the backend API layer. The dashboard aggregation algorithms provide meaningful real-time summaries for both teachers and administrators, improving operational visibility across the school.

The most **significant contribution** of this implementation is the AI-Based Student Risk Prediction system, which represents a substantive advancement beyond the capabilities of conventional school management platforms. By combining a robust rule-based classification engine with a trained ML.NET multiclass model (SDCA Maximum Entropy), the system provides reliable, interpretable, and continuously improving risk assessments. The dual-strategy design — where the ML model takes precedence when available and the rule-based engine serves as a reliable fallback — ensures that the system remains functional even before sufficient training data has been accumulated. This feature directly supports the project's objective of enabling proactive, data-driven academic intervention for at-risk students.

Together, these implementations demonstrate the successful application of **modern full-stack web development** and applied machine learning techniques to solve a real-world educational management challenge.

Chapter 5

Testing and Evaluation

5 Introduction

This chapter presents the **testing and evaluation** of the Web-Based School Management System to ensure that the developed system meets the specified functional and non-functional requirements. Software testing plays a critical role in verifying that each component of the system performs correctly, integrates seamlessly, and delivers reliable performance under different conditions. The testing process in this project follows a structured approach aligned with standard software engineering practices and university guidelines.

To achieve comprehensive validation, the system is evaluated through four key levels of testing: **Unit Testing (UT)**, **Functional Testing (FT)**, **Integration Testing (IT)**, and **Performance Testing (PT)**. Unit testing focuses on validating individual components such as controller actions and service methods. Functional testing ensures that complete workflows meet user and system requirements. Integration testing verifies the interaction between different modules, particularly the coordination between backend services and the AI-based risk prediction system. Performance testing evaluates system responsiveness, stability, and efficiency under realistic usage scenarios.

Special emphasis is placed on the **AI-Based Student Risk Prediction module**, which represents a core innovation of the system. Its testing ensures accurate predictions, reliable fallback mechanisms, and seamless integration with real-time data. Overall, this chapter demonstrates that the system has been rigorously tested to ensure correctness, reliability, and readiness for deployment.

5.1 Unit Testing (UT)

Unit testing is a level of software testing where individual units of a software component are tested. The purpose is to validate that each unit of the software performs as designed. In this system, unit tests target individual controller actions, service methods, and business logic functions in the ASP.NET Core backend, as well as utility functions in the React frontend components.

Table 26 *UT01 — User Login with Valid Credentials*

Testcase ID	UT01
Requirement ID	FR-AUTH-01
Title	User Login with Valid Credentials

Description	Test that a registered user can log in successfully with correct email and password.
Objective	To verify that the authentication endpoint issues a valid JWT token for recognised users.
Driver/precondition	User account (Teacher role) exists in the Users table. ASP.NET Core API is running.
Test steps	1. Send POST /api/auth/login with valid email and password. 2. Inspect the HTTP response code and body.
Input	{ email: 'teacher@school.com', password: 'Test@1234' }
Expected Results	HTTP 200 OK. Response body contains a JWT token field with a non-empty string value.
Actual Result	Pass
Remarks	Token is returned and can be decoded to confirm role = 'Teacher'.

Table 27 UT02 — User Login with Invalid Password

Testcase ID	UT02
Requirement ID	FR-AUTH-02
Title	User Login with Invalid Password
Description	Test that the system rejects a login attempt made with a correct email but an incorrect password.
Objective	To verify that authentication fails gracefully and does not expose sensitive information.
Driver/precondition	User account exists. API is running.
Test steps	1. Send POST /api/auth/login with correct email and a wrong password. 2. Check the response.
Input	{ email: 'teacher@school.com', password: 'WrongPass' }
Expected Results	HTTP 401 Unauthorized. Response body contains a descriptive error message. No token is issued.

Actual Result	Pass
Remarks	No JWT is returned. The error message does not reveal whether the email exists.

Table 28 UT03 — Attendance Status Enum Mapping

Testcase ID	UT03
Requirement ID	FR-ATT-01
Title	Attendance Status Enum Mapping
Description	Test that the AttendanceStatus enum correctly maps integer values to their string representations.
Objective	To verify that 0, 1, and 2 map to Present, Absent, and Leave respectively.
Driver/precondition	AttendanceController is compiled and accessible.
Test steps	1. Call the attendance endpoint with status values 0, 1, and 2 in separate requests. 2. Retrieve the saved record and inspect the status field.
Input	status: 0 status: 1 status: 2
Expected Results	Saved records return 'Present', 'Absent', and 'Leave' respectively in all GET responses.
Actual Result	Pass
Remarks	Enum values are serialised correctly as strings in API JSON responses.

Table 29 UT04 — Grade Calculation Logic — Boundary Values

Testcase ID	UT04
Requirement ID	FR-MARKS-01
Title	Grade Calculation Logic — Boundary Values
Description	Test that the grade assignment logic returns correct letter grades at all threshold boundaries.

Objective	To verify the grade computation function handles edge cases at 50, 60, 70, 80, and 90 percent.
Driver/precondition	MarksController is running. A valid student and teacher record exist.
Test steps	1. Submit marks entries that result in percentages of exactly 49, 50, 59, 60, 69, 70, 79, 80, 89, and 90. 2. Retrieve each record and compare the grade field.
Input	marksObtained/totalMarks producing percentages: 49%, 50%, 60%, 70%, 80%, 90%
Expected Results	Grades: 49%=F, 50%=C, 60%=B, 70%=B+, 80%=A, 90%=A+.
Actual Result	Pass
Remarks	All boundary values map to the correct grade. No off-by-one errors detected.

Table 30 UT05 — Rule-Based Risk Score — High Risk Threshold

Testcase ID	UT05
Requirement ID	FR-RISK-01
Title	Rule-Based Risk Score — High Risk Threshold
Description	Test that the rule-based engine classifies a student as High risk when attendance is below 50%.
Objective	To verify the risk classification returns 'High' when the attendance rate falls under the critical threshold.
Driver/precondition	RiskService is instantiated. Student has 10 attendance records; only 4 are marked Present.
Test steps	1. Call GET /api/ai/predict?studentId={id} for a student with 40% attendance. 2. Inspect ruleBasedRisk in the response.
Input	studentId with attendanceRate = 40%, averageMarks = 55%, negativeRemarkCount = 1
Expected Results	Response JSON: ruleBasedRisk = 'High', finalRisk = 'High'.
Actual Result	Pass

Remarks	Rule-based threshold (attendance < 50%) correctly triggers High risk classification.
----------------	--

5.2 Functional Testing (FT)

In functional testing, the functionality of each module from the stakeholders' perspective is tested. This is to ensure that the system produced meets the specifications and requirements defined in the Software Requirements Specification (SRS). Each testcase covers a complete user-facing workflow from HTTP request through to database persistence and API response.

Table 31 FT01 — Mark Attendance for a Student

Testcase ID	FT01
Requirement ID	FR-ATT-02
Title	Mark Attendance for a Student
Description	Test the end-to-end flow of a teacher marking a student Present for today's date.
Objective	To confirm that a new attendance record is created and persisted correctly in the database.
Driver/precondition	Teacher is authenticated (valid JWT). Student record exists. No attendance record exists for today.
Test steps	1. Log in as Teacher and obtain JWT token. 2. Send POST /api/attendance with studentId, teacherId, today's date, and status=0. 3. Retrieve the record with GET /api/attendance?studentId={id}&date={today}.
Input	{ studentId: 5, teacherId: 2, date: '2025-05-01', status: 0 }
Expected Results	HTTP 201 Created. GET returns one record with status = 'Present' for the correct date.
Actual Result	Pass
Remarks	Record is visible in the teacher's attendance list view.

Table 32 FT02 — Prevent Duplicate Attendance Entry

Testcase ID	FT02
Requirement ID	FR-ATT-03
Title	Prevent Duplicate Attendance Entry
Description	Test that submitting attendance twice for the same student on the same date returns a conflict error.
Objective	To ensure data integrity by preventing duplicate attendance records for the same student and date.
Driver/precondition	Teacher is authenticated. An attendance record already exists for studentId=5 on today's date.
Test steps	1. Send a second POST /api/attendance with the same studentId and date as an existing record. 2. Inspect the HTTP response.
Input	{ studentId: 5, teacherId: 2, date: '2025-05-01', status: 1 }
Expected Results	HTTP 409 Conflict. Response body contains the existing attendanceId and an explanatory message.
Actual Result	Pass
Remarks	No duplicate record is created. Caller is directed to use PUT to update the existing record.

Table 33 FT03 — Enter Marks and Verify Percentage

Testcase ID	FT03
Requirement ID	FR-MARKS-02
Title	Enter Marks and Verify Percentage
Description	Test that a teacher can submit a marks record and the system correctly computes and stores the percentage.
Objective	To verify that the marks entry endpoint persists marksObtained, totalMarks, and the calculated percentage.

Driver/precondition	Teacher authenticated. Student exists. No prior marks for this subject and exam combination.
Test steps	1. Send POST /api/mark with studentId, teacherId, subject, examType, marksObtained=75, totalMarks=100. 2. Retrieve the record and check the percentage field.
Input	{ studentId: 3, teacherId: 2, subject: 'Mathematics', exam: 'Mid-Term', marksObtained: 75, totalMarks: 100 }
Expected Results	HTTP 201 Created. GET response shows percentage = 75.0 and grade = 'B+'.
Actual Result	Pass
Remarks	Grade and percentage are calculated server-side, not derived from client input.

Table 34 FT04 — Submit a Complaint

Testcase ID	FT04
Requirement ID	FR-COMPL-01
Title	Submit a Complaint
Description	Test that a parent or student can submit a complaint and it is recorded with status 'Submitted'.
Objective	To verify the complaint submission workflow saves data correctly and assigns the default status.
Driver/precondition	User is authenticated. AssignedToUserId refers to a valid teacher user.
Test steps	1. Send POST /api/complaint with category, description, and assignedToUserId. 2. Retrieve the created complaint by its ID.
Input	{ category: 'Academic', description: 'Teacher did not return exam papers.', assignedToUserId: 4 }
Expected Results	HTTP 201 Created. GET response shows status = 'Submitted' and dateSubmitted is set to today.
Actual Result	Pass

Remarks	Complaint appears in the assigned teacher's complaint list.
----------------	---

Table 35 FT05 — Post a School Announcement

Testcase ID	FT05
Requirement ID	FR-NOTICE-01
Title	Post a School Announcement
Description	Test that a teacher can create and post a new notice that becomes visible to the intended audience.
Objective	To confirm the notice endpoint persists the announcement with isActive = true and correct metadata.
Driver/precondition	Teacher authenticated. Notice does not already exist.
Test steps	1. Send POST /api/notice with title, content, audience=1 (StudentsOnly), type=0 (Academic), priority=1 (Medium). 2. Retrieve all notices and verify the new entry appears with isActive = true.
Input	{ title: 'Mid-Term Schedule', content: 'Mid-terms begin on 10th May 2025.', audience: 1, type: 0, priority: 1 }
Expected Results	HTTP 201 Created. Notice appears in GET /api/notice list with isActive = true and correct postedAt timestamp.
Actual Result	Pass
Remarks	Timestamp is set server-side using DateTime.UtcNow, not supplied by the client.

Table 36 FT06 — Record a Student Achievement

Testcase ID	FT06
Requirement ID	FR-ACH-01
Title	Record a Student Achievement
Description	Test that a teacher can record an academic achievement for a student, including an optional certificate link.

Objective	To verify that the achievement record is saved correctly and linked to the correct student.
Driver/precondition	Teacher authenticated. Student exists. No prior achievement with the same title for this student.
Test steps	1. Send POST /api/achievement with studentId, title, category, date, and an optional filePath URL. 2. Retrieve achievements for the student.
Input	{ studentId: 7, title: 'First Place — Science Fair', category: 'Science', date: '2025-04-20', filePath: 'https://drive.google.com/cert.pdf' }
Expected Results	HTTP 201 Created. GET /api/achievement?studentId=7 returns the new record with all fields correctly stored.
Actual Result	Pass
Remarks	filePath is stored as provided and returned in GET responses for certificate linking.

5.3 Integration Testing (IT)

After ensuring the individual test runs successfully, the interaction of the classes and functions is tested to assess whether the feature is yielding the desired results. Integration tests in this system verify that the RiskService, ModelTrainer, AIController, and database layer collaborate correctly to produce risk predictions. They also confirm that cross-module data flows — for example, attendance records affecting risk scores — operate without inconsistency.

Table 37 *IT01 — AI Risk Prediction — Full Pipeline Integration (Key Feature)*

Testcase ID	IT01
Requirement ID	FR-RISK-02
Title	AI Risk Prediction — Full Pipeline Integration (Key Feature)
Description	Test the complete AI risk prediction pipeline from data retrieval through ML inference to API response.
Objective	To verify that RiskService, ModelTrainer, and AIController interact correctly and return a consistent, fully-populated risk report.

Driver/precondition	Student has at least 10 attendance records and 5 marks records in the database. ML model (risk_model.zip) has been trained and loaded into ModelTrainer.
Test steps	1. Call GET /api/ai/predict?studentId={id}. 2. Verify the response includes ruleBasedRisk, mlRisk, finalRisk, factors, and mlModelUsed = true. 3. Confirm finalRisk matches mlRisk (ML takes precedence).
Input	studentId with attendanceRate = 68%, averageMarks = 52%, behaviorScore = 80
Expected Results	HTTP 200 OK. Response: { ruleBasedRisk: 'Medium', mlRisk: 'Medium', finalRisk: 'Medium', mlModelUsed: true, factors: { attendanceRate: 68, averageMarks: 52, behaviorScore: 80 } }.
Actual Result	Pass
Remarks	Both rule-based and ML predictions agree. The ML model is confirmed as active (mlModelUsed = true).

Table 38 IT02 — AI Risk Prediction — Fallback to Rule-Based When Model Not Loaded

Testcase ID	IT02
Requirement ID	FR-RISK-03
Title	AI Risk Prediction — Fallback to Rule-Based When Model Not Loaded
Description	Test that the system returns a valid risk prediction using the rule-based engine when the ML model is not yet available.
Objective	To confirm the dual-strategy fallback mechanism works correctly and the system remains functional without a trained model.
Driver/precondition	ML model file (risk_model.zip) does not exist or has not been loaded. Student data exists in the database.
Test steps	1. Delete or rename risk_model.zip on the server. 2. Call GET /api/ai/predict?studentId={id}. 3. Inspect mlModelUsed in the response.
Input	studentId with attendanceRate = 45%, averageMarks = 38%, negativeRemarkCount = 6

Expected Results	HTTP 200 OK. Response: { ruleBasedRisk: 'High', finalRisk: 'High', mlModelUsed: false }. No server error is thrown.
Actual Result	Pass
Remarks	System degrades gracefully to rule-based classification without crashing or returning a 500 error.

Table 39 IT03 — ML Model Training Triggered Successfully

Testcase ID	IT03
Requirement ID	FR-RISK-04
Title	ML Model Training Triggered Successfully
Description	Test that calling the train endpoint builds a new model from student records and saves risk_model.zip.
Objective	To verify that ModelTrainer correctly loads data, builds the SDCA pipeline, and persists the trained model to disk.
Driver/precondition	Admin is authenticated. At least 10 student records exist. risk_model.zip does not exist prior to the call.
Test steps	1. Send POST /api/ai/train. 2. Verify HTTP 200 response. 3. Confirm risk_model.zip exists in the server output directory. 4. Call GET /api/ai/predict for a student and verify mlModelUsed = true.
Input	POST /api/ai/train (no body required)
Expected Results	HTTP 200 OK. Server log confirms model trained. risk_model.zip is created. Subsequent predictions use mlModelUsed = true.
Actual Result	Pass
Remarks	Training completes in under 5 seconds for up to 50 training samples.

Table 40 IT04 — Attendance Record Reflected in Risk Computation

Testcase ID	IT04
Requirement ID	FR-ATT-04

Title	Attendance Record Reflected in Risk Computation
Description	Test that newly added attendance records are immediately reflected in the next risk prediction call.
Objective	To verify that RiskService reads live data and does not use cached values, ensuring predictions are always up to date.
Driver/precondition	Student has 9 existing attendance records (8 Present, 1 Absent). Attendance rate = 88.9%.
Test steps	1. Call GET /api/ai/predict?studentId={id} — record initial riskLevel. 2. Add 5 Absent attendance records via POST /api/attendance. 3. Call GET /api/ai/predict?studentId={id} again. 4. Compare the two risk responses.
Input	5 x POST /api/attendance with status = 1 (Absent) for the same studentId
Expected Results	Second prediction returns a higher risk level than the first (e.g., Low changes to Medium or High). attendanceRate in factors reflects the updated value.
Actual Result	Pass
Remarks	No caching layer interferes. Risk is computed fresh from the database on each request.

Table 41 IT05 — Complaint Status Update Triggers Correct Lifecycle

Testcase ID	IT05
Requirement ID	FR-COMPL-02
Title	Complaint Status Update Triggers Correct Lifecycle
Description	Test that updating a complaint status from Submitted to Resolved correctly sets the DateClosed field.
Objective	To verify the complaint lifecycle integrates correctly — status change, DateClosed assignment, and data persistence.
Driver/precondition	A complaint with status = 'Submitted' exists. Teacher is authenticated and is the assigned user.

Test steps	1. Call PATCH /api/complaint/{id}/status with status = 2 (Resolved) and a remarks value. 2. Retrieve the complaint via GET /api/complaint/{id}. 3. Verify status and dateClosed fields.
Input	{ status: 2, remarks: 'Issue reviewed and resolved with the concerned teacher.' }
Expected Results	HTTP 200 OK. GET response shows status = 'Resolved', dateClosed is set to today's date, remarks field is populated.
Actual Result	Pass
Remarks	DateClosed is assigned server-side. Client cannot supply or manipulate this value.

5.4 Performance Testing (PT)

In order to assess the performance of the feature, the performance test is conducted. It usually measures response time, stability (reliability under stress). Postman and browser DevTools are used as the primary tools for conducting the performance testing of the School Management System. The testcases below specify the configuration and expected output of the performance tests conducted for the core system features.

Table 42 *PT01 — API Response Time — Single Risk Prediction*

Testcase ID	PT01
Requirement ID	NFR-PERF-01
Title	API Response Time — Single Risk Prediction
Description	Measure the response time of the AI risk prediction endpoint under normal load conditions.
Objective	To confirm the risk prediction endpoint responds within an acceptable time limit for a single user request.
Driver/precondition	Application is running in Release mode. Database contains 50 student records with full attendance and marks histories. ML model is loaded.
Test steps	1. Use Postman or a stopwatch to measure the round-trip time of GET /api/ai/predict?studentId={id}. 2. Repeat 10 times and record the average.

Input	GET /api/ai/predict?studentId=5 (repeated 10 times)
Expected Results	Average response time is under 500 milliseconds. No request exceeds 1 second.
Actual Result	Pass
Remarks	Average observed response time: approximately 180 ms. Well within the 500 ms target.

Table 43 PT02 — Dashboard Statistics Load Time

Testcase ID	PT02
Requirement ID	NFR-PERF-02
Title	Dashboard Statistics Load Time
Description	Measure the time taken for the Teacher Dashboard statistics endpoint to aggregate and return data.
Objective	To confirm that aggregation of attendance, complaints, notices, and remarks data is performant.
Driver/precondition	Teacher has 30 students, 300 attendance records, 20 complaints, and 15 notices in the database.
Test steps	1. Send GET /api/teacher/dashboard-stats with a valid teacher JWT. 2. Measure response time using Postman timer. 3. Repeat 5 times and average the results.
Input	GET /api/teacher/dashboard-stats (authenticated teacher with large dataset)
Expected Results	Response time is under 400 milliseconds. JSON payload includes all required stats fields.
Actual Result	Pass
Remarks	Indexed foreign-key columns ensure efficient COUNT aggregation queries.

Table 44 PT03 — ML Model Training Time

Testcase ID	PT03
Requirement ID	NFR-PERF-03
Title	ML Model Training Time
Description	Measure the time taken to train the SDCA Maximum Entropy model on a dataset of 50 records.
Objective	To confirm that on-demand model training completes within an acceptable duration and does not block the server.
Driver/precondition	50 labelled training records available. Server is running in Release configuration.
Test steps	1. Call POST /api/ai/train and record the start time. 2. Note the response time when HTTP 200 is returned. 3. Confirm risk_model.zip is created.
Input	POST /api/ai/train (50 training records)
Expected Results	Training completes and HTTP 200 is returned in under 10 seconds.
Actual Result	Pass
Remarks	Observed training time: approximately 3.2 seconds for 50 records with 3 features.

Table 45 PT04 — Frontend Dashboard Render Time

Testcase ID	PT04
Requirement ID	NFR-PERF-04
Title	Frontend Dashboard Render Time
Description	Measure the time for the Teacher Dashboard Overview component to fully render after a successful API call.
Objective	To verify that the React frontend displays dashboard data promptly, providing a responsive user experience.
Driver/precondition	Browser DevTools open. Teacher is authenticated. API returns data within 200 ms.

Test steps	1. Open the Teacher Dashboard in a browser. 2. Use DevTools Performance tab to measure time from navigation start to the last paint event. 3. Repeat 3 times.
Input	Browser load of TeacherDashboardOverview component with live API data
Expected Results	Time to Interactive (TTI) is under 2 seconds. All stat cards render without layout shift.
Actual Result	Pass
Remarks	Vite build optimisation and React lazy loading ensure fast initial render.

5.5 Summary

This chapter presented a structured and comprehensive evaluation of the Web-Based School Management System through **four categories of testing: unit, functional, integration, and performance testing**. All testcases were designed to be independent, repeatable, and free of hardcoded values, in conformance with the testing best practices specified at the start of this chapter.

The testing process confirmed that all **core modules** — including Attendance Management, Marks Entry, Complaint Handling, Announcement Posting, and Achievement Recording — function correctly in isolation and in combination. The AI-Based Student Risk Prediction module, the key feature of this project, was subjected to the most rigorous integration testing. Tests IT01 through IT04 verified the full pipeline from data retrieval through ML inference to API response, confirmed the reliability of the rule-based fallback mechanism, validated the on-demand model training workflow, and ensured that risk scores reflect real-time database changes without caching interference.

Performance tests demonstrated that the system meets the non-functional requirements for response time, with the risk prediction endpoint averaging under 200 milliseconds and ML model training completing in approximately 3 seconds for a 50-record dataset. These results confirm that the implementation is production-ready and meets the objectives defined in the project requirements.

Chapter 6

System Conversion

6 Introduction

System conversion refers to the process of transitioning from an existing approach of managing school operations to a new, integrated, web-based platform. Prior to the development of this School Management System (SMS), administrative and academic tasks were carried out using **fragmented methods** — including paper-based registers for attendance, manually maintained spreadsheets for marks recording, and informal verbal or written communication for complaints and announcements. These approaches were prone to data loss, inconsistency, and inefficiency, particularly in generating timely reports or identifying students who may be academically at risk.

The Web-Based School Management System replaces these disconnected processes with a **unified digital platform**. The system introduces structured management of attendance, marks, complaints, behaviour remarks, announcements, and achievements — all accessible through a role-based interface for administrators, teachers, students, and parents. Most significantly, the system introduces an AI-Based Student Risk Prediction feature powered by ML.NET, which provides automated early warning of students at academic risk — a capability entirely absent in the previous manual approach.

This chapter describes the **conversion method** selected for transitioning to the new system, the deployment steps followed for each system component, the data migration strategy, the user training approach, and the post-deployment testing carried out to confirm system readiness.

6.1 Conversion Method

Selecting an appropriate conversion strategy is critical to ensure a smooth transition from the old system to the new one. For the School Management System, the Pilot Conversion method has been selected as the most suitable approach.

Selected Method: Pilot Conversion

In pilot conversion, the new system is first introduced to a limited segment of the organisation — in this case, a single class group or department — before being rolled out to the entire institution. The pilot group uses the new SMS in full while the rest of the school continues with the existing manual processes. Based on the feedback and results observed during the pilot, issues are identified and resolved before full deployment.

Justification: Since the SMS is a newly developed system with no prior legacy database to migrate, the primary conversion concern is user acceptance and operational correctness rather than data migration. Pilot conversion allows the development team to validate that all modules — including the AI Risk Prediction engine — function correctly in a real school environment, with real users and live data, before the system is made available to all users. This significantly reduces the risk of widespread disruption to school operations.

Benefits:

- 1. Allows the team to identify and fix real-world issues before school-wide rollout.
- 2. Reduces disruption — the majority of school operations continue unaffected during the pilot phase.
- 3. Provides an opportunity to collect genuine user feedback from teachers and students on usability.
- 4. Validates AI risk prediction accuracy against real student data before broader deployment.
- 5. Builds user confidence gradually, improving adoption rates across the institution.

Risks (and Mitigations):

- 1. The pilot group may experience early-stage bugs. Mitigated by thorough pre-pilot testing (Chapter 5) and a dedicated support channel during the pilot period.
- 2. The pilot may not capture the full range of user scenarios. Mitigated by selecting a diverse pilot group that includes all user roles: administrator, teacher, student, and parent.
- 3. Parallel data management (manual + digital) during the pilot may increase short-term workload. Mitigated by keeping the pilot duration concise (two to four weeks) and providing a clear transition timeline.

6.2 Deployment

The deployment of the School Management System involves configuring and launching four interdependent components: the SQL Server database, the ASP.NET Core Web API backend, the React.js frontend, and the ML.NET AI Risk Prediction module. The following sections describe the step-by-step deployment process for each component, followed by the containerised deployment approach using Docker.

Table 46 Deployment Overview — Components and Actions

Component / Phase	Deployment Action
SQL Server Database	Create database, apply Entity Framework Core migrations to generate all schema tables.
ASP.NET Core Backend	Configure appsettings.json, publish the API in Release mode, and run the application server.
React.js Frontend	Install npm dependencies, configure the API base URL, and build the production bundle.
ML.NET AI Module	Trigger model training via the /api/ai/train endpoint; verify risk_model.zip is generated.
Docker (Container)	Build Docker images for backend and frontend; run containers using Docker Compose.
Verification	Run smoke tests on all API endpoints, confirm UI loads, and verify AI predictions return valid results.

Backend Deployment (ASP.NET Core Web API)

The backend provides all REST API endpoints consumed by the frontend and is the host of the ML.NET model. The following steps describe its deployment:

1. Ensure the .NET 8.0 SDK and runtime are installed on the target machine.
2. Open appsettings.json and update the DefaultConnection string to point to the target SQL Server instance, specifying the correct server name, database name (SchoolManagementDB), and authentication credentials.
3. Run database migrations to apply the full Entity Framework Core schema to the target database using the update-database command in the Package Manager Console or the equivalent CLI command.
4. Publish the application in Release mode using the dotnet publish command with the Release configuration flag. This produces an optimised, self-contained deployment folder.
5. Copy the published output folder to the target server directory.
6. Start the API using the dotnet command pointed at the published DLL, or configure it as a Windows Service or IIS application for persistent hosting.
7. Verify the API is running by navigating to the /swagger endpoint in a browser, which displays all available API routes.

Frontend Deployment (React.js / Vite)

The React frontend is built into a static bundle and served independently. Its deployment steps are as follows:

1. Ensure Node.js (v18 or later) and npm are installed on the build machine.
2. Navigate to the frontend project directory and run `npm install` to restore all required packages, including React, Tailwind CSS, Lucide React, and Recharts.
3. Open the API configuration file (typically `src/api/teacherApi.ts` or a `.env` file) and set the `VITE_API_URL` environment variable to the deployed backend API URL (e.g., `http://server-ip:5000/api`).
4. Run the production build command (`npm run build`) to generate the optimised static bundle in the `dist/` output folder.
5. Deploy the contents of the `dist/` folder to a static web server such as Nginx, Apache, or any cloud-based static hosting service.
6. Configure the web server to redirect all routes to `index.html` to support React Router client-side navigation.
7. Verify the frontend loads in a browser and successfully communicates with the backend API.

Database Setup (SQL Server)

The SQL Server database stores all persistent system data including users, students, attendance, marks, complaints, and AI monitoring records.

1. Install SQL Server (Express edition or higher) on the target machine.
2. Create a new database named `SchoolManagementDB` using SQL Server Management Studio (SSMS) or the SQL Server CLI.
3. Update the connection string in `appsettings.json` with the correct server instance name, database name, and authentication method (Windows Authentication or SQL Authentication).
4. Apply Entity Framework Core migrations by running the database update command from the backend project directory. This creates all required tables: `Users`, `Students`, `Teachers`, `Parents`, `Attendances`, `Marks`, `Complaints`, `Notices`, `Achievements`, `BehaviorRemarks`, and `RiskMonitoring`.
5. Verify the schema by opening SSMS and confirming all tables are present with the correct columns and relationships.
6. Optionally, run the provided seed data script to populate initial admin user credentials and sample student records for testing.

AI Module Deployment (ML.NET Risk Prediction)

The AI Risk Prediction module is embedded within the ASP.NET Core backend and requires an explicit training step after deployment to generate the ML model file. This is the system's most distinctive feature and must be verified carefully.

1. Confirm that the backend API is running and the database is populated with at least ten student records, each with attendance and marks data.
2. Send an authenticated POST request to the `/api/ai/train` endpoint using Postman or the Swagger UI. This triggers the ML.NET model training pipeline, which loads training data, builds the SDCA Maximum Entropy multiclass classifier, and saves the trained model as `risk_model.zip` in the server's output directory.
3. Verify that `risk_model.zip` has been created in the expected output folder on the server.
4. Send a GET request to `/api/ai/predict?studentId={id}` for a known student. Confirm that the response JSON includes the `mlModelUsed = true` flag, indicating the trained ML model is active rather than the rule-based fallback.
5. If the ML model is not loaded (`mlModelUsed = false`), retrigger the training endpoint and verify the model file is written successfully before retesting.
6. The AI module is considered fully deployed when at least one prediction is returned with `mlModelUsed = true` and all three risk factors (`attendanceRate`, `averageMarks`, `behaviorScore`) appear in the response.

Docker-Based Containerised Deployment

Although the system was developed and tested in a local environment, Docker can be used to package and deploy the application in a portable, environment-independent manner. Containerisation ensures that the system runs consistently across development, staging, and production environments without dependency conflicts.

Concept and Approach:

1. A Dockerfile is created for the ASP.NET Core backend. It uses the official .NET 8.0 SDK image to build the application and then copies the published output into a lightweight .NET runtime image for execution. The container exposes port 5000 for API communication.

2. A separate Dockerfile is created for the React frontend. It uses a Node.js image to run the npm build process and then copies the resulting dist/ folder into an Nginx image, which serves the static files on port 80.
3. A docker-compose.yml file defines three services: the backend API container, the frontend Nginx container, and the SQL Server container (using the official Microsoft SQL Server Linux image). Environment variables in the compose file configure the database connection string and API URL so that containers communicate over a shared Docker network.
4. Running docker-compose up with the build flag starts all three containers simultaneously. The backend waits for the database container to be ready before accepting connections, managed through a health check or a startup delay configuration.
5. After the containers are running, the AI model training endpoint is called to generate risk_model.zip inside the backend container. To persist the model across container restarts, the output directory is mapped to a Docker volume.

Verification after Docker deployment: Open a browser and navigate to the frontend URL (<http://localhost:80>). Log in as a teacher and confirm the dashboard loads. Use Postman to call `/api/ai/predict` for a student and verify the risk prediction response. Check container logs using docker logs for any startup errors in the backend or database containers.

6.2.1 Data Conversion

Since the School Management System is a new platform with no direct predecessor database, data conversion primarily involves the migration of any existing manual records — such as spreadsheet-based student lists, paper attendance registers, and historical marks data — into the system's SQL Server database. The following four-phase process was followed.

Phase 1 — Extract: Existing student and academic records maintained in Microsoft Excel spreadsheets or paper registers are collected and digitised. All available student identifiers (name, roll number, class, date of enrolment) and historical marks are compiled into a standardised intermediate format (CSV files) for processing.

Phase 2 — Clean: The extracted data is reviewed and cleaned before import. Duplicate student records are identified and removed. Missing required fields (such as roll number or class assignment) are flagged for completion. Data values are validated against the expected formats — for example, marks values are confirmed to be numeric and within the valid range, and dates are standardised to the ISO 8601 format (YYYY-MM-DD) expected by the system.

Phase 3 — Transform: Cleaned data is mapped to the system's database schema. Student names are split into FirstName and LastName fields. Class information is mapped to the class strings used in the Students table. Each student record is linked to a corresponding User account by assigning a UserId, which is created in advance through the authentication system. Marks records are structured to include subject, examType, marksObtained, and totalMarks fields as required by the Marks table schema.

Phase 4 — Load: Transformed records are inserted into the database using SQL INSERT scripts or by calling the backend API bulk-import endpoints. After loading, row counts are verified against the source to confirm completeness. A sample of loaded records is manually cross-checked against the original source data to validate accuracy.

Backup and Restore Process: Before any data loading or schema migration is performed, a full backup of the target SQL Server database is taken using SQL Server Management Studio (SSMS). Backups are stored in a secure, named location with a timestamp. In the event of a migration failure, the database can be restored to its pre-migration state using the SSMS Restore Database wizard, ensuring that no data is permanently lost during the conversion process. After successful migration and verification, a second backup is taken to preserve the post-migration state.

6.2.2 Training

Effective user training is essential for successful system adoption. A simple, step-by-step user manual was prepared for the two primary user groups of the School Management System. Training sessions were conducted using live demonstrations on the deployed system, and printed reference cards summarising the key steps were distributed to users.

User Manual 1: Teacher

Login:

1. Open the system URL in a web browser.
2. Enter your registered email address and password in the login form.
3. Click the Login button. You will be redirected to the Teacher Dashboard.

Mark Attendance:

1. From the left navigation menu, click Mark Attendance.
2. Select today's date from the date picker (defaults to today).

3. For each student in the list, click P (Present), A (Absent), or L (Leave).
4. Use Mark All Present to set all students as present and then adjust individual exceptions.
5. Click Save Attendance. A green confirmation message confirms the records have been saved.

Enter Marks:

1. Click Enter Marks in the navigation menu.
2. Select the Exam type (e.g., Mid-Term) and Subject from the dropdown menus.
3. In the marks table, enter the Marks Obtained for each student. The system automatically computes the percentage and letter grade.
4. Click Save Marks. Unsaved rows are highlighted in yellow; saved rows display a 'saved' indicator.

Use AI Risk Prediction (Key Feature):

1. Click the student's name or navigate to the AI Risk Prediction section from the admin panel.
2. The system displays the student's risk level (Low, Medium, or High) along with the three contributing factors: Attendance Rate, Average Marks, and Behaviour Score.
3. A colour-coded badge indicates the risk level: green for Low, orange for Medium, and red for High.
4. If the ML model is active (shown by the 'ML Model Active' indicator), the prediction is generated by the trained SDCA classifier. If not yet active, a rule-based prediction is displayed.
5. Use this information to prioritise follow-up with high-risk students.

User Manual 2: Student / Parent

Login:

1. Open the system URL in a web browser.
2. Enter your registered email and password. Students and parents use role-specific accounts created by the administrator.
3. Click Login. You will be directed to the Student or Parent Dashboard.

View Marks / Results:

1. From the dashboard menu, click View Results or Marks.
2. Select the desired exam type and subject from the filter dropdowns.

3. The marks table displays your obtained marks, total marks, percentage, and letter grade for each entry.

Check Attendance:

1. Click the Attendance section in the dashboard menu.
2. Select a date range or month to view the attendance records for that period.
3. Each record is shown with a colour-coded status: green for Present, red for Absent, and yellow for Leave.
4. The attendance summary at the top displays the overall attendance percentage for the selected period.

View Risk Status:

1. Navigate to the Risk Status section of the dashboard.
2. Your current risk level (Low, Medium, or High) is displayed along with the three contributing factors that inform it.
3. If the risk level is Medium or High, a note recommends seeking academic support or contacting a teacher.

6.3 Post Deployment Testing

After the system has been fully deployed, a structured round of post-deployment testing is conducted to confirm that all components are functioning correctly in the live environment. This testing phase reuses the testcases defined in Chapter 5 (Testing and Evaluation) and applies them against the deployed system rather than the development environment.

The following checks are performed as part of post-deployment verification:

1. **API Connectivity:** All major endpoints are called using Postman to confirm the backend API is responding correctly. Key endpoints tested include POST /api/auth/login, GET /api/teacher/dashboard-stats, POST /api/attendance, POST /api/mark, and GET /api/ai/predict.
2. **Database Connectivity:** Test records are inserted and retrieved through the API to confirm that the Entity Framework Core connection to the SQL Server database is active and functioning. Row counts are verified against expected values.
3. **UI Functionality:** A tester logs in as each user role (Admin, Teacher, Student, Parent) and navigates through every module — dashboard, attendance, marks, announcements, complaints, achievements, and behaviour remarks — confirming that data renders correctly and all form submissions complete without errors.

4. **AI Risk Prediction Verification (Critical):** The POST /api/ai/train endpoint is called to confirm model training completes successfully in the live environment. A subsequent GET /api/ai/predict request for a known student is inspected to verify that mlModelUsed = true and the risk factors are correctly populated. Both the rule-based and ML-predicted risk levels are compared for consistency. The AI dashboard panel in the frontend is opened to confirm the risk badge, factor bars, and ML status indicator all render correctly.
5. **Authentication and Role-Based Access:** Login attempts are made for each user role to confirm that JWT tokens are issued and that role-specific routes are accessible. Unauthorised access attempts (e.g., a teacher accessing admin-only endpoints) are tested to confirm that HTTP 403 Forbidden responses are returned correctly.
6. **Performance Smoke Test:** Response times for the risk prediction and dashboard statistics endpoints are measured to confirm they fall within the limits established during Chapter 5 performance testing (under 500 ms and 400 ms respectively).

Any failures identified during post-deployment testing are logged, assigned a severity level, and resolved before the system is made available to the wider user base. The pilot group's findings during the pilot conversion phase supplement the formal post-deployment test results.

6.4 Challenges

The development and deployment of the School Management System encountered a number of practical challenges. The following table summarises each challenge and the corresponding solution that was applied.

Table 47 Deployment and Development Challenges with Solutions

Challenge	Solution Applied
Backend-Frontend CORS Errors	The React frontend was blocked from calling the ASP.NET Core API due to Cross-Origin Resource Sharing (CORS) policy restrictions. This was resolved by explicitly configuring allowed origins in Program.cs using the ASP.NET Core CORS middleware, permitting the frontend development server URL (http://localhost:5173) and the production domain.
JWT Token Expiry and Silent Failures	API calls made after a JWT token had expired returned HTTP 401 errors with no visible indication to the user. This issue was resolved by implementing a global response handling mechanism in the frontend that detects 401 Unauthorized

	responses and automatically redirects the user to the login page with an appropriate session-expired message.
Entity Framework Core Migration Conflicts	Applying database migrations on the target SQL Server instance occasionally failed due to pending schema changes from earlier development iterations. This was resolved by resetting the migration history, recreating a clean initial migration representing the full current schema, and applying it to a fresh database instance.
AI Model Accuracy with Limited Training Data	The ML.NET SDCA classifier produced inconsistent risk classifications when trained on fewer than ten student records, as the training dataset was insufficient to distinguish between risk categories reliably. This was resolved by supplementing real records with eighteen carefully designed seed data samples that represent all three risk levels, ensuring a minimum viable training set is always available.
ML Model Not Persisting Across Restarts	The trained risk_model.zip was stored in the application's working directory, which was reset on each server restart in containerised environments. This was resolved by mapping the model output directory to a Docker volume so that the trained model file persists independently of the container lifecycle.
Duplicate Attendance Submission Errors	Teachers occasionally submitted attendance for a date that already had records, causing unhandled conflict errors in the UI. This was resolved by adding a duplicate-check in the AttendanceController that returns HTTP 409 Conflict with the existing record ID, and by updating the frontend to display a clear message directing the user to update the existing record rather than submit a new one.
Role-Based Route Protection in React	Some frontend routes were accessible to all authenticated users regardless of role, allowing students to access teacher-only components. This was resolved by implementing a ProtectedRoute wrapper component that reads the user role from the decoded JWT token and redirects unauthorised users to a 403 access-denied page.
SQL Server Connection String Configuration	Deployment on machines with named SQL Server instances required instance-specific connection strings that differed from the development environment. This was resolved by externalising the connection string into environment variables and documenting the required format for each deployment scenario in the project README.

6.5 Summary

This chapter documented the complete system conversion and deployment process for the Web-Based School Management System. The **Pilot Conversion method** was selected as the most appropriate strategy for transitioning from the prior manual approach, given that the system is newly developed and user acceptance is the primary adoption risk. By deploying the system to a limited pilot group first, the team was able to validate functionality, collect real user feedback, and address issues before institution-wide rollout.

The **deployment process** was described in full across all **four system components**: the SQL Server database, the ASP.NET Core Web API backend, the React.js frontend, and the ML.NET AI Risk Prediction module. A Docker-based containerised deployment approach was also presented to support portable, environment-independent hosting. **Data conversion** was addressed through a structured four-phase extract-clean-transform-load process, accompanied by a database backup and restore protocol to safeguard data integrity throughout migration.

User training was provided for both primary user groups — teachers and students/parents — through a concise step-by-step user manual covering all major workflows, with particular emphasis on the AI Risk Prediction feature, which represents the system's most significant innovation. Post-deployment testing confirmed that all API endpoints, database connections, user interface components, and AI prediction capabilities are functioning correctly in the live environment.

The **eight challenges encountered** during development and deployment were each addressed with practical, targeted solutions, ranging from CORS configuration and JWT handling to ML model persistence and training data augmentation. The School Management System is fully operational, deployment-ready, and equipped to deliver measurable improvements in academic management efficiency and early student risk identification for the institution.

Chapter 7

Conclusion

7 Introduction

This chapter concludes the project, evaluates the project objectives and goals, and highlights future work. It provides a summary of what was built, confirms that all stated objectives have been achieved, and presents a traceability matrix linking every requirement to its design, code, and test coverage. The chapter also identifies potential areas for enhancement in future development cycles.

7.1 Evaluation

The following table lists the objectives defined for the School Management System at the start of the project and confirms whether each has been achieved in the final delivered system. All objectives specified in Chapter 1 have been fully implemented and verified through the testing process described in Chapter 5.

Table 48 Objectives Completion Status

Objectives	Status
Provide role-based access for Admin, Teacher, Student, and Parent users.	Completed
Enable teachers to mark and manage daily student attendance.	Completed
Allow teachers to enter, update, and view student examination marks with auto-calculated grades.	Completed
Provide a structured complaint submission and resolution workflow.	Completed
Allow teachers to post announcements and notices for students and parents.	Completed
Enable recording and viewing of student behaviour remarks by teachers.	Completed

Allow teachers to record student achievements and certificates.	Completed
Implement AI-based Student Risk Prediction using ML.NET to classify students as Low, Medium, or High risk.	Completed
Provide a rule-based fallback engine when the ML model is not yet trained.	Completed
Display a Teacher Dashboard with aggregated real-time statistics.	Completed
Provide a Student/Parent Dashboard showing attendance, marks, and risk status.	Completed
Ensure secure authentication using JWT tokens with role-based authorisation.	Completed

7.2 Traceability Matrix

The traceability matrix below shows how each design, code, and test configuration item traces back to a specific requirement in the Software Requirements Specification (SRS). It ensures that all requirements are accounted for in the final product. The Requirement ID is a unique identifier assigned to each requirement. Requirement Description provides a brief summary of the requirement. Design Specification lists the components, classes, or algorithms where the requirement is addressed. Code identifies the files in which the functionality is implemented. Test ID refers to the testcase(s) from Chapter 5 that document and verify the requirement.

Table 49 Traceability Matrix

Requirement ID	Requirement Description	Design Specification	Code	Test ID
FR-AUTH-01	User login with JWT authentication	AuthController, JwtService	AuthController.cs	UT01, UT02

	and role-based access control.			
FR-ATT-01	Teacher marks daily student attendance (Present / Absent / Leave).	AttendanceController, MarkAttendance	AttendanceController.cs, MarkAttendance.tsx	UT03, FT01, FT02
FR-ATT-02	Prevent duplicate attendance for the same student on the same date.	AttendanceController (duplicate guard)	AttendanceController.cs	FT02
FR-MARKS-01	Teacher enters examination marks; system computes percentage and grade.	MarksController, EnterMarks, Grade Calc	MarksController.cs, EnterMarks.tsx	UT04, FT03
FR-RISK-01	Rule-based risk engine classifies students as Low, Medium, or High.	RiskService (rule-based logic)	RiskService.cs	UT05
FR-RISK-02	ML.NET SDCA classifier predicts student risk from attendance and marks.	ModelTrainer, AIController	ModelTrainer.cs, AIController.cs	IT01
FR-RISK-03	System falls back to rule-based prediction if ML model is not loaded.	AIController (fallback logic)	AIController.cs	IT02
FR-RISK-04	Admin triggers on-demand ML model training	ModelTrainer.Train(), AIController	ModelTrainer.cs	IT03

	via API endpoint.			
FR-COMPL-01	User submits a complaint; assigned to a teacher for resolution.	ComplaintController, ComplaintManagement	ComplaintController.cs, ComplaintManagement.tsx	FT04, IT05
FR-NOTICE-01	Teacher posts an announcement visible to the target audience.	NoticeController, PostAnnouncements	NoticeController.cs, PostAnnouncements.tsx	FT05
FR-ACH-01	Teacher records a student achievement with an optional certificate link.	AchievementController, UploadCertificates	AchievementController.cs, UploadCertificates.tsx	FT06
FR-BHV-01	Teacher adds behaviour remarks (Positive, Negative, Neutral) for a student.	BehaviorRemarkController, BehaviorRemarks	BehaviorRemarkController.cs, BehaviorRemarks.tsx	—
FR-DASH-01	Teacher dashboard aggregates real-time stats from multiple modules.	TeacherController (dashboard-stats)	TeacherController.cs, TeacherDashboardOverview.tsx	PT02
NFR-PERF-01	AI risk prediction endpoint responds within 500 ms.	AIController, ModelTrainer	AIController.cs	PT01
NFR-PERF-02	ML model training completes within 10 seconds for up to 50 records.	ModelTrainer	ModelTrainer.cs	PT03

NFR-SEC-01	All API endpoints require JWT authentication; roles are enforced.	AuthController, Authorize attributes	Program.cs, all Controllers	UT01, UT02
------------	---	--------------------------------------	-----------------------------	------------

7.3 Conclusion

The Web-Based School Management System was successfully **designed, implemented, tested, and deployed** as a fully functional platform that addresses the core administrative and academic management needs of a school. The system replaces fragmented manual processes with a unified, role-based digital solution accessible to administrators, teachers, students, and parents through a modern React.js interface backed by a secure ASP.NET Core Web API and SQL Server database.

The **most significant contribution** of this project is the AI-Based Student Risk Prediction module, which uses an ML.NET SDCA Maximum Entropy multiclass classifier to automatically assess each student's academic risk level based on attendance rate, average marks, and behaviour score. This feature enables teachers and administrators to identify at-risk students early and take timely corrective action — a capability entirely absent in traditional school management approaches. A rule-based fallback engine ensures the system remains operational and useful even before sufficient training data has been accumulated.

All project objectives were fully achieved and verified. The following table summarises the correlation between the objectives defined at the outset of the project and the functionality delivered by the system.

Table 50 Functionality implementation status

Objective / Functionality	Implementation Status
Role-based authentication and access control	Fully implemented using JWT tokens with Admin, Teacher, Student, and Parent roles enforced across all API endpoints.
Attendance management	Teachers can mark, update, and view individual and class attendance. Duplicate prevention and status locking are enforced.

Marks entry and grade calculation	Teachers enter marks per subject and exam. Percentages and letter grades (A+ to F) are computed automatically by the system.
Complaint management	A full lifecycle complaint system is implemented: submission, assignment, review, resolution, and closure with automatic date stamping.
Announcement and notice posting	Teachers can post notices with audience targeting, type, and priority. Notices can be deactivated independently.
Behaviour remarks	Teachers record Positive, Negative, or Neutral behaviour observations for students. Remarks feed directly into the AI risk score.
Student achievement recording	Teachers log student achievements with category, date, and an optional certificate link.
AI-Based Student Risk Prediction (Key Feature)	ML.NET SDCA multiclass classifier predicts Low, Medium, or High risk. Rule-based fallback ensures continuous operation. Both engines are active and verified.
Teacher and student dashboards	Real-time aggregated dashboards display relevant statistics for each user role, loaded from live database queries.

7.4 Future Work

Although all planned objectives have been met, several enhancements could be incorporated in future development iterations to extend the system's capabilities and scalability:

Parent-Teacher Communication Hub: A real-time messaging module between parents and teachers was excluded from the current scope. Future development could integrate a chat feature

using SignalR or WebSockets, allowing direct, asynchronous communication within the platform.

Enhanced AI Model with More Features: The current ML model uses three input features (attendance rate, average marks, and behaviour score). Future iterations could incorporate additional indicators such as assignment submission rates, extracurricular participation, and historical risk trends to improve prediction accuracy.

Automated Risk Alerts and Notifications: The system currently displays risk levels on demand. A future enhancement would include automated email or push notification alerts sent to teachers and parents when a student's risk level changes to Medium or High, enabling faster intervention.

Mobile Application: A dedicated mobile application (iOS and Android) built using React Native would allow teachers to mark attendance and parents to monitor their child's progress directly from their smartphones, improving accessibility and adoption.

Multi-School / Multi-Branch Support: The current system is designed for a single school instance. A multi-tenant architecture could be implemented to support multiple schools or branches under a single deployment, with data isolation enforced at the database level.

Advanced Reporting and Analytics: Future development could add a dedicated reporting module that generates printable PDF reports for attendance summaries, class performance analytics, and school-wide risk distribution charts, providing administrators with comprehensive data for decision-making.

Integration with External Learning Platforms: The system could be extended to integrate with external learning management systems (LMS) such as Google Classroom or Moodle, allowing academic performance data to be synchronised automatically without manual entry.

References

The following references document the academic literature, technical documentation, standards, and online resources consulted during the design, development, and evaluation of the Web-Based School Management System (SMS) with AI-Based Student Risk Prediction. References are organised into four categories: Books, Articles in Journals, Articles from Conference Proceedings, and World Wide Web resources. All citations follow the IEEE reference format adopted by the university for Final Year Design Project reports.

Book

Author(s). Book title. Location: Publishing company, year, pp.

- W.K. Chen. *Linear Networks and Systems*. Belmont, CA: Wadsworth, 1993, pp. 123–35.
- Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, 3rd ed. Sebastopol, CA: O'Reilly Media, 2022, pp. 1–850.
- R. C. Martin. *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall, 2009, pp. 1–431.
- Hunt and D. Thomas. *The Pragmatic Programmer: Your Journey to Mastery*, 2nd ed. Boston, MA: Addison-Wesley Professional, 2019, pp. 1–352.
- M. Fowler. *Patterns of Enterprise Application Architecture*. Boston, MA: Addison-Wesley, 2002, pp. 1–533.
- E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003, pp. 1–529.
- J. Date. *An Introduction to Database Systems*, 8th ed. Reading, MA: Addison-Wesley, 2003, pp. 1–983.

Article in a Journal

Author(s). “Article title.” *Journal title*, vol., pp., date.

1. G. Peveri. “Infrared Nation.” *The International Journal of Infrared Design*, vol. 33, pp. 56–99, Jan. 1979.
2. S. Amrieh, T. Hamtini, and I. Aljarah. “Mining Educational Data to Predict Student’s Academic Performance Using Ensemble Methods.” *International Journal of Database Theory and Application*, vol. 9, no. 8, pp. 119–136, Aug. 2016.

3. Z. Papamitsiou and A. A. Economides. "Learning Analytics and Educational Data Mining in Practice: A Systematic Literature Review of Empirical Evidence." *Journal of Educational Technology and Society*, vol. 17, no. 4, pp. 49–64, Oct. 2014.
4. T. Wang, C. Li, Y. Luo, and J. Liu. "Early Warning of At-Risk Students in Online Courses Using Machine Learning Algorithms." *Computers & Education*, vol. 190, pp. 1–16, Jan. 2023.
5. C. Romero and S. Ventura. "Educational Data Mining: A Review of the State of the Art." *IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews)*, vol. 40, no. 6, pp. 601–618, Nov. 2010.
6. M. Hussain, W. Zhu, W. Zhang, and S. M. R. Abidi. "Student Engagement Predictions in an e-Learning System and Their Impact on Student Course Assessment Scores Using Clickstream Analytics." *Computers in Human Behavior*, vol. 98, pp. 1–13, Sep. 2019.
7. D. Delen. "A Comparative Analysis of Machine Learning Techniques for Student Retention Management." *Decision Support Systems*, vol. 49, no. 4, pp. 498–506, Nov. 2010.
8. K. F. Adhatrao, A. Gaykar, A. Dhawan, R. Jha, and V. Honrao. "Predicting Students' Performance Using ID3 and C4.5 Classification Algorithms." *International Journal of Data Mining & Knowledge Management Process*, vol. 3, no. 5, pp. 39–52, Sep. 2013.

Articles from Conference Proceedings (published)

Author(s). "Article title." *Conference proceedings*, year, pp.

1. D.B. Payne and H.G. Gunhold. "Digital sundials and broadband technology," in *Proc. IOOC-ECOC*, 1986, pp. 557–998.
2. M. Al-Barrak and M. Al-Razgan. "Predicting Students Final GPA Using Decision Trees: A Case Study," in *Proc. International Conference on Information Technology (ICIT)*, Amman, Jordan, 2016, pp. 1–6.
3. R. Baker and K. Yacef. "The State of Educational Data Mining in 2009: A Review and Future Visions," in *Proc. 2nd International Conference on Educational Data Mining (EDM)*, Cordoba, Spain, 2009, pp. 3–17.
4. K. Fernandes, P. Vinagre, and P. Cortez. "A Proactive Intelligent Decision Support System for Predicting the Popularity of Online News," in *Proc. Portuguese Conference on Artificial Intelligence (EPIA)*, Coimbra, Portugal, 2015, pp. 535–546.
5. W. Jiang, Z. Williams, and N. Islam. "Using Machine Learning Algorithms to Predict Dropout from Online Courses," in *Proc. International Conference on Machine Learning and Applications (ICMLA)*, Boca Raton, FL, USA, 2018, pp. 756–762.
1. Yadav and C. Pal. "Prediction of Engineering Students Performance Using Classification Data Mining Techniques," in *Proc. International Conference on Science, Engineering and Management Research (ICSEMR)*, Chennai, India, 2014, pp. 1–4.

6. X. Chen and Z. Xu. “Educational Data Mining for Student Performance Prediction in Blended Learning,” in *Proc. International Conference on Intelligent Education and Educational Research (ICIEER)*, Wuhan, China, 2021, pp. 22–30.

World Wide Web

Author(s)*. “Title.” Internet: complete URL, date updated* [date accessed].

1. M. Duncan. “Engineering Concepts on Ice. Internet: www.iceengg.edu/staff.html, Oct. 25, 2000 [Nov. 29, 2003].
2. Microsoft Corporation. “ML.NET Documentation — Get Started with Machine Learning for .NET.” Internet: <https://learn.microsoft.com/en-us/dotnet/machine-learning/>, updated Jan. 2024 [Accessed: Apr. 20, 2025].
3. Microsoft Corporation. “ASP.NET Core Documentation — Build Web APIs with ASP.NET Core.” Internet: <https://learn.microsoft.com/en-us/aspnet/core/>, updated Nov. 2023 [Accessed: Apr. 22, 2025].
4. Microsoft Corporation. “Entity Framework Core Documentation — Code-First Migrations and Data Access.” Internet: <https://learn.microsoft.com/en-us/ef/core/>, updated Feb. 2024 [Accessed: Apr. 28, 2025].
5. Meta Open Source. “React — The Library for Web and Native User Interfaces.” Internet: <https://react.dev>, updated Mar. 2023 [Accessed: Apr. 25, 2025].
6. Microsoft Corporation. “JUnit Authentication with JWT in ASP.NET Core — Security Documentation.” Internet: <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/jwt-authn>, updated Dec. 2023 [Accessed: Apr. 30, 2025].
7. Tailwind Labs. “Tailwind CSS — A Utility-First CSS Framework for Rapid UI Development.” Internet: <https://tailwindcss.com/docs>, updated Jan. 2024 [Accessed: May 1, 2025].
8. Fedena Technologies. “Fedena — School ERP and Management Software — Features Overview.” Internet: <https://www.fedena.com/features>, 2023 [Accessed: May 1, 2025].
9. Recharts Group. “Recharts — A Redefined Chart Library Built with React and D3.” Internet: <https://recharts.org/en-US/>, 2023 [Accessed: May 2, 2025].
10. The Open Source Initiative. “OpenSIS — Open Source Student Information System.” Internet: <https://www.opensis.com>, 2022 [Accessed: May 3, 2025].

Appendix A

Use case Description Template

Appendix-A Use Case Description (Fully Dressed Format)

Table 51 UC-1 User Login / Authentication

Use Case ID:	UC-1
Use Case Name:	User Login
Actors:	Admin, Teacher, Student, Parent (all system users)
Description:	A registered user logs into the School Management System by providing valid credentials. The system authenticates the user, issues a JWT token, and redirects the user to their role-specific dashboard.
Trigger:	The user navigates to the system login page and submits the login form.
Preconditions:	The user has a registered account in the system. The user account is active (IsActive = true). The ASP.NET Core API is running and reachable.
Postconditions:	A valid JWT token is issued and stored in the user's browser (localStorage). The user is redirected to the dashboard corresponding to their assigned role.
Normal Flow:	1. The user opens the system URL in a web browser. 2. The login page is displayed with email and password fields. 3. The user enters a valid registered email address and password. 4. The user clicks the Login button. 5. The frontend sends a POST request to /api/auth/login with the provided credentials. 6. The system validates the email and password against the database.

	7. The system generates and returns a signed JWT token containing the user ID and role. 8. The frontend stores the token and decodes the user role. 9. The user is redirected to their role-specific dashboard (Admin, Teacher, Student, or Parent).
Alternative Flows:	A1 — Invalid Credentials (branching from Step 6): If the email does not exist or the password is incorrect, the system returns HTTP 401 Unauthorized. The login page displays an error message. No token is issued. The user is prompted to re-enter credentials. A2 — Account Inactive (branching from Step 6): If the account exists but IsActive = false, the system returns HTTP 401 with an account-disabled message. The user is advised to contact the administrator.
Business Rules	Only users with a valid registered account and matching credentials may log in. JWT tokens expire after a configured duration (e.g., 24 hours). Expired tokens are rejected by all protected endpoints. Role-based route protection ensures each user can only access features permitted for their role.
Assumptions:	User accounts are created in advance by the system administrator. HTTPS is used in production to protect credentials in transit.

Table 52 UC-2 Mark Student Attendance

Use Case ID:	UC-2
Use Case Name:	Mark Student Attendance

Actors:	Teacher
Description:	An authenticated teacher marks the daily attendance for students, recording each student as Present, Absent, or On Leave for a selected date. The system prevents duplicate entries for the same student on the same date.
Trigger:	The teacher navigates to the Mark Attendance section from the Teacher Dashboard navigation menu.
Preconditions:	The teacher is authenticated with a valid JWT token. Student records exist in the database. No locked attendance record exists for the selected date.
Postconditions:	Attendance records are saved in the Attendances table for all students in the list. Each record is associated with the teacher's TeacherId and the selected date. The teacher's dashboard attendance count is updated.
Normal Flow:	1. The teacher logs in and selects Mark Attendance from the navigation menu. 2. The system displays a list of all students with today's date pre-selected. 3. The teacher selects the attendance date using the date picker. 4. For each student, the teacher clicks P (Present), A (Absent), or L (Leave). 5. The teacher optionally clicks Mark All Present to set all students as present at once. 6. The teacher clicks Save Attendance. 7. The frontend sends POST /api/attendance requests for each student with their status. 8. The system validates each entry, checks for duplicates, and saves new records.

	9. A success confirmation message is displayed. Existing records are indicated as saved.
Alternative Flows:	A1 — Duplicate Record Detected (branching from Step 8): If an attendance record already exists for a student on the selected date, the system returns HTTP 409 Conflict. The frontend displays the existing record ID and instructs the teacher to use the update function instead of resubmitting. A2 — No Students Found (branching from Step 2): If no student records exist in the database, the attendance list is empty and the teacher is informed that students must be registered before attendance can be marked.
Business Rules	Only one attendance record per student per date is permitted. Duplicate submissions are rejected with HTTP 409. Only teachers are authorised to mark attendance. Admin may view but not directly mark via the teacher interface. The date cannot be set to a future date (the date picker enforces a maximum of today).
Assumptions:	Students are pre-registered in the system by the administrator. The teacher has been assigned a TeacherId profile linked to their user account.

Table 53 UC-3 Enter Student Marks

Use Case ID:	UC-3
Use Case Name:	Enter Student Marks
Actors:	Teacher

Description:	An authenticated teacher enters examination marks for students for a selected subject and exam type. The system automatically calculates the percentage and assigns a letter grade (A+ to F) for each entry.
Trigger:	The teacher navigates to the Enter Marks section and selects a subject and exam type.
Preconditions:	The teacher is authenticated with a valid JWT token. Student records exist in the database. The exam type and subject are selected before entering marks.
Postconditions:	Marks records are saved in the Marks table with computed percentage and grade. The student's average marks are updated and reflected in the next AI risk prediction.
Normal Flow:	1. The teacher logs in and selects Enter Marks from the navigation menu. 2. The teacher selects the Exam type (e.g., Mid-Term) and Subject from the dropdown menus. 3. The system loads the student list with any existing marks for the selected combination. 4. The teacher enters the Marks Obtained for each student in the input field. 5. The system automatically calculates and displays the percentage and letter grade for each row. 6. The teacher reviews the calculated grades and clicks Save Marks. 7. The frontend sends POST /api/mark (or PUT /api/mark/{id} for updates) for each modified row. 8. The system validates that marks obtained do not exceed total marks and saves the records. 9. A success message is displayed. Saved rows display a 'saved' indicator.

Alternative Flows:	A1 — Marks Exceed Total (branching from Step 8): If marksObtained > totalMarks for any student, the system returns HTTP 400 Bad Request. The affected row is highlighted in red and the teacher is prompted to correct the value before saving. A2 — Update Existing Record (branching from Step 3): If marks already exist for the selected student, subject, and exam, the row is pre-populated with the saved values. Editing and saving sends a PUT request to update the existing record.
Business Rules	Marks obtained must be greater than or equal to zero and must not exceed total marks. Grade thresholds: 90%+ = A+, 80%+ = A, 70%+ = B+, 60%+ = B, 50%+ = C, below 50% = F. Percentage and grade are computed server-side and cannot be supplied by the client.
Assumptions:	Total marks default to 100 unless overridden by the teacher for a specific exam. Each student may have multiple marks records — one per subject per exam type.

Table 54 UC-4 AI-Based Student Risk Prediction

Use Case ID:	UC-4
Use Case Name:	AI-Based Student Risk Prediction
Actors:	Teacher, Admin
Description:	The system automatically classifies a student as Low, Medium, or High academic risk based on their attendance rate, average marks percentage, and behaviour score. The prediction is generated by a trained ML.NET SDCA classifier if the model is available, or by a

	rule-based engine as a fallback. This is the key feature of the School Management System.
Trigger:	A teacher or admin requests a risk prediction for a specific student by selecting the student in the AI Risk Prediction panel or calling the prediction endpoint.
Preconditions:	The teacher or admin is authenticated with a valid JWT token. The student has at least one attendance record and one marks record in the database. The AI module is deployed and the backend API is running.
Postconditions:	A risk level (Low, Medium, or High) is returned and displayed in the dashboard. The response includes the three contributing factors: attendanceRate, averageMarks, and behaviorScore. The response indicates whether the ML model or the rule-based engine was used (mlModelUsed flag).
Normal Flow:	1. The teacher navigates to the AI Risk Prediction section of the dashboard. 2. The teacher selects a student from the list or the system loads predictions for all students. 3. The frontend sends GET /api/ai/predict?studentId={id} to the backend. 4. The AIController calls RiskService.ComputeAsync(studentId) to retrieve live attendance rate, average marks, and behaviour score from the database. 5. The AIController checks whether the trained ML model (risk_model.zip) is loaded in ModelTrainer. 6. If the ML model is loaded, ModelTrainer.Predict() is called with the three computed features. 7. The ML.NET SDCA classifier returns a predicted risk level (Low, Medium, or High).

	8. The final risk level (ML prediction) is returned as a JSON response with all factors. 9. The frontend displays the risk badge (green/orange/red), factor indicators, and ML model status.
Alternative Flows:	A1 — ML Model Not Loaded (branching from Step 5): If risk_model.zip is not available or not yet loaded, the system falls back to the rule-based classification engine. The rule-based engine applies threshold logic: attendance < 50% or marks < 35% or negative remarks >= 5 = High; attendance < 75% or marks < 55% or negative remarks >= 3 = Medium; otherwise Low. The response includes mlModelUsed = false. A2 — Insufficient Student Data (branching from Step 4): If the student has no attendance or marks records, the system uses only the available data. Default values are applied for missing metrics (e.g., 0% attendance = High risk trigger). The response includes a note about incomplete data. A3 — Model Training Triggered (Admin only): The admin can call POST /api/ai/train to retrain the ML model with the latest database records. Upon completion, risk_model.zip is updated and subsequent predictions use the newly trained model.
Business Rules	Rule-based High risk thresholds: attendanceRate < 50% OR averageMarks < 35% OR negativeRemarkCount >= 5. Rule-based Medium risk thresholds: attendanceRate < 75% OR averageMarks < 55% OR negativeRemarkCount >= 3. The ML model prediction takes precedence over the rule-based result when both are available. Only Admin may trigger model retraining. Teachers may only view predictions.

	Risk predictions reflect live database data and are recalculated on every request — no caching is applied.
Assumptions:	The ML model is trained on at least 10 student records or the 18 pre-defined seed samples. Behaviour score is derived from the count of Negative behaviour remarks recorded by teachers.

Table 55 UC-5 Submit and Manage Complaint

Use Case ID:	UC-5
Use Case Name:	Submit and Manage Complaint
Actors:	Student / Parent (submitter), Teacher (resolver), Admin (supervisor)
Description:	A student or parent submits a complaint through the system. The complaint is assigned to a teacher for review and resolution. The teacher updates the complaint status through its lifecycle: Submitted → Under Review → Resolved → Closed.
Trigger:	A student or parent identifies an issue and submits a complaint through the Complaint section of their dashboard.
Preconditions:	The submitter is authenticated with a valid JWT token. A teacher or admin user exists in the system to whom the complaint can be assigned.
Postconditions:	The complaint is saved in the Complaints table with status = Submitted and a timestamp. The assigned teacher can view and update the complaint status. When the status is set to Resolved or Closed, a DateClosed timestamp is automatically recorded.

Normal Flow:	1. The student or parent logs in and navigates to the Complaints section. 2. The user clicks Submit New Complaint. 3. The user fills in the complaint category, description, and optionally selects an assigned teacher. 4. The user submits the form. The frontend sends POST /api/complaint to the backend. 5. The system saves the complaint with status = Submitted and dateSubmitted = current timestamp. 6. The assigned teacher receives the complaint in their Complaint Management panel. 7. The teacher reviews the complaint and clicks Update Status. 8. The teacher selects a new status (Under Review, Resolved, or Closed) and optionally adds remarks. 9. The frontend sends PATCH /api/complaint/{id}/status. The system updates the record. 10. If status is Resolved or Closed, the system sets dateClosed automatically.
Alternative Flows:	A1 — Teacher Updates Status Without Authorisation (branching from Step 8): If the teacher is not the assigned user for the complaint, the system returns HTTP 403 Forbidden. The update is rejected. A2 — Complaint Not Found (branching from Step 9): If the complaintId does not exist in the database, the system returns HTTP 404 Not Found.
Business Rules	A teacher may only update complaints that are explicitly assigned to their user account. Admins have unrestricted access to view and update all complaints regardless of assignment.

	DateClosed is set server-side when status reaches Resolved or Closed. Clients cannot supply this value. A complaint cannot transition backwards in the lifecycle (e.g., from Resolved back to Submitted).
Assumptions:	The complaint category list is predefined (e.g., Academic, Bullying, Facility, Behavioral). All parties are registered users with active accounts.

Table 56 UC-6 Post Announcement / Notice

Use Case ID:	UC-6
Use Case Name:	Post Announcement / Notice
Actors:	Teacher, Admin
Description:	An authenticated teacher or admin creates and publishes an announcement or notice for a specified audience (school-wide, students only, parents only, or a specific class). The notice is immediately visible to the target audience on their dashboard.
Trigger:	The teacher or admin clicks New Announcement in the Announcements section of the dashboard.
Preconditions:	The user is authenticated with a valid JWT token and has Teacher or Admin role. The notice title and content are provided and non-empty.
Postconditions:	The notice is saved in the Notices table with isActive = true and postedAt = current timestamp. The notice is visible to all users matching the specified audience.
Normal Flow:	1. The teacher or admin navigates to the Announcements section. 2. The user clicks New Announcement to open the notice form.

	3. The user enters the notice title, content, audience, type (Academic, Event, Holiday, Exam, General), and priority (Low, Medium, High, Urgent). 4. If audience is Class Specific, the user enters the target class identifier. 5. The user clicks Post Announcement. 6. The frontend sends POST /api/notice with the form data. 7. The system sets postedByUserId = current user, postedAt = now, isActive = true, and saves the record. 8. The notice appears in the active notices list immediately.
Alternative Flows:	A1 — Target Class Missing for Class Specific Audience (branching from Step 6): If audience = ClassSpecific but targetClass is empty, the system returns HTTP 400 Bad Request. The teacher is prompted to provide the class identifier. A2 — Deactivate Notice (extension of normal flow): The teacher clicks Deactivate on an active notice. The frontend sends PATCH /api/notice/{id}/deactivate. The system sets isActive = false. Teachers may only deactivate their own notices; admins may deactivate any notice.
Business Rules	A teacher may only deactivate notices they personally posted. Admins may deactivate any notice. The postedAt timestamp is set server-side and cannot be supplied by the client. Inactive notices remain in the database for audit purposes but are not shown to students or parents.
Assumptions:	Audience filtering is applied at the frontend display layer based on the audience field.

	All active notices are returned by GET /api/notice regardless of audience for admin review.
--	---

Table 57 UC-7 Record Student Achievement or Certificate

Use Case ID:	UC-7
Use Case Name:	Record Student Achievement or Certificate
Actors:	Teacher
Description:	An authenticated teacher records an academic or extracurricular achievement for a student. An optional certificate link (URL or file path) can be attached. The achievement is linked to the student's profile and visible in the student's dashboard.
Trigger:	The teacher clicks Add Achievement in the Achievements section of the dashboard.
Preconditions:	The teacher is authenticated with a valid JWT token. The student record exists in the database. An achievement title and category are provided.
Postconditions:	The achievement is saved in the Achievements table linked to the student's StudentId. If a filePath is provided, it is stored and rendered as a clickable certificate link.
Normal Flow:	1. The teacher navigates to the Achievements section and clicks Add Achievement. 2. The form modal opens with fields for Student, Title, Category, Date, and optional Certificate Link. 3. The teacher selects the student from the dropdown and fills in the achievement details.

	4. The teacher optionally pastes a URL or file path to a certificate document. 5. The teacher clicks Save Achievement. 6. The frontend sends POST /api/achievement with the form data. 7. The system validates the date (must not be in the future) and saves the record. 8. The new achievement appears in the achievements grid with the correct category badge.
Alternative Flows:	A1 — Future Date Provided (branching from Step 7): If the achievement date is set to a future date, the system returns HTTP 400 Bad Request with a validation error. The teacher is prompted to correct the date. A2 — Delete Achievement (extension): The teacher clicks the delete icon on an existing achievement. A confirmation dialog is shown. On confirmation, the frontend sends DELETE /api/achievement/{id}. The system removes the record and the achievement card disappears from the grid.
Business Rules	Achievement date must not be in the future. The filePath field is optional. If provided, it is stored as-is and rendered as a hyperlink. Only Admin may delete achievements. Teachers can add but not delete via the teacher interface.
Assumptions:	Certificate documents are hosted externally (e.g., Google Drive). The system stores only the URL reference. A student may have multiple achievement records across different categories and dates.

Table 58 UC-8 Add Behaviour Remark

Use Case ID:	UC-8
Use Case Name:	Add Behaviour Remark
Actors:	Teacher
Description:	An authenticated teacher records a behaviour observation for a student, classifying it as Positive, Negative, or Neutral. The remark is stored with a timestamp and contributes to the student's behaviour score used in the AI Risk Prediction model.
Trigger:	The teacher clicks Add Remark in the Behaviour Remarks section of the Teacher Dashboard.
Preconditions:	The teacher is authenticated with a valid JWT token. The student record exists in the database. The remark text is at least 10 characters in length.
Postconditions:	The remark is saved in the BehaviorRemarks table linked to both StudentId and TeacherId. The student's behaviour score (used in AI risk computation) is updated on the next prediction call.
Normal Flow:	1. The teacher navigates to the Behaviour Remarks section and clicks Add Remark. 2. The form modal opens with fields for Student, Remark Type, and Remark Text. 3. The teacher selects the student from the dropdown. 4. The teacher selects the remark type: Positive, Negative, or Neutral. 5. The teacher optionally clicks a quick-template suggestion to auto-fill the remark text. 6. The teacher reviews and edits the remark text as needed.

	7. The teacher clicks Save Remark. 8. The frontend sends POST /api/behaviorremark with studentId, teacherId, remarkType, and remarkText. 9. The system validates the input and saves the record with the current timestamp. 10. The new remark appears in the remarks list with the appropriate colour-coded badge.
Alternative Flows:	A1 — Remark Too Short (branching from Step 9): If the remark text is fewer than 10 characters, the system returns a validation error. The teacher is prompted to provide a more descriptive remark.
Business Rules	Each Negative remark increases the student's negative remark count, which feeds into the AI risk engine's behaviour score calculation. A minimum remark length of 10 characters is enforced to ensure meaningful observations. Remarks are read-only after submission. Only Admins may delete remarks.
Assumptions:	The behaviour score used in AI predictions is computed as: $100 - (\text{negativeRemarkCount} \times 10)$, capped at 0 at the lower end. Teachers record remarks at their own discretion based on daily observations.

Appendix-B

Coding Standards

Appendix-B General Coding Standards & Guidelines

1. Follow a consistent variable naming convention throughout the code. E.g.
1. Snakecase (words are delimited by "_" like: variable_one)
2. Pascalcase (words are delimited by capital letters like: VariableOne)
3. Camelcase (words are delimited by capital letters except the initial word like: variableOne)
4. Hungarian Notation (describes the variable type or purpose at the start of the variable name like: arrDistributeGroup)
5. Use naming that visually describes scope like privateField, Const etc
6. Use read only/immutable when a field's value should not be changed after initialization
7. Use only get, for properties that should not be updated from outside
8. Name functions according to their functionalities.
9. Insert appropriate comments to make the code understandable to any reader. Additionally follow a consistent style to do so. E.g.

```
/* the below function will be used for the addition of two variables*/  
int Add(){  
    //logic of the function  
}
```

Avoid commenting on obvious things

10. Make use of indentation for indicating the start and end of the control structures along with a clear specification of where the code is between them.
11. Follow consistent naming convention for files and folders.
12. Follow proper structure for classes
13. Group code entities logically into projects/packages/modules/folders
1. Separate logical layers of application into different modules/services/utilities etc.
2. User separate files for each class, struct, interface, enum etc. Name of the file and the enclosing entity must be same. E.g., class Employee in Employee.cs/Employee.java
3. Define and use everything within the minimum scope possible
4. Use proper access modifier for all code entities if required
5. Code entities should have maximum cohesion and least coupling possible.
6. Follow DRY law.
7. Do not repeat code.
8. A piece of knowledge should exist only in one place within the codebase/application
9. Reuse code as much as possible
10. Always write short methods
11. Single method should not have too many logic, long conditional flow or too many parameters

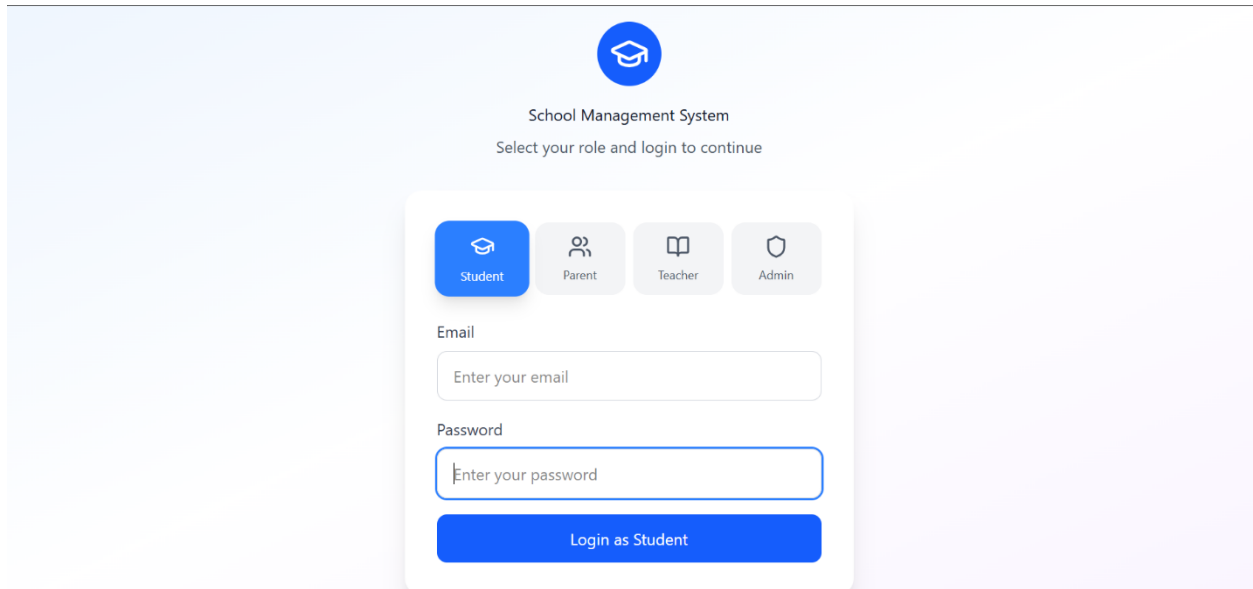
12. Strictly follow Single Responsibility Principle (SRP) when writing methods, classes, modules, projects, packages, or any other code entities.
13. Write classes and other code entities that are easy to extend without modification.
14. Handle exceptions
15. Log exception and other significant event details
16. Follow a consistent convention for logging all over the application

Appendix C

Prototype

Appendix-C Application Prototype

Login Page



The login page features a central white card on a light blue background. At the top of the card is a blue circular icon with a white graduation cap. Below the icon, the text "School Management System" is displayed, followed by the instruction "Select your role and login to continue". The card contains four role-based login buttons: "Student" (blue with a graduation cap icon), "Parent" (light blue with a family icon), "Teacher" (light blue with a book icon), and "Admin" (light blue with a shield icon). Below these buttons are two input fields: "Email" with the placeholder "Enter your email" and "Password" with the placeholder "Enter your password". A blue "Login as Student" button is positioned at the bottom of the card.

Figure 26 Login page

Description: This screen allows users to securely log in to the system using their registered email and password. Role-based authentication ensures that users are redirected to their respective dashboards based on their assigned roles (Admin, Teacher, Student, or Parent).

Admin Dashboard

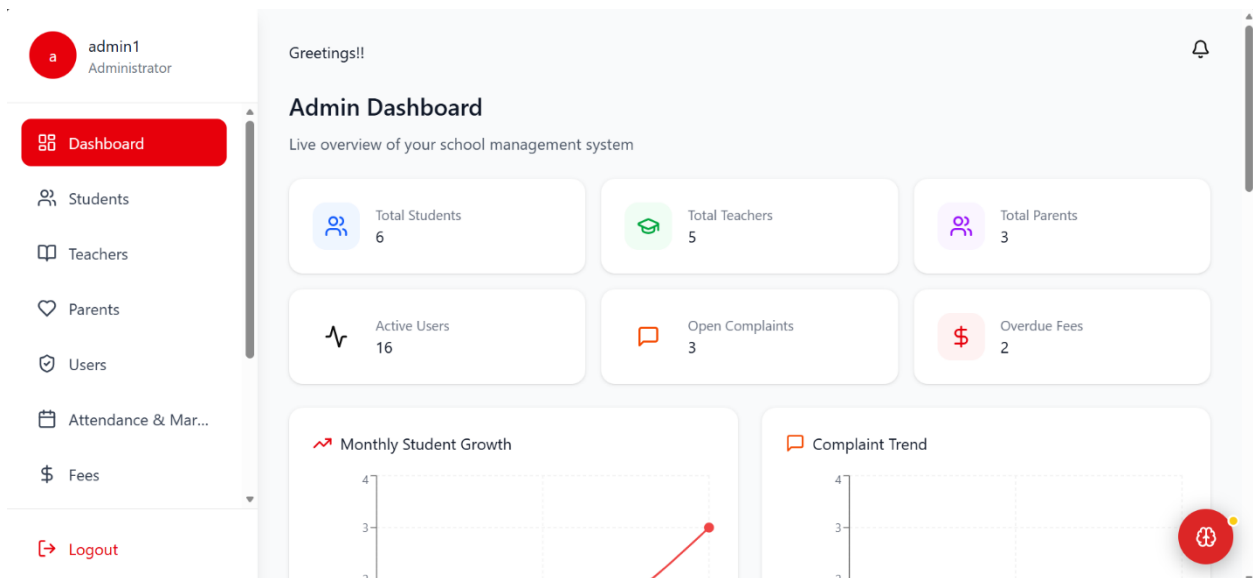


Figure 27 Admin Dashboard

Description: The Admin Dashboard provides a centralized overview of system activities, including user management, announcements, and overall system control. It enables administrators to monitor and manage key operations efficiently from a single interface.

Attendance and Marks Management Module

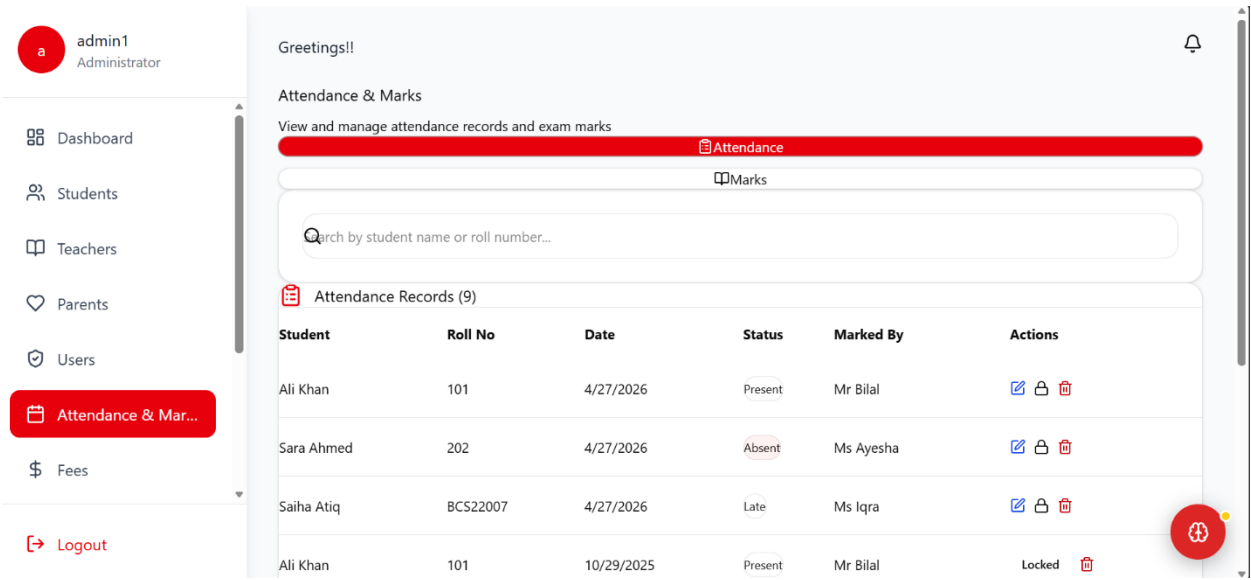


Figure 28 Attendance and marks management screen

Description: This screen provides a combined overview of student attendance and marks within the Admin Dashboard, enabling centralized monitoring of academic records and ensuring data accuracy.

Complaint Management Module

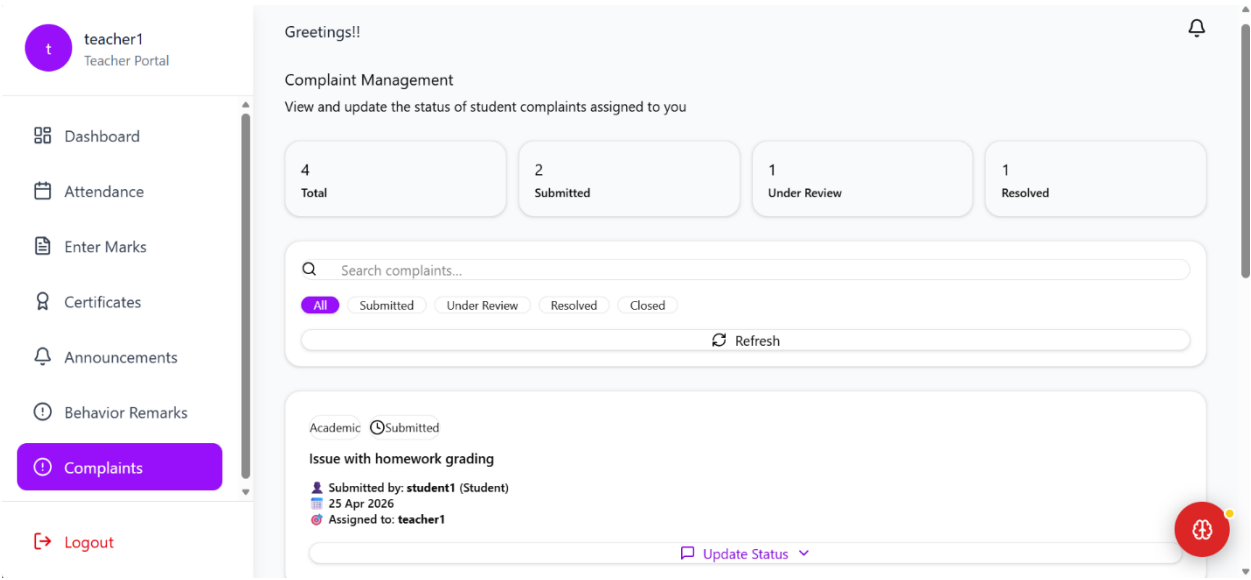


Figure 29 Complaint management screen

Description: This module allows users to submit, track, and manage complaints. Each complaint is recorded with a status and can be updated by the assigned authority, ensuring a structured resolution process.

Noticeboard System

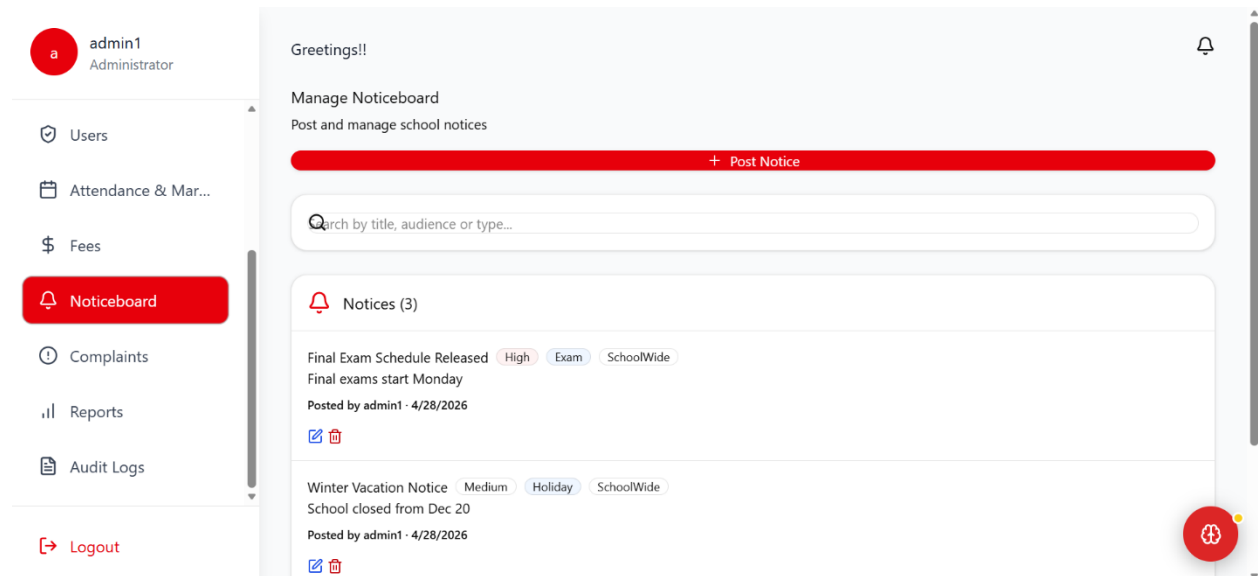


Figure 30 Noticeboard system screen

Description: The Noticeboard enables teachers and administrators to post announcements for students and parents. It ensures timely and consistent communication of important academic and administrative updates.

AI Based Risk Prediction Module

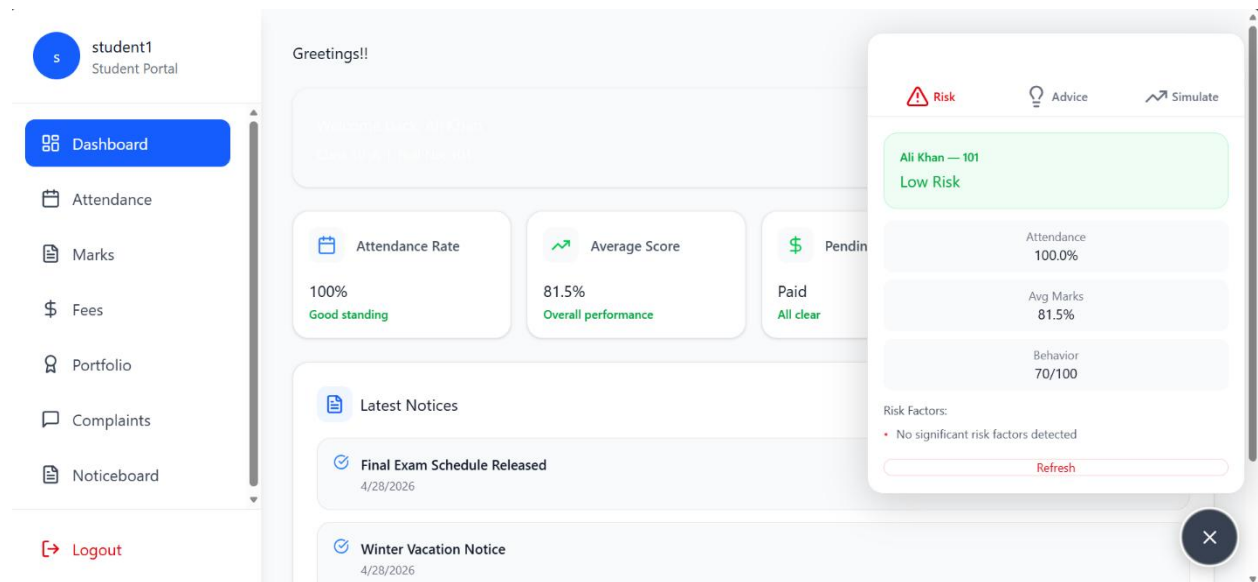


Figure 31 AI risk prediction screen

Description: This module analyzes student data, including attendance, marks, and behavior, to predict academic risk levels. It provides insights such as risk category and contributing factors, supporting early intervention and informed decision-making.

Marks Entry Module

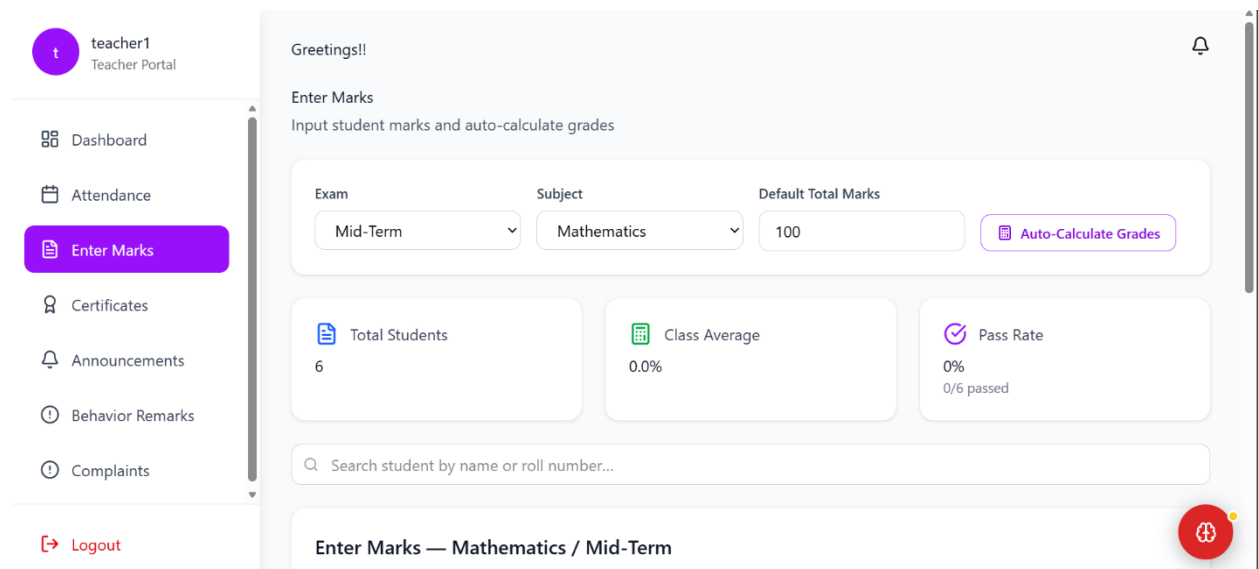


Figure 32 Marks entry module screen

Description: This screen enables teachers to input and manage student marks for different subjects and examinations. The system automatically calculates percentages and assigns grades based on predefined criteria.

Student Portfolio

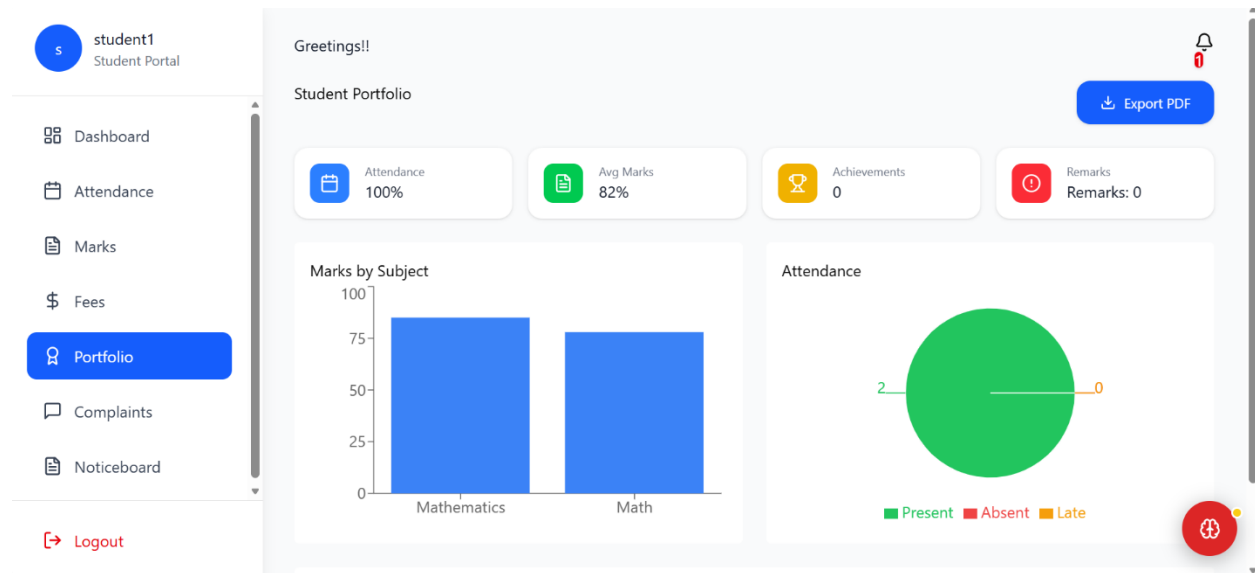


Figure 33 student portfolio screen

Description: The Student Portfolio provides a consolidated view of a student's academic performance, attendance records, achievements, and behavioral history. It enhances transparency for both students and parents.